Version: 1.0 · **Status:** Draft for team approval · **Date:** November 7, 2026

Delivery: AWS × A10 hackathon, November 7–8, 2026

Owners: Masha — product and QA; Tishneet — AI and workflow; Mariam — security and inventory integrity; Ishani — user experience and public interface.

SentinelOS helps an operator turn a natural-language inventory request into an authorized, simulated, and traceable action. The first release must demonstrate that an individually permitted action can still produce an invalid warehouse outcome, and that the system prevents that outcome before changing inventory.

Requirements below preserve the integration contract's product rules. Items marked **Proposed** are additions for team review. All targets are release goals, not measured results. Approval remains pending.

1 Executive summary

SentinelOS is a controlled execution product for AI-generated warehouse plans. An operator describes a desired package movement; the product generates a candidate, checks each action's permissions, and simulates the complete ordered plan. A valid low-risk plan executes automatically. A valid elevated-risk plan requires an administrator's approval. An unsafe plan is blocked, and the AI may propose an alternative only when the operator's goal permits it.

The hackathon release covers one demonstration warehouse, three inventory zones, one inventory agent, and one operation: moving packages. Success means completing the three agreed demonstrations with exact inventory results, showing why rejected plans were blocked, and proving that stale approvals, revoked permissions, and repeated requests cannot produce unauthorized or duplicate changes.

The main value proposition is **safe delegation with visible evidence**: operators can see the proposal, the permission decision, the projected effect, the approval decision when required, and the actual committed result.

2 Problem statement and opportunity

An AI can produce an action that is permitted by policy but invalid in the current operating state. For example, moving five packages into Zone B meets the agent's per-action transfer limit, but exceeds B's remaining capacity. Several individually valid actions can also become invalid when evaluated together or in sequence.

Operators therefore need more than a plausible AI answer. They need confidence that the full plan respects inventory constraints, preserves their intent, remains authorized at execution time, and leaves an explainable record.

The initial opportunity is to demonstrate these controls in a bounded warehouse workflow. Broader demand, willingness to pay, and measurable labor savings remain hypotheses for later validation.

Positioning hypothesis: SentinelOS combines action-level authorization, whole-plan simulation, human approval, and execution traceability in one operator experience. Competitive superiority and market size are not established by this release.

3 Target users and core journeys

Persona	Primary need	Successful outcome
Inventory operator	Delegate a package movement without manually checking every consequence	The requested goal is completed safely or receives a clear, actionable rejection
Operations administrator	Control elevated-risk actions and revoke an agent's authority	Only current, valid, explicitly approved plans execute when approval is required
Product reviewer or hackathon judge	Understand and verify the safety claims	Can trace a real request through decisions to exact before-and-after inventory
QA owner	Reproduce normal and failure scenarios	Can reset safely, run known cases, and inspect the resulting evidence

The inventory agent is the acting system identity; it is distinct from the human operator and administrator.

Journey A — Routine transfer: The operator submits “Move three packages from Zone A to Zone C.” SentinelOS validates, authorizes, and simulates the plan, determines that approval is unnecessary, rechecks current conditions, and commits the transfer.

Journey B — Unsafe preference with an allowed alternative: The operator requests moving five packages out of A, preferably to B, while permitting another destination. SentinelOS blocks a capacity-violating candidate, explains the violation, proposes a valid alternative, and obtains approval before execution.

Journey C — Revocation: An administrator revokes the agent. A subsequent request, or a previously prepared plan that has not yet executed, is blocked at the next authorization check.

Journey D — State changes while waiting: Another successful plan changes inventory while approval is pending. The old approval cannot authorize execution against the new state; the operator receives a clear re-evaluation outcome.

4 Product vision and goals

Vision: Make AI-assisted operational changes inspectable and controllable before they affect real state.

Goal	Required product behavior	Evidence
Prevent invalid inventory changes	Evaluate every intermediate step of the entire plan before commit	Safety test results and rejected counterfactuals
Preserve operator intent	Retain hard constraints when proposing alternatives	Original request and each candidate visible together
Keep authority current	Recheck permissions and approval validity immediately before execution	Revocation and stale-approval tests
Balance autonomy with oversight	Auto-execute valid low-risk plans; require approval for valid elevated-risk plans	Demonstrations on both sides of the risk threshold

Make outcomes explainable	Show decisions, reasons, revisions, and actual inventory changes	Complete per-plan history and recent activity
Deliver a reproducible demo	Use fixed seed data and a safe reset	Repeatable expected-state walkthrough

Commercial outcomes for later discovery: time saved per inventory request, operator trust, adoption frequency, and willingness to pay. Baselines and targets require user research.

5 Release scope and product rules

Scope

P0 — Required for the complete hackathon release: request submission; AI planning; independent validation; per-action authorization; sequential simulation; bounded replanning; deterministic risk classification; administrator approval; execution-time rechecks; safe inventory commit; revocation; audit history; status dashboard; controlled reset.

P1 — Add after P0 works: a clearly labeled preflight test preset to reliably show a capacity rejection, clearer visual before-and-after comparisons, and presentation polish.

Future scope: multiple warehouses, multiple acting agents, additional operation types, configurable business policies, enterprise administration, and production reliability commitments.

The MVP operates on aggregate package counts. SKU-level tracking, physical robot control, purchasing, shipment routing, and automatic policy changes are outside this release.

Fixed demonstration model

Zone	Starting inventory	Capacity
A	20	30

B	8	10
C	5	20

The starting warehouse version is **1** and total inventory is **33 packages**. HAZMAT is a reserved, restricted destination, not a normal inventory zone.

The acting agent is **inventory-agent-01**, initially ACTIVE, with the INVENTORY_MANAGER role and a maximum of **10 packages per action**.

Binding business rules

- A candidate contains 1–10 ordered package-movement actions.
- Each action identifies a distinct source and destination and a positive integer quantity. Zero, negative, fractional, and boolean quantities are invalid.
- A quantity above 10 is structurally valid but is denied by policy. The 10-package ceiling applies to each action; the source does not define a 10-package ceiling for the whole plan.
- Every action must be authorized. Any denied action rejects the entire candidate and request; the AI must not replan to evade that denial.
- Every intermediate state must have nonnegative inventory and remain within each zone's capacity. Total inventory must remain 33 throughout these transfer-only demonstrations.
- The maximum is **three AI candidates per request, including the initial candidate**.
- Risk uses the sum of all action quantities, not net inventory movement or unique packages: totals 1–4 are LOW; totals of 5 or more are ELEVATED.
- Only a plan that has already passed authorization and simulation can request approval.
- Approval applies to one exact candidate revision and warehouse state. A plan revision or inventory-state change invalidates it.
- Failed checks, rejected or expired approvals, and unsuccessful commits must leave actual inventory unchanged.

6 Detailed functional requirements

All requirements in this section are P0 unless marked P1. Acceptance criteria define the required behavior; completion must be demonstrated through QA.

FR01 Submit an inventory goal

Owner: Ishani; **support:** Tishneet.

The operator must be able to enter a natural-language movement request and receive a trackable plan reference.

Acceptance criteria: Blank or unusable input returns a clear error without changing inventory. A valid submission displays the original instruction and planning status. The product determines the acting agent from trusted identity; the operator cannot substitute another agent or supply approval or execution authority.

FR02 Generate and independently validate a candidate

Owner: Tishneet; **reviewer:** Mariam.

AI output must be checked independently before it can proceed to authorization.

Acceptance criteria: Only the supported movement operation and its source, destination, and quantity are accepted; unexpected action attributes are rejected. A candidate with no actions, more than 10 actions, identical source and destination, invalid zone-label form, or an invalid quantity cannot proceed. AI-generated claims about identity, authorization, approval, or execution have no authority. Validated actions remain unchanged within that revision.

Proposed failure disposition: Invalid AI output ends the request as FAILED with an explanation. Automatic repair of malformed output requires an explicit team decision and must count toward the three-candidate maximum.

FR03 Authorize every action

Owner: Mariam.

Each action must be evaluated using current trusted agent status and the applicable policy.

Acceptance criteria: An ACTIVE agent may move up to 10 packages per action to an unrestricted destination. REVOKED agents, SUSPENDED agents if supported, quantities above 10, and HAZMAT destinations are denied. Missing authorization data or an authorization-service error denies execution. One denial blocks the whole plan, produces an action-specific reason, and does not trigger a bypass-oriented replan.

FR04 Simulate the complete ordered plan

Owner: Mariam.

The product must calculate the consequences of the full sequence without changing actual inventory.

Acceptance criteria: Each step uses the preceding step's projected state. Unknown zones, insufficient source inventory, restricted destinations, and capacity violations are identified. Any intermediate violation fails the candidate, even if a later action would restore a valid final state. Failure details identify the action, zone, rule, and projected versus permitted quantity where applicable. Counterfactual inventory is visibly distinguished from committed inventory.

FR05 Replan within the operator's intent

Owner: Tishneet; **reviewer:** Masha.

After a simulation failure, the AI may propose a revised candidate when an alternative is compatible with the original request.

Acceptance criteria: The new candidate uses the original instruction and failure details, receives a new revision, and passes validation, authorization, and simulation again. All candidates remain inspectable. No fourth candidate is generated. A request that explicitly requires destination B cannot silently redirect packages to C. If no permitted alternative exists or the candidate limit is reached, the workflow stops with a reason and no inventory change.

FR06 Classify plan risk

Owner: Tishneet.

Risk must be determined consistently from the complete valid candidate.

Acceptance criteria: A total of 4 moved packages requires no approval; a total of 5 requires approval. A split of 2 packages to B and 3 to C requires approval. Moving the same packages in successive actions counts every movement. Failed authorization or simulation never produces an approval-eligible plan.

FR07 Obtain an administrator decision

Owners: Tishneet and Mariam; **UI:** Ishani.

An authorized administrator must be able to approve or deny a valid elevated-risk candidate.

Acceptance criteria: The decision view includes the original instruction, exact revision, actions, relevant warehouse version, projected inventory, risk, and validation results. The product records decision identity and time. Denial prevents execution. No decision before the configured deadline causes expiry and prevents execution. Repeated decisions cannot trigger multiple executions. Only an authorized administrator can decide.

Proposed timeout: 10 minutes for the demo; confirm before approval-flow acceptance.

FR08 Invalidate stale approval and recheck execution eligibility

Owners: Mariam and Tishneet.

Immediately before execution, SentinelOS must reauthorize the agent, confirm the approved candidate and relevant state are unchanged, and resimulate against current inventory.

Acceptance criteria: Revocation after planning or approval blocks execution. A revised candidate requires its own checks and approval when elevated risk applies. A warehouse-state change prevents use of the old approval even if the plan would still fit. A change between the final checks and commit also prevents a stale write. No previous approval is carried forward to a new state.

Proposed MVP recovery: End a conflicting request as FAILED with a VERSION_CONFLICT reason and offer a fresh evaluation. Automatic recovery remains an open decision.

FR09 Apply inventory changes once and completely

Owner: Mariam.

Only the controlled workflow may commit a validated plan.

Acceptance criteria: All transfers in a successful plan take effect together, with one warehouse-version increment. Failure applies none of them. Repeated execution of the same plan revision returns its prior outcome or “already executed” and changes neither inventory nor version again. The browser cannot invoke an unrestricted execution operation.

FR10 Revoke an agent

Owner: Mariam; **UI:** Ishani.

An authorized administrator must be able to revoke the current inventory agent and see the resulting status.

Acceptance criteria: A subsequent authorization check denies the revoked agent. Pending, unexecuted plans are subject to the same current-status check. Revocation does not reverse already completed transfers. Ordinary operators cannot revoke agents.

FR11 Display progress and committed outcomes

Owner: Ishani; **workflow support:** Tishneet.

The operator must be able to follow a request through planning, checks, replanning, approval, execution, and its terminal outcome.

Acceptance criteria: Progress updates approximately every two seconds while active. The UI shows the current attempt and relevant decision details. Candidate projections appear separately from actual inventory. After completion, the warehouse display refreshes to the committed result. Denied, failed, and expired plans remain available with explanations. A connection error does not falsely display success.

FR12 Provide an audit history

Owner: Mariam; **workflow support:** Tishneet; **UI:** Ishani.

Reviewers must be able to inspect the decisions and state changes associated with a plan, as well as recent events across plans.

Acceptance criteria: Events identify the plan, candidate revision, timestamp, event type, outcome, and reason; approval events identify the deciding administrator. The history covers candidate generation, validation, authorization, simulation, replanning, approval, execution checks, and final outcome. Events can be ordered consistently. Secrets and credentials are excluded from operator-visible history.

FR13 Reset the demonstration safely

Owner: Mariam; **QA:** Masha.

An authorized administrator must be able to restore the agreed seed warehouse and demonstration agent for a new walkthrough.

Acceptance criteria: Reset is blocked while any workflow is active or awaiting approval. A completed reset restores A=20, B=8, C=5, version=1, and the demo agent to ACTIVE. Old approvals and prior plan references cannot cause execution against the reset state.

Proposed reset-history rule: Retain the previous run’s evidence and clearly separate it from the new run. Resetting the agent and handling prior runs require confirmation in the decision register.

FR14 Provide a labeled preflight example

Priority: P1; **owners:** Masha and Ishani; **support:** Tishneet and Mariam.

A reviewer may run a preset that demonstrates the invalid A-to-B transfer even when the AI initially proposes a valid split.

Acceptance criteria: The UI labels the initial candidate as a scripted preflight test. Its failed simulation does not change inventory. Any subsequent AI replan uses the real model or is explicitly labeled otherwise. The demonstration never presents the scripted candidate as an unexpected autonomous model mistake.

7 User experience requirements

The primary experience must make four things easy to distinguish: **the user's goal, the current candidate, its projected outcome, and the actual warehouse state**.

Surface	Required information and actions
Request entry	Instruction field, submit action, validation feedback, acting agent status
Warehouse overview	Inventory and capacity for A, B, and C; current version; committed-state label
Plan detail	Original instruction, revision and attempt, ordered actions, per-action decisions, simulation violations
Approval view	Exact plan and state under review, risk and total movement, administrator approve and deny actions
Activity history	Chronological outcomes and reasons for the selected plan; recent cross-plan events
Administration	Agent revocation and safe reset, available only to authorized administrators

Errors must explain what happened and what the user can do next. For example: "Zone B would contain 13 packages, above its capacity of 10" or "Inventory changed after this plan was reviewed. Evaluate it again."

Workflow vocabulary: PLANNING, VALIDATING, AUTHORIZING, SIMULATING, REPLANNING, AWAITING_APPROVAL, REAUTHORIZING, REVALIDATING, EXECUTING, COMPLETED, REJECTED, FAILED, and EXPIRED.

Terminal outcome	Operator meaning
COMPLETED	Inventory changes were committed successfully
REJECTED	Policy or an administrator denied the requested execution
FAILED	Planning, validation, simulation, execution checks, or a service failure prevented completion
EXPIRED	Required approval was not received within its allowed window

Specific failure-to-status mappings beyond the source's explicit rules are proposed and must be frozen at kickoff.

Proposed usability acceptance: Core controls work with a keyboard; statuses use text as well as color; the demo layout is readable on the presentation laptop; expired or invalidated approvals cannot still appear actionable.

8 Nonfunctional requirements

ID and owner	Requirement	Acceptance evidence
NFR01 Security — Mariam	Enforce operator versus administrator permissions; keep credentials and approval-control secrets out of the client; prevent AI output from granting authority	Unauthorized approval, revocation, reset, identity-substitution, and direct-execution attempts cause no state change

NFR02 Integrity — Mariam	Preserve total inventory, capacity limits, nonnegative counts, version correctness, and all-or-nothing execution	Sequential, stale-state, duplicate-execution, and competing-plan tests pass
NFR03 Durability — Tishneet and Mariam	Keep pending approvals, candidate identity, and prior execution outcomes available across refreshes and service restarts	Resume a pending plan and replay a completed request without losing binding or duplicating movement
NFR04 Failure handling — All engineers	Fail closed when authorization or required state is unavailable; expose a recoverable explanation where possible	Dependency-failure cases show no false success and no unsafe inventory change
NFR05 Responsiveness — Tishneet and Ishani	Refresh active status about every 2 seconds. Proposed: normal, single-candidate evaluation reaches a decision within 30 seconds at the 95th percentile, excluding human wait	Timestamped demo-environment measurements; proposed latency target requires team confirmation
NFR06 Supported scale — Mariam and Tishneet	Support one warehouse, one agent, up to 10 actions per candidate, and three candidates per request; competing plans cannot corrupt shared state	Maximum-size plan and two-competing-plan tests; no production throughput claim
NFR07 Traceability — Mariam and Masha	Make plan outcomes and their reasons retrievable without exposing secrets	QA can reconstruct each judged scenario and failed execution attempt

NFR08 Operational readiness — All engineers	Provide a reproducible startup, reset, known deployed version, and documented limitations	Another team member can run the full walkthrough from the recorded release

The demo uses synthetic inventory. Applicable production compliance obligations, retention periods, service availability commitments, and customer data requirements remain future discovery items. No certification or production SLA is claimed.

9 Success metrics and KPIs

Metric	Definition	Release target	Owner
Core demo pass rate	Correctly completed scenarios divided by the three agreed scenarios	3 of 3, with exact final counts and versions	Masha
Unsafe commit count	Commits caused by failed policy, simulation, approval, or stale-state checks	0 in the defined QA suite	Mariam
Duplicate movement count	Extra inventory mutations after replaying a completed revision	0	Mariam
Approval enforcement	Elevated-risk executions with a valid current approval divided by all elevated-risk executions	100% in tested flows	Tishneet and Mariam
Revocation enforcement	Denied attempts after revocation divided by tested post-revocation attempts	100%	Mariam

Trace completeness	Test plans with all applicable decision and outcome evidence divided by all test plans	100%	Masha
Candidate bound	Maximum number of AI candidates generated for one request	No more than 3	Tishneet
Evaluation latency	Time from accepted request to the first eligible decision or failure, excluding administrator wait	Proposed p95 ≤30 seconds for a single candidate; record actual sample size	Tishneet
Presentation freshness	Delay between available workflow status and the next regular UI refresh	Approximately 2-second polling while active	Ishani

Proposed repeatability gate: Three consecutive complete walkthroughs pass after controlled resets.

Record actual AI candidate counts and request costs where measurable. Any cost estimate must be labeled as an estimate. Replanning success outside the controlled demo is an observation to collect, not a validated reliability claim.

10 Acceptance scenarios and QA

Judged walkthrough

Run these scenarios **in order on the same warehouse state**, starting at A=20, B=8, C=5, version=1.

Scenario	Request and expected behavior	Required actual outcome

D1 Low-risk transfer	“Move three packages from Zone A to Zone C.” Authorization and simulation pass; no approval required	A=17/30, B=8/10, C=8/20; version=2
D2 Capacity rejection and approved alternative	“Move five packages out of Zone A, preferably into Zone B. Use another zone if necessary.” A→B ×5 would project B=13/10 and must fail simulation. A→B ×2 plus A→C ×3 passes and requires approval	Before approval: unchanged from D1. After approval: A=12/30, B=10/10, C=11/20; version=3
D3 Revocation	Administrator revokes inventory-agent-01; a subsequent A→C ×1 request is denied	A=12/30, B=10/10, C=11/20; version remains 3

The AI may produce the valid D2 split immediately. Use the labeled P1 preflight preset when a guaranteed rejection demonstration is needed; keep the source of each candidate visible.

Additional release tests

Test	Required result	Requirement
Zero, negative, fractional, and boolean quantities	Validation fails; no authorization or execution	FR02
Empty candidate, 11 actions, unsupported operation, or extra action attributes	Candidate rejected as invalid; inventory unchanged	FR02
One action moves 11 packages	Validation accepts the quantity’s integer form; authorization denies the action	FR02–03

HAZMAT destination	Denied as restricted, not treated as an ordinary missing zone	FR03
Unknown normal zone or insufficient source stock	Simulation fails with a specific reason	FR04
Two transfers each fit B separately but exceed its capacity together	Sequential simulation fails the whole plan	FR04
An intermediate step exceeds capacity but a later step would repair it	Candidate still fails	FR04
A request explicitly requires destination B	No unauthorized redirection to C during replanning	FR05
Three unsuccessful candidate evaluations	No fourth candidate and no commit	FR05
Totals of 4 and 5, including split actions	Respect the approval threshold exactly	FR06
Denied or timed-out approval	No movement; clear terminal outcome	FR07
Agent revoked after approval but before execution	Final authorization blocks the commit	FR08, FR10

State changes after simulation or approval	Old evaluation and approval cannot commit	FR08
Two plans compete for the same remaining capacity	At most the currently valid commit succeeds; the other cannot use stale state	FR08–09
Completed plan execution is repeated	Counts and version do not change again	FR09
Unauthorized administrator action or substituted agent	Request denied with no protected mutation	FR01, FR07, FR10, FR13
Reset requested while a plan is active	Reset blocked	FR13
Old plan or approval is replayed after reset	No execution against the new demo state	FR13
Authorization service unavailable or trusted attributes missing	Execution denied	FR03, NFR04
Refresh or restart while approval is pending	Plan binding and approval state remain recoverable	NFR03

Masha records expected versus observed results, the tested release, pass or fail, and any limitation. A release target is not marked passed without observed evidence.

11 Delivery plan and responsibilities

The source schedule is retained below. **Planning assumption:** event times are local to America/Los_Angeles; confirm with the team. The 12:30 AM freeze falls on **Sunday, November 8**, not Saturday.

Milestone	Date and time	Accountable owner	Acceptance deliverable
Kickoff	Sat Nov 7, 11:00 AM	All	Authorized coding window begins; shared scope and ownership confirmed
Requirement freeze	Sat, 11:15 AM	All	Seed, business rules, status meanings, and blocking decisions agreed
Simulation ready	Sat, 1:00 PM	Mariam	Passing simulator tests, including cumulative invalidity
AI and UI foundations	Sat, 2:00 PM	Tishneet and Ishani	Validated AI candidates; local dashboard with clearly labeled sample data
Replanning integrated	Sat, 4:00 PM	Tishneet and Mariam	Real plan → simulation failure → permitted replan; stale-write protection verified
Authorization ready	Sat, 6:00 PM	Mariam	Live allow, deny, and revocation behavior; current warehouse available
Low-risk path complete	Sat, 8:30 PM	Tishneet and Ishani	End-to-end automatic execution with visible status and inventory result

Complete walkthrough ready	Sat, 11:00 PM	All engineers; Masha verifies	Approval flow, deployed experience, and known limitations documented
Feature freeze	Sun Nov 8, 12:30 AM	All	Stable release, reproducible reset, QA underway
Submission	Sun, 8:30 AM	Masha	Demonstration and submission materials match observed functionality

All implementation, scaffolding, tests, and deployment code must be created within the event's authorized coding window. This PRD is planning documentation.

Ownership: Tishneet owns planning and orchestration; Mariam owns authorization, simulation, committed state, and revocation; Ishani owns the operator/admin interface; Masha owns business narrative, acceptance evidence, and submission readiness. Cross-functional decisions require agreement from affected owners.

Delivery choices

Choice	Benefit	Limitation	Recommendation
Full controlled execution demo	Demonstrates the complete product promise	Greatest integration effort	Target this release and prioritize P0 flows
Simulation-only fallback	Preserves AI planning, rejection, replanning, and visible projections	Does not demonstrate real inventory execution	Use if safe execution cannot be completed; label it clearly
Simpler trusted approval experience	Can reduce workflow integration effort	Requires explicit evidence that identity, expiry, and plan binding still work	Accept only if all approval acceptance criteria still pass

Additional UI polish	Improves presentation	Does not close correctness gaps	Start after the end-to-end path and safety tests pass

Release boundary: If safe execution or required approval controls are incomplete, disable actual inventory writes for the fallback. Reduced implementation complexity must not weaken the product's mandatory execution rules.

12 Dependencies constraints and open decisions

Dependencies and constraints

- Access to the AI service and authorization capabilities needed for the judged experience.
- A trusted administrator identity and an agreed operator-access mechanism.
- Reliable access to current inventory, agent status, pending approvals, and prior execution outcomes.
- A common interpretation of actions, revisions, workflow states, errors, and test fixtures across the team.
- An accessible demo environment, a recorded release, and a repeatable reset.
- A short hackathon delivery window; infrastructure choices must support the product acceptance criteria.
- An external engineering specification for service selection, data structures, interfaces, and deployment details.

AWS Bedrock, Verified Permissions, and the other AWS services named in the integration contract are project dependencies and engineering choices. Their interfaces, storage schemas, and deployment configuration belong in the technical specification.

Decisions to close

Decision	Recommendation or current constraint	Owner	Due

Administrator and operator authentication	Use real enforced identities with administrator-only approval, revocation, and reset	Mariam and Ishani	Kickoff
Approval duration	Proposed: 10 minutes; show expiry to the administrator	Masha and Tishneet	Before approval acceptance
Malformed AI output	Proposed: fail clearly; any repair attempt counts toward the three-candidate limit	Tishneet and Mariam	Requirement freeze
Stale-state recovery	Proposed: fail safely and start a fresh evaluation; require fresh elevated-risk approval	Tishneet and Mariam	Before execution integration
Hard versus flexible operator constraints	Preserve explicit constraints; reject or request clarification when an alternative would change intent	Masha and Tishneet	Before replanning acceptance
Reset and historical evidence	Block active workflows; restore agent and seed; isolate previous plans and preserve labeled evidence	Mariam and Masha	Before walkthrough
Aggregate transfer policy	Retain the source's per-action limit of 10; confirm that no separate per-request ceiling is intended	Mariam and Masha	Requirement freeze
Performance and repeatability targets	Confirm proposed latency target and three consecutive walkthroughs	All	Before feature freeze

Service availability and operating cost	Confirm accessible model and services in one Region; assign a budget and measure actual usage	Tishneet and Mariam	Kickoff
Detailed terminal-status mapping	Freeze mappings for validation, exhaustion, conflict, denial, and timeout	Tishneet and Ishani	Requirement freeze
Schedule timezone	Confirm America/Los_Angeles assumption and event deadlines	Masha	Kickoff

No decision in this register is treated as approved merely because a recommendation is present.

13 Assumptions and risks

Risk or assumption	Product consequence	Response and owner
AI may produce a valid split immediately	The planned rejection moment is not guaranteed	Labeled preflight preset; Tishneet and Masha
AI may change the operator’s goal while repairing a plan	A technically valid result may still be wrong for the user	Preserve hard constraints and test explicit-destination requests; Tishneet
Inventory changes during approval or execution	Prior simulation and approval become stale	Invalidate and re-evaluate; Mariam

Approval or execution state is lost or replayed	Unauthorized or duplicate movement becomes possible	Durable binding, execution replay protection, and restart tests; Tishneet and Mariam
Authorization integration or model access fails	Full judged flow may be unavailable	Verify access early; use the explicitly labeled simulation-only fallback if necessary; Tishneet and Mariam
Team members interpret shared states differently	UI may misrepresent eligibility or completion	Freeze meanings and verify handoffs against acceptance scenarios; Ishani
Numerical examples are mixed across different starting states	Demonstration evidence becomes misleading	Use the sequential D1–D3 state table as the canonical walkthrough; Masha
Reset reuses seed version 1	Old approvals may appear to match the new warehouse version	Ensure previous-run references cannot execute after reset; Mariam
Product demand and operator value are unvalidated	Hackathon success may not establish commercial viability	Conduct operator interviews and baseline task measurement after the event; Masha
Timeline and latency targets prove unrealistic	Safety work may be crowded out by presentation work	Reduce scope through the documented fallback; all owners

14 Release criteria

The complete MVP is ready only when:

- All P0 functional requirements and mandatory integrity/security requirements pass.
- D1, D2, and D3 produce the exact required counts and warehouse versions.
- Elevated-risk transfers require a valid current administrator approval.
- Revocation, stale state, duplicate requests, intermediate violations, and unsafe reset cases cannot cause an invalid commit.
- The UI and audit history show actual outcomes and distinguish projections, scripted tests, and mocks.
- Blocking decisions are resolved; accepted limitations are recorded.
- A stable release, accessible demo, reset procedure, and test evidence are available.
- Each accountable owner signs off on their area and Masha confirms demo readiness.

Proposed additional gate: Three consecutive complete walkthroughs pass from controlled resets.

A simulation-only fallback must be described and judged as such. It does not satisfy the complete MVP's real-execution criteria.

15 Future considerations

After validating the MVP, prioritize discovery around operator workflow fit and measurable value before expanding the platform.

Potential extensions include multiple warehouses and agents; additional inventory operations; configurable approval thresholds and aggregate transfer limits; richer simulation constraints; notifications and escalation for pending approvals; audit export and retention controls; enterprise access management; and production availability and performance commitments.

Each extension requires its own user need, success measure, authorization boundaries, and acceptance criteria. Changes to risk or approval policy must remain explicitly governed.

16 Approval

Approver	Area	Approval status	Date
Masha	Product scope, demo narrative, QA, and submission readiness	Pending	—

Tishneet	AI behavior, candidate limits, orchestration, and approval lifecycle	Pending	—
Mariam	Policy, simulation, inventory integrity, and security	Pending	—
Ishani	Operator/admin experience, status meanings, and error handling	Pending	—

Approval confirms the agreed scope and resolved decisions. Subsequent changes affecting requirements, business rules, or acceptance criteria require review by the affected owners.

Source reference: SentinelOS Team Integration Contract in Pasted markdown.md, version 1.0. The integration contract remains the companion engineering reference.

SentinelOS — Team Integration Contract

Version: 1.0 (freeze before coding)
Hackathon: AWS × A10, November 7–8, 2026
Owners: Tishneet (AI & orchestration), Mariam (security & data), Ishani (frontend & API), Masha (product & QA)
Code rule: All implementation, scaffolding, tests, and deployment code is created only during the event's authorized coding window (11:00 AM Saturday). This file is a planning specification, not executable code.

0. Golden rules

1. **One repo, one source of truth.** Use these contracts. No parallel independent app scaffolds.
2. **AI proposes; backend decides.** Bedrock cannot call the executor or write warehouse state.
3. **An individual authorized action can still cause a collectively invalid plan.** Authorize every action; simulate the full sequence.
4. **The model cannot certify its own output.** Validate plan schema, authorization, and simulation independently.
5. **Approval is specific to a plan revision.** A revision or state change invalidates the previous approval.
6. **Reauthorize and resimulate at execution time.** Use a conditional, idempotent commit.
7. **No secrets in React.** AWS credentials and Step Functions callback tokens stay on the trusted backend.
8. **No mocks in the judged demo without labeling them as mocks.** Preserve a working minimum viable demo over extra infrastructure.

1. Canonical conventions

- All HTTP JSON properties use `snake_case`; all status strings use `UPPER_SNAKE_CASE`.
- Quantities are positive integers; booleans and fractional values are invalid quantities.
- All zone names are uppercase (`A`, `B`, `C`). `HAZMAT` is *not* an inventory zone in the seed model; it is a reserved, restricted destination. Requests targeting it are denied rather than treated as a normal missing zone.
- The demo principal is `inventory-agent-01`, selected **server-side**; clients never choose a principal.
- ISO-8601 UTC timestamps, e.g., `2026-11-07T20:00:00Z`.

- `plan_id` identifies the whole user request; `revision` starts at 1 and increments for each Bedrock candidate; `attempt` equals the candidate revision number and maxes at 3.
- `warehouse_version` is an integer; start at 1 and increment on every committed inventory change.
- A revision's `actions` are immutable after validation. Store or hash the canonical JSON array for approval/execution binding.
- API errors use the same `error` object (see §3).
- Money/costs, if shown, are estimates until calculated from measured AWS calls.

2. Seed state and allowable operations

2.1 `WarehouseState`

```
{
  "warehouse_id": "demo-warehouse",
  "version": 1,
  "zones": {
    "A": {"inventory": 20, "capacity": 30},
    "B": {"inventory": 8, "capacity": 10},
    "C": {"inventory": 5, "capacity": 20}
  },
  "restricted_zones": ["HAZMAT"]
}
```

2.2 `AgentRecord`

```
{
  "agent_id": "inventory-agent-01",
  "status": "ACTIVE",
  "role": "INVENTORY_MANAGER",
  "max_transfer": 10
}
```

Valid agent states: `ACTIVE`, `REVOKED` (optional `SUSPENDED` handled as denied).

2.3 `PlanActions` — the only Bedrock-generated payload

```
{
  "actions": [
    {"type": "move_packages", "from": "A", "to": "B", "count": 5}
  ]
}
```

- `actions`: 1–10 entries.
- Each action has exactly `type`, `from`, `to`, `count`; no extra properties.
- `type` is the single allowed string `move_packages`.
- `from` and `to` must be distinct uppercase zone labels. A restricted target is separately rejected by authorization/simulation.
- `count` is a JSON integer ≥ 1; an action count > 10 is schema-valid but denied by authorization. This distinction is intentional.
- Do **not** ask Bedrock to generate `plan_id`, `agent_id`, authorization flags, warehouse versions, approval flags, or execution status. The backend adds those fields.

2.4 `PlanCandidate` — backend envelope


```
{
  "plan_id": "plan-001",
  "revision": 1,
  "agent_id": "inventory-agent-01",
  "instruction": "Move five packages out of A, preferably into B; another zone is acceptable.",
  "warehouse_version": 1,
  "plan": {"actions": [{"type": "move_packages", "from": "A", "to": "B", "count": 5}]}
}
```


The browser submits only the `instruction`. The trusted backend assigns the other fields.

3. Shared response and error contracts

3.1 `ValidationResult`

Valid:

{"valid": true, "errors": []}

Invalid:

{"valid": false, "errors": [{"code": "INVALID_COUNT", "message": "count must be a positive integer", "action_index": 0}]}

3.2 `AuthorizationResult`


```
{
  "allowed": true,
  "decisions": [
    {"action_index": 0, "allowed": true, "reason_code": "POLICY_ALLOWED"}
  ]
}
```

Denied example: `{"allowed":false,"decisions":[{"action_index":0,"allowed":false,"reason_code":"AGENT_REVOKED"}]}`. All actions must be allowed for overall `allowed = true`. Authorization errors fail closed.

3.3 `SimulationResult`

Successful:


```
{
  "passed": true,
  "warehouse_version": 1,
  "reason": null,
  "violations": [],
  "projected_state": {
    "A": {"inventory": 15, "capacity": 30},
    "B": {"inventory": 10, "capacity": 10},
    "C": {"inventory": 8, "capacity": 20}
  }
}
```


Failed:


```
{
  "passed": false,
  "warehouse_version": 1,
  "reason": "Zone B would exceed capacity",
  "violations": [
    {"code": "CAPACITY_EXCEEDED", "action_index": 0, "zone": "B", "projected": 13, "capacity": 10}
  ],
  "projected_state": {
    "A": {"inventory": 15, "capacity": 30},
    "B": {"inventory": 13, "capacity": 10},
    "C": {"inventory": 5, "capacity": 20}
  }
}
```


On failure, `projected_state` is a **counterfactual**, never a committed state. The simulator works on a copy and never updates DynamoDB. Unknown-zone, invalid-count, insufficient-source, restricted-destination, and empty-plan errors use the same `violations` array with suitable codes.

3.4 `RiskResult`

{"risk": "ELEVATED", "approval_required": true, "total_packages": 5}

Rules: `total_packages = sum(count for each action)`; 1–4 → `LOW`/false; ≥5 → `ELEVATED`/true. A failed authorization or failed simulation is **never** approval-eligible.

3.5 `ExecutionResult`

```
{
  "plan_id": "plan-001",
  "revision": 2,
  "executed": true,
  "status": "COMPLETED",
  "warehouse_version_before": 2,
  "warehouse_version_after": 3,
  "warehouse": {
    "warehouse_id": "demo-warehouse",
    "version": 3,
    "zones": {
      "A": {"inventory": 12, "capacity": 30},
      "B": {"inventory": 10, "capacity": 10},
      "C": {"inventory": 11, "capacity": 20}
    },
    "restricted_zones": ["HAZMAT"]
  }
}
```

A repeated request with the same `(plan_id, revision)` must never apply transfers again; return `ALREADY_EXECUTED` or the prior result. A stale version returns `VERSION_CONFLICT` without changing warehouse state.

3.6 Standard error object

```
{
  "error": {
    "code": "VERSION_CONFLICT",
    "message": "Warehouse changed since simulation. Re-evaluate this plan.",
    "retryable": true
  }
}
```

Suggested codes: `INVALID_REQUEST`, `MODEL_ERROR`, `PLAN_VALIDATION_FAILED`, `AUTHORIZATION_DENIED`, `SIMULATION_FAILED`, `MAX_ATTEMPTS_REACHED`, `APPROVAL_REQUIRED`, `APPROVAL_DENIED`, `APPROVAL_EXPIRED`, `VERSION_CONFLICT`, `ALREADY_EXECUTED`, `INTERNAL_ERROR`.

4. Six module contracts (owner → consumer)

Function	Owner	Inputs	Outputs	Caller
`generate_plan`	**Tishneet**	`instruction: str`, `warehouse: WarehouseState`	`PlanActions`	Workflow
`replan`	**Tishneet**	`instruction`, failed `PlanActions`, `SimulationResult`, `warehouse`, `attempt`	`PlanActions`	Workflow
`authorize_plan`	**Mariam**	server-trusted `agent_id`, `PlanActions`, trusted agent record	`AuthorizationResult`	Workflow / executor
`simulate_plan`	**Mariam**	`PlanActions`, `WarehouseState`	`SimulationResult`	Workflow / executor
`request_approval`	**Tishneet + Mariam**	`plan_id`, `revision`, plan fingerprint, simulation details, task token (backend only)	approval record (`PENDING`)	Workflow callback adapter
`execute_plan`	**Mariam**	trusted `plan_id`, `revision`, `agent_id`, immutable plan, `warehouse_version`, trusted approval state	`ExecutionResult` or standard error	Workflow only

Additional shared responsibilities: `classify_risk` (Tishneet; deterministic), database access/revocation/audit (Mariam), HTTP handlers and `api.js` (Ishani).

Execution boundary: Only the trusted workflow invokes `execute_plan`; the client has no direct `/execute` endpoint.

5. HTTP API contracts — Ishani owns the public interface

Base path: `/v1` (use a configured environment URL in React). All mutating endpoints require appropriate authenticated identity. In a demo, choose and document a real authentication mechanism; **do not put a shared admin key in frontend source**.

Method	Path	Request	Response	Backend dependency
`POST`	`/v1/plans`	`{ "instruction": "..." }`	`202 {"plan_id":"plan-001","status":"PLANNING"}`	Tishneet: start workflow
`GET`	`/v1/plans/{plan_id}`	—	`200 PlanStatus`	Tishneet: workflow status/history
`GET`	`/v1/warehouse`	—	`200 WarehouseState`	Mariam: get_warehouse
`GET`	`/v1/agents/current`	—	`200 AgentRecord`	Mariam: get_agent
`GET`	`/v1/audit?plan_id=...`	Optional plan ID	`200 {"events":[AuditEvent,...]}`	Mariam: audit data
`POST`	`/v1/approvals/{approval_id}`	`{"decision":"APPROVED"}` or `{"decision":"DENIED"}`	`202 {"approval_id":"approval-001","status":"RECORDED"}`	Tishneet/Mariam: trusted approval callback
`POST`	`/v1/agents/current/revoke`	`{}`	`200 AgentRecord`	Mariam: revoke_agent
`POST`	`/v1/demo/reset`	`{}`	`200 WarehouseState`	Mariam: reset only when no active workflows

5.1 `PlanStatus` — primary UI polling result


```
{
  "plan_id": "plan-001",
  "status": "AWAITING_APPROVAL",
  "agent_id": "inventory-agent-01",
  "instruction": "Move five packages out of A, preferably to B; use another zone if needed.",
  "attempt": 2,
  "warehouse_version": 2,
  "candidates": [
    {
      "revision": 1,
      "plan": {"actions": [{"type": "move_packages", "from": "A", "to": "B", "count": 5}]},
      "authorization": {"allowed": true, "decisions": [{"action_index": 0, "allowed": true,
"reason_code": "POLICY_ALLOWED"}]},
      "simulation": {"passed": false, "warehouse_version": 2, "reason": "Zone B would exceed
capacity", "violations": [{"code": "CAPACITY_EXCEEDED", "action_index": 0, "zone": "B",
"projected": 13, "capacity": 10}], "projected_state": {"A": {"inventory": 12, "capacity": 30}, "B":
{"inventory": 13, "capacity": 10}, "C": {"inventory": 8, "capacity": 20}}}
    },
    {
      "revision": 2,
      "plan": {"actions": [{"type": "move_packages", "from": "A", "to": "B", "count": 2}, {"type":
"move_packages", "from": "A", "to": "C", "count": 3}]},
      "authorization": {"allowed": true, "decisions": [{"action_index": 0, "allowed": true,
"reason_code": "POLICY_ALLOWED"}, {"action_index": 1, "allowed": true, "reason_code":
"POLICY_ALLOWED"}]},
      "simulation": {"passed": true, "warehouse_version": 2, "reason": null, "violations": [],
"projected_state": {"A": {"inventory": 12, "capacity": 30}, "B": {"inventory": 10, "capacity": 10}, "C":
{"inventory": 11, "capacity": 20}}}
    }
  ],
  "risk": {"risk": "ELEVATED", "approval_required": true, "total_packages": 5},
  "approval": {"approval_id": "approval-001", "status": "PENDING", "revision": 2},
  "execution": null,
  "updated_at": "2026-11-08T01:02:03Z"
}
```


In other states, `candidates` may be empty, `risk`/`approval`/`execution` may be `null`, and the appropriate fields appear as the workflow proceeds.

Allowed `PlanStatus.status` values:

`PLANNING`, `VALIDATING`, `AUTHORIZING`, `SIMULATING`, `REPLANNING`, `AWAITING_APPROVAL`, `REAUTHORIZING`, `REVALIDATING`, `EXECUTING`, `COMPLETED`, `REJECTED`, `FAILED`, `EXPIRED`.

Terminal: `COMPLETED`, `REJECTED`, `FAILED`, `EXPIRED`. UI polls every 2 seconds while nonterminal; refresh warehouse after `COMPLETED`.

5.2 `AuditEvent`

```
{
  "plan_id": "plan-001",
  "event_id": "2026-11-08T01:02:03.000Z#0002",
  "timestamp": "2026-11-08T01:02:03Z",
  "revision": 1,
  "event_type": "SIMULATION_FAILED",
  "outcome": "REJECTED",
  "reason_code": "CAPACITY_EXCEEDED",
  "message": "Projected Zone B inventory is 13, capacity is 10"
}
```

The audit query must be able to retrieve recent events across plans; define a suitable secondary index or small-demo bounded query, not an unbounded table scan in a real application.

6. DynamoDB schema — Mariam owns logic and table requirements

Table	Partition key	Sort key	Fields
`sentinel_warehouse`	`warehouse_id` (S)	—	`version` (N), `zones` (Map), `restricted_zones` (List)
`sentinel_agents`	`agent_id` (S)	—	`status`, `role`, `max_transfer`
`sentinel_audit`	`plan_id` (S)	`event_id` (S)	`timestamp`, `revision`, `event_type`, `outcome`, `reason_code`, `message`, optional details

Additional necessary durable data: pending approvals and plan-execution idempotency records. For the hackathon, place typed records in a small separate `sentinel_workflow` table keyed by `record_id`, or add carefully namespaced records to an existing suitable table.
Decision required at kickoff: choose one approach and write it into `docs/decisions.md`

before implementing callback handling. Do not rely on in-memory token storage across Lambda invocations.

For the single-item warehouse, an execution transaction should conditionally update `version` and atomically create a unique execution marker `(plan_id, revision)`, preventing duplicate commits. Approval record must be tied to the exact plan revision/fingerprint. Resolve what happens to previously queued plans during `POST /demo/reset` (recommended: only permit reset when no workflows are active).

7. Authorization contract — Mariam

- `principal`: server-established `Sentinel::Agent::"inventory-agent-01"`.
- `action`: `Sentinel::Action::"MovePackages"`.
- `resource`: `Sentinel::Warehouse::"demo-warehouse"`.
- Trusted context or entity attributes: `count`, `destination`, `agent_status` from DynamoDB.
- Policy intent: active agent + count ≤10 + non-HAZMAT destination → allow; revoked/suspended, count >10, and HAZMAT → deny.
- `IsAuthorized` for **every action of every candidate**; deny entire plan if any action denied.
- **Reauthorize immediately before execution** with current trusted status. Missing attributes/errors → deny.
- Never ask Bedrock to bypass an authorization denial.
- Browser access to the approval and revocation endpoints requires admin authorization distinct from the inventory-agent identity.

8. Workflow ownership — Tishneet

Canonical Standard Step Functions outline:

`GENERATE → VALIDATE → AUTHORIZE → SIMULATE → (REPLAN if failed; max 3 candidates) → CLASSIFY_RISK → (WAIT_APPROVAL if >=5 total packages) → REAUTHORIZE → REVALIDATE → EXECUTE → AUDIT`.

- Denied authorization → `REJECTED` and audit; **do not** replan merely to evade policy.
- Simulation failure → Bedrock revision if attempts remain and the operator goal permits an alternative.
- Approval is allowed only for plans that have **already passed** authorization and simulation.
- Approval callback token is written only to trusted durable storage and never sent to the browser.
- Deny / timeout → no execution.

- Executor rechecks both authorization and warehouse version; DynamoDB conditional/transactional commit is authoritative.
- Tishneet coordinates Lambda function ARNs, state-machine inputs/outputs, and IAM integration, but does not own other engineers' implementation.

9. Source control and integration rules

9.1 Branch ownership

- `main` — always demoable; only tested PRs merged.
- `feature/ai-workflow` — Tishneet (`backend/ai/`, `backend/workflows/`).
- `feature/security-backend` — Mariam (`backend/core/`, `backend/data/`, `backend/security/`).
- `feature/frontend-api` — Ishani (`frontend/`, `backend/api/`).
- Masha — content, QA, submission docs in a separate branch or collaborative document.

9.2 Shared resource ownership

Artifact / AWS resource	Primary owner	Integration reviewer
Bedrock prompts, model settings, planning Lambda	Tishneet	Mariam
State-machine definition and callback orchestration	Tishneet	Mariam
Simulator, authorization, executor Lambdas	Mariam	Tishneet
DynamoDB schema, seed, data access	Mariam	Tishneet
Verified Permissions policy store / Cedar	Mariam	Tishneet
API Gateway and HTTP API handlers	Ishani	Mariam / Tishneet
React frontend and Amplify Hosting	Ishani	Masha
Common SAM `template.yaml` and IAM resources	**One integration editor at a time, coordinated by Tishneet**	All affected owners

Test matrix, business deck, Devpost	Masha	All engineers

9.3 Merge rules

1. One laptop creates the initial scaffold at 11:00 AM; commit and push before other branches begin.
2. Each engineer branches from updated `main` and edits only owned files.
3. All shared contract changes require agreement by all affected owners **before** code changes.
4. Before merging: tests pass, no secrets committed, author can explain the change, and the consumer verifies the interface.
5. Keep changes small and merge into `main` roughly every 1–2 hours when tested.
6. Avoid competing edits to SAM templates, workflow definition, IAM, and shared JSON contracts.
7. Deploy only from a known-good commit; record deployed commit and AWS Region.
8. If a component is unavailable, mark mock/fallback status explicitly in QA and Devpost.

9.4 Handoff deadlines

Time (Saturday)	Producer	Consumer	Must hand over
11:15 AM	All	All	Frozen seed data, schemas, status enum, resource owner choices
1:00 PM	Mariam	Tishneet	`simulate_plan` response + passing tests
2:00 PM	Tishneet	Mariam / Ishani	Bedrock `PlanActions` + validation examples
2:00 PM	Ishani	All	Mock frontend and API request samples
4:00 PM	Tishneet + Mariam	Ishani	Working plan/replan/simulation response shape
6:00 PM	Mariam	Tishneet / Ishani	Live authorization, warehouse, revocation interfaces
8:30 PM	Tishneet	Ishani / Mariam	Workflow statuses + approval state handoff
11:00 PM	All engineers	Masha	Working demo, deployed URLs, known limitations

12:30 AM	All	All	Feature freeze, working commit ID, reproducible reset

10. Judging demos — exact expected states

All three demos run **in order on the same state**.

Demo 1 — valid low-risk transfer

Instruction: `Move three packages from Zone A to Zone C.`
Candidate: `A → C x3`; allowed, passes simulation, auto-executes.
Actual after Demo 1: A 17/30, B 8/10, C 8/20; warehouse version 2.

Demo 2 — authorized but collectively invalid plan

Instruction: `Move five packages out of Zone A, preferably into Zone B. Use another zone if necessary.`
Preferred candidate: `A → B x5`; authorized but projects B 13/10 (reject).
Revised candidate: `A → B x2, A → C x3`; passes, **requires approval** (5 total).
Actual after approval: A 12/30, B 10/10, C 11/20; warehouse version 3.

Demo reliability note: Bedrock might propose the valid split immediately. For a guaranteed demonstration of plan rejection, provide a clearly labeled `Run preflight test` preset using the original `A → B x5` plan, while keeping real AI replanning. Never misrepresent a scripted initial candidate as an unexpected autonomous model mistake.

Demo 3 — revocation

Admin revokes `inventory-agent-01`; subsequent `A → C x1` request is denied by Verified Permissions.
Actual after Demo 3: A 12/30, B 10/10, C 11/20; version remains 3.

11. Quality gates and fallback

1:00 PM: pure simulator tests pass (Mariam).
2:00 PM: Bedrock returns validated plans; React dashboard runs locally (Tishneet + Ishani).
4:00 PM: Bedrock → simulator → replan works; DynamoDB executor passes stale-write test.
6:00 PM: Verified Permissions deny/allow/revocation works.
8:30 PM: complete low-risk flow through integrated backend.
11:00 PM: approval flow works or a simpler trusted approval mechanism is documented.

12:30 AM: freeze features; run QA.
8:30 AM Sunday: submit.

Fallback order:

1. Preserve Bedrock + simulator + AI-replanning + visible before/after comparison.
2. Add real DynamoDB commit with version checks.
3. Add Verified Permissions and revocation.
4. Add Step Functions workflow / callback.
5. Add UI polish only after working end-to-end tests.

12. Kickoff decisions (write answers before coding)

Question	Recommendation	Decision
AWS Region	Choose one Region with Bedrock model + required services	____
Bedrock model	Actual accessible model; configured env var	____
Shared infrastructure editor	Tishneet coordinates; one editor at a time	____
Admin authentication	Choose an actual backend-enforced mechanism; never public client secret	____
Approval/token/idempotency storage	Small `sentinel_workflow` DynamoDB table recommended	____
Exact approval SLA/timeout	e.g., 10 minutes for demo	____
Simulation/authorization result shape	Use §§3–4 unchanged	AGREED / AMEND
Local mock mode	Explicit flag only; disabled for judging	AGREED / AMEND
Reset behavior	Only with no active plans	AGREED / AMEND

Contract freeze: All three engineers verbally approve this document at kickoff. Any subsequent schema change must be recorded here and communicated to every consumer.

SENTINELOS — COMPLETE TEAM EXECUTION PLAN

AWS × A10 Hackathon | November 7–8, 2026

Project: SentinelOS — Preflight Security for Enterprise AI Agents
Team: Tishneet, Mariam, Ishani, Masha
Hacking begins: Saturday, 11:00 AM
Submission deadline: Sunday, 9:00 AM
Internal submission target: Sunday, 8:30 AM

Your team has approximately **22 hours to build, test, and submit SentinelOS**.

The most important rule is that everyone must work on a separate part of the project while building toward the same system.

You do **not** want four people asking Claude Code to build SentinelOS independently. That would create incompatible implementations and waste hours.

1. Everyone's responsibilities

Person	Role	Owns	Must deliver
Tishneet	AI & Workflow Engineer	Bedrock, plan generation, replanning, Step Functions	Working AI planning and orchestration
Mariam	Security & Backend Engineer	Simulation, DynamoDB, Verified Permissions, execution	Working safety and authorization engine
Ishani	Frontend & API Engineer	React, dashboard, API endpoints, UI integration	Working interactive application
Masha	Product, Business & QA Lead	Presentation, Devpost, business model, testing	Competition-ready submission and demo

What success looks like

By Sunday morning:

Tishneet: A user instruction becomes an AI-generated plan, failed plans can be revised, and the workflow handles human approval.

Mariam: Plans are independently validated, unauthorized actions are rejected, and valid plans safely update warehouse data.

Ishani: Judges can enter requests, see warehouse inventory, compare plans, approve operations, and revoke an agent.

Masha: The team has a polished presentation, reliable live demonstration, realistic business explanation, and complete Devpost submission.

PART 1 — BEFORE THE HACKATHON

Friday night: November 6

No coding. Only preparation permitted by the event rules.

All four members should understand the three demonstrations and agree on the exact data exchanged between components.

Tishneet — AI preparation

Research and document:

1. How to invoke Amazon Bedrock using the Converse API.
2. Which model IDs are available in your hackathon AWS Region.
3. How to require structured JSON output.
4. How to send previous simulation failures back to Bedrock.
5. How Step Functions Standard workflows operate.
6. How callback-task human approval works.

Deliverable: A shared technical document describing the Bedrock request/response format and the planned workflow.

Do not create implementation files tonight.

Mariam — Security and simulation preparation

Research and document:

1. Amazon DynamoDB tables and conditional writes.
2. Amazon Verified Permissions and Cedar policies.
3. The `IsAuthorized` API.

4. How revocation will be enforced.
5. How the warehouse simulation will detect invalid transfers.
6. How an execution will avoid modifying stale warehouse state.

Write down the intended data structures.

For example, the warehouse will begin with:

Zone	Inventory	Capacity
A	20	30
B	8	10
C	5	20

Deliverable: A written simulation specification, proposed Cedar rules, and database schema.

Ishani — Frontend preparation

Prepare:

1. Dashboard wireframe.
2. Component list.
3. Basic page layout.
4. The API request and response formats.
5. A rough design for the original-versus-revised plan comparison.
6. Approval and revocation user flows.

The dashboard should have four major areas:

- Warehouse overview
- AI instruction console
- Preflight results
- Security controls and audit log

Deliverable: A wireframe and interface specification.

Do not build the React application before the event begins.

Masha — Product and presentation preparation

Prepare:

1. Problem statement.
2. Target customer.

3. Competitor comparison.
4. Business model.
5. Slide outline.
6. Devpost outline.
7. Demo presentation responsibilities.
8. A QA checklist.

Deliverable: Presentation outline, draft pitch narrative, business-model assumptions, and testing plan.

Masha should also understand the project well enough to demonstrate it without engineering assistance.

PART 2 — SATURDAY BUILD SCHEDULE

11:00 AM–12:00 PM — Project setup

Goal: Create one shared repository and establish the interfaces.

Tishneet

- Verify Bedrock credentials and model availability.
- Test the Bedrock API from the hackathon AWS account.
- Start the plan-generation module.

Mariam

- Create the local Python simulator.
- Define the initial warehouse state.
- Write the first simulation tests.

Ishani

- Initialize the React/Vite project.
- Create the dashboard shell.
- Build warehouse zone cards using test data.

Masha

- Confirm everybody's responsibilities.

- Create the task tracker.
- Verify the judging requirements.
- Confirm mentor availability.
- Track which components have been started.

Team checkpoint — 12:00 PM

You should have:

✅ One shared GitHub repository
✅ Agreed JSON contracts
✅ Bedrock access confirmed or a documented fallback
✅ Basic Python simulator started
✅ React application running locally

Important: Claude Code should not be allowed to silently modify another teammate's owned files. Each engineer should work on a separate branch or worktree.

12:00–2:00 PM — Core development

This is the most important early work period.

Tishneet — AI planner

Build:

generate_plan(instruction, warehouse)

The function should accept a natural-language instruction and the current warehouse state.

It should return validated structured actions.

Example output:

```
{
  "actions": [
    {
      "type": "move_packages",
      "from": "A",
      "to": "C",
      "count": 3
    }
  ]
```

}

Definition of done: Three different natural-language instructions produce valid structured plans.

Mariam — Warehouse simulator

Build:

simulate_plan(plan, warehouse)

It should check:

- Zone existence
- Valid transfer counts
- Available source inventory
- Destination capacity
- Restricted zones
- Sequential effects of multiple transfers

Definition of done: Tests pass for valid transfers and capacity violations, including a plan where individually valid transfers collectively exceed capacity.

Ishani — Dashboard

Build:

- Header
- Three zone cards
- Instruction input
- Submit button
- Plan result panel
- Audit-log layout

Use local mock data initially.

Definition of done: A user can type a request and see mock plan results in the interface.

Masha — Business and documentation

Work on:

- Slide 1: Problem
- Slide 2: Solution
- Slide 3: Why conventional authorization is insufficient on its own
- Slide 4: Architecture

- Slide 5: Business value
- Slide 6: Demo and closing

Definition of done: A complete presentation outline with draft speaker notes.

Team checkpoint — 2:00 PM

The team must demonstrate a manually supplied plan failing the simulator while the frontend displays the relevant warehouse state.

No Step Functions integration yet.

2:00–4:00 PM — First working end-to-end demo

Tishneet

- Complete Bedrock integration.
- Validate AI output.
- Implement `replan()`.
- Limit attempts to three candidates.
- Test the failed capacity-transfer scenario.

Mariam

- Create DynamoDB tables.
- Seed the warehouse.
- Build `execute_plan()`.
- Implement conditional version checks.
- Create authorization request contracts.

Ishani

- Connect the dashboard to a backend endpoint.
- Display real generated plans.
- Build original/revised plan comparison.
- Implement loading and failure states.

Masha

- Prepare demo narration.
- Review frontend usability.
- Confirm business metrics to capture.

- Document bugs and integration problems.

Team checkpoint — 4:00 PM

The ideal result:

1. User enters a request.
2. Bedrock generates a plan.
3. Simulator rejects a capacity violation.
4. Bedrock produces a valid alternative.
5. Dashboard displays both plans.

This is your first complete demonstration of SentinelOS's key innovation.

If this doesn't work, fix it before adding features.

4:00–6:00 PM — Security integration

Tishneet

- Finish AI edge cases.
- Begin the Step Functions state-machine definition.
- Define workflow input/output formats.

Mariam

- Configure the Verified Permissions policy store.
- Implement authorization checks.
- Implement agent revocation.
- Test authorized and unauthorized operations.

Ishani

- Implement agent status display.
- Build the revoke button.
- Add audit-log rendering.
- Connect authorization results to the UI.

Masha

- Begin testing at 6:00 PM.
- Verify that each security event produces an understandable explanation.

- Check whether the dashboard communicates the distinction between authorization and simulation.

Team checkpoint — 6:00 PM

You should be able to demonstrate:

✅ Authorized transfer
✅ Unauthorized transfer blocked
✅ Capacity violation detected
✅ AI-generated revised plan
✅ Revoked agent denied future operations

6:00–8:30 PM — Workflow integration

Tishneet

- Build the Step Functions state machine.
- Connect Bedrock, authorization, and simulation.
- Implement the replanning loop.
- Handle workflow errors and retries.

Mariam

- Integrate Lambda functions with the state machine.
- Recheck authorization before execution.
- Ensure database version checks work.
- Add backend audit records.

Ishani

- Implement workflow polling.
- Display the workflow's current stage.
- Build pending-approval interface.
- Integrate backend execution status.

Masha

- Run the test checklist.
- Confirm successful plans don't mutate data twice.
- Record failures.
- Ask a mentor to review the architecture and product distinction.

Team checkpoint — 8:30 PM

A low-risk request should run through the actual Step Functions workflow and update DynamoDB.

8:30–11:00 PM — Human approval

Tishneet

- Implement the Step Functions callback task.
- Configure timeout behavior.
- Handle approval and denial outcomes.

Mariam

- Build trusted approval handling.
- Ensure callback tokens remain server-side.
- Reauthorize approved plans before execution.
- Revalidate warehouse state.

Ishani

- Complete Approve and Deny buttons.
- Display pending approval details.
- Add confirmation/error states.

Masha

- Test the complete approval workflow.
- Verify the correct plan is being approved.
- Confirm that a rejected plan cannot execute.
- Begin drafting the final Devpost description.

Team checkpoint — 11:00 PM

The complete five-package demonstration should work, including replanning and human approval.

11:00 PM–12:30 AM — Feature freeze

No new capabilities.

Focus exclusively on:

- Fixing integration bugs
- Improving reliability
- Finalizing the audit log
- Testing revocation
- Preparing demo reset
- Capturing successful screenshots

At 12:30 AM, freeze the architecture.

PART 3 — OVERNIGHT SCHEDULE

The sleep schedule you proposed is workable as a rotation, but it is demanding. Everyone should get rest before the final demonstration, and no one should be responsible for deploying or executing destructive AWS operations while severely fatigued.

Time	Tishneet	Mariam	Ishani	Masha
12:30–3:00 AM	Backend testing	Backend testing	**Sleep**	**Sleep**
3:00–5:30 AM	**Sleep**	**Sleep**	UI polish and screenshots	Devpost and presentation
5:30–6:30 AM	Integration review	Security review	Frontend review	Full QA
6:30–7:30 AM	Fix critical AI bugs	Fix critical backend bugs	Fix critical UI bugs	Record demo and rehearse
7:30–8:00 AM	Final verification	Final verification	Final verification	Final presentation rehearsal
8:00–8:30 AM	Support submission	Support submission	Verify deployed website	**Submit to Devpost**
8:30–9:00 AM	No new development	No new development	No new development	Confirm successful submission

Overnight responsibilities

12:30–3:00 AM — Tishneet and Mariam

Focus on backend reliability.

Test the full pipeline repeatedly, including revocation while an approval is pending.

If Step Functions is still unstable, preserve the working Bedrock-and-simulator demo. Do not risk breaking it to finish orchestration.

3:00–5:30 AM — Ishani and Masha

Ishani should concentrate on presentation readiness: clear status messages, consistent zone counts, screenshots, and responsiveness.

Masha should finish Devpost, presentation materials, and the demo recording.

Neither should make architectural changes while the backend engineers are sleeping.

5:30–8:30 AM — Everyone

Run the complete demonstration, correct only critical failures, and submit early.

I strongly recommend having an emergency recorded demonstration available if the live AWS application fails during judging.

PART 4 — EXACT PROMPTS FOR EACH ENGINEER

These prompts are designed for Claude Code.

Do not run them before 11:00 AM Saturday, assuming the hackathon's on-site coding rule is as described.

Before using the prompts, create one repository and agree on the JSON contracts.

Each engineer should work in their own branch or separate working tree. Only one person should modify shared infrastructure definitions at a time.

4.1 Tishneet — AI Planning and Step Functions

Primary responsibility: Make Amazon Bedrock generate and revise plans, then integrate those capabilities into the AWS workflow.

Files she owns:

```
backend/
├── ai/
│   ├── planner.py
│   ├── replan.py
│   ├── schemas.py
│   └── prompts.py
├── workflows/
│   └── statemachine.asl.json
└── tests/
    ├── test_planner.py
    └── test_replan.py
```

Claude Code prompt

You are helping me build the AI planning and AWS Step Functions components for SentinelOS during the AWS × A10 Hackathon.

I am Tishneet, responsible for Amazon Bedrock plan generation, structured JSON validation, AI replanning, and Step Functions workflow orchestration.

Other teammates own the React frontend, warehouse simulator, DynamoDB data model, and security engine. Do not modify their files without coordinating.

PROJECT CONCEPT

SentinelOS evaluates AI-generated business plans before execution. Our demonstration uses warehouse inventory transfers between three zones:

- A: 20 packages, capacity 30
- B: 8 packages, capacity 10
- C: 5 packages, capacity 20

The AI must generate structured inventory-transfer plans from natural-language instructions.

TECHNOLOGIES

- Python 3.12
- Amazon Bedrock Converse API
- JSON Schema validation
- AWS Step Functions Standard workflows
- AWS Lambda

- pytest

MY RESPONSIBILITIES

1. Implement generate_plan(instruction, warehouse_state).
2. Return a structured JSON plan.
3. Validate generated actions against our agreed schema.
4. Implement replan(original_goal, failed_plan, rejection_reason, warehouse_state).
5. Limit planning to three candidate plans total.
6. Handle malformed outputs, API failures, and timeouts.
7. Build the Step Functions workflow after planning works.

EXAMPLE PLAN

```
{
"actions": [
{
"type": "move_packages",
"from": "A",
"to": "B",
"count": 5
}
]
}
```

REPLANNING SCENARIO

Initial warehouse:
A = 20/30
B = 8/10
C = 5/20

Instruction:
"Move five packages out of Zone A, preferably into Zone B. Use another zone if necessary."

First plan:
Move 5 from A to B.

Simulator response:
Rejected. B would contain 13 packages, exceeding capacity 10.

Expected alternative:
Move 2 from A to B and 3 from A to C.

The AI is responsible for proposing plans. It must never approve, execute, or directly update warehouse state.

STEP FUNCTIONS

After the planning module works, implement the following workflow:

GeneratePlan
ValidatePlan
AuthorizePlan
SimulatePlan
Replan when necessary
ClassifyRisk
WaitForApproval when required
Reauthorize
Revalidate
Execute
Audit

The workflow should support:

- Maximum three planning attempts
- Human-approval callback tasks
- Approval timeouts
- Explicit failure states
- Logging of failed executions

The approval task token must remain on the trusted backend.

IMPLEMENTATION RULES

Use the model and AWS Region available in our hackathon account.

Do not assume structured-output features exist for every Bedrock model. Validate capabilities before choosing the implementation.

Use mocks for local tests.

Keep the functions modular, readable, and easy to explain.

Follow the shared contracts in docs/contracts.md.

Do not build frontend components or rewrite the simulator.

FIRST TASK

Inspect the repository and shared contracts.

Implement only the Bedrock planning module, schema validation, and tests.

Stop after completing those components. Explain how to run the tests before moving on to replanning or Step Functions.

Tishneet's checkpoints

Time	Required result
12:00 PM	Bedrock access confirmed
2:00 PM	Natural language → valid JSON
4:00 PM	Replanning works
6:00 PM	Workflow states defined
8:30 PM	Core state machine works
11:00 PM	Human approval works or fallback documented

4.2 Mariam — Security, Simulation, and Database

Your responsibility is especially important because you're implementing SentinelOS's main technical differentiator.

You own the logic that determines whether a plan can actually be executed.

You're building three connected systems:

Simulation: What would happen if the plan executed?

Authorization: Is the agent permitted to perform the requested actions?

Execution: Can the validated plan safely change the database?

Files you own

```
backend/
├── core/
│   ├── simulator.py
│   ├── executor.py
│   ├── authorization.py
│   └── validation.py
├── data/
│   ├── warehouse.py
│   ├── agents.py
```

```
│   └── audit.py
├── security/
│   ├── schema.cedar
│   └── policies/
├── scripts/
│   └── seed.py
└── tests/
    ├── test_simulator.py
    ├── test_executor.py
    └── test_authorization.py
```

Your Claude Code prompt

You are helping me build the security, simulation, and data layer for SentinelOS during the AWS × A10 Hackathon.

I am Mariam, responsible for the core plan-level simulation engine, Amazon DynamoDB state management, Amazon Verified Permissions, safe execution, and agent revocation.

Other engineers own Bedrock planning, Step Functions, React, and frontend API integration.

Do not modify their files without coordination.

PROJECT

SentinelOS is a preflight security system for enterprise AI agents.

An AI agent proposes warehouse inventory movements.

My subsystem independently determines whether the proposed actions are authorized and whether the complete plan violates warehouse constraints.

The AI must never directly modify warehouse state.

INITIAL WAREHOUSE STATE

Zone A:
Inventory 20
Capacity 30

Zone B:
Inventory 8
Capacity 10

Zone C:
Inventory 5
Capacity 20

Restricted zone:
HAZMAT

INITIAL AGENT

agent_id: inventory-agent-01
status: ACTIVE
max_transfer: 10

MY RESPONSIBILITIES

PART A — SIMULATOR

Implement:

simulate_plan(plan, warehouse_state)

The function must:

1. Deep-copy warehouse state.
2. Validate action types and arguments.
3. Reject unknown zones.
4. Reject negative, zero, noninteger, and Boolean transfer quantities.
5. Reject transfers into restricted zones.
6. Reject transfers when source inventory is insufficient.
7. Apply actions sequentially to the copied state.
8. Reject destination-capacity violations.
9. Return the projected state and clear rejection reasons.
10. Never modify the original warehouse object.

This is the central differentiator of SentinelOS.

PART B — DYNAMODB

Create the data-access layer for three tables:

sentinel_warehouse
sentinel_agents
sentinel_audit

The warehouse should be stored as one versioned item for the MVP.

Implement:

get_warehouse()
get_agent()
record_audit_event()
reset_demo()

PART C — EXECUTION

Implement:

execute_plan(plan, expected_version, agent_id)

The executor must:

1. Re-read current warehouse state.
2. Recheck current agent authorization.
3. Verify that the approved plan is exactly the plan being executed.
4. Revalidate the plan against current state.
5. Verify that the stored warehouse version matches the expected version.
6. Use a conditional DynamoDB write to prevent stale updates.
7. Ensure duplicate approval requests do not execute the same plan twice.
8. Return the committed state or a safe failure result.

Use an idempotency record or another durable mechanism rather than relying on frontend button disabling.

PART D — VERIFIED PERMISSIONS

Implement action-level authorization using Amazon Verified Permissions.

Define a Cedar schema and policies for:

- An active inventory agent may move up to ten packages per action.
- Transfers into HAZMAT are prohibited.
- Revoked agents cannot perform operations.

Implement:

authorize_plan(agent_id, plan)

The backend must retrieve the trusted agent status and pass the appropriate attributes or context into Verified Permissions.

Revocation must actually affect authorization decisions.

The browser must not be trusted to establish arbitrary agent identity.

SECURITY RULE

A plan is executable only if it passes BOTH authorization and deterministic simulation.

A valid simulation never overrides a denial from Verified Permissions.

A human approval never overrides a hard operational constraint.

TEST CASES

Test:

1. Move 3 from A to C — pass.
2. Move 5 from A to B — fail due to capacity.
3. Move 2 from A to B and 3 from A to C — pass.
4. Move more inventory than the source contains — fail.
5. Move to an unknown zone — fail.
6. Move to HAZMAT — reject.
7. Transfer zero or negative packages — fail.
8. Move more than ten packages in one authorized operation — deny.
9. Revoke the agent — subsequent operations denied.
10. Execute against an outdated warehouse version — fail.
11. Submit the same approved plan twice — execute at most once.
12. Simulate multiple individually valid transfers that collectively exceed capacity — fail.

Use pytest.

Development should work locally without AWS credentials using simple repository abstractions and mocks.

FIRST TASK

Inspect docs/contracts.md and the repository structure.

Implement simulator.py and test_simulator.py only.

Run the tests and show the results.

Do not proceed to DynamoDB, Verified Permissions, or Lambda deployment until the simulator tests pass.

Your exact order of work

Time	Your task	Result
11:00–12:00	Build the simulator	Local simulation works
12:00–1:00	Test simulator edge cases	Core tests pass

1:00–2:00	Build DynamoDB data-access layer	Warehouse can be read
2:00–3:00	Build safe executor	Valid plans change state
3:00–4:00	Add version checks and audit records	Stale writes rejected
4:00–5:00	Configure Verified Permissions	Authorization works
5:00–6:00	Implement revocation	Revoked agents denied
6:00–8:30	Integrate with Tishneet	Real workflow works
8:30–11:00	Implement approval security checks	Approved execution is safe
11:00–12:30	Resolve critical issues	Backend feature freeze

The one concept you personally must understand

The difference between simulation and execution.

Simulation asks:

"What would the warehouse look like if these actions happened?"

Execution says:

"The plan is authorized, approved if necessary, and still valid against current state. Now update the actual warehouse."

Those must be two distinct operations.

4.3 Ishani — Frontend and API Integration

Ishani owns everything the judges see and interact with.

Her most important responsibility is making a complicated AWS workflow feel simple.

Files she owns

```
frontend/
├── src/
│   ├── App.jsx
│   ├── components/
```

```
│   │   ├── Header.jsx
│   │   ├── WarehouseOverview.jsx
│   │   ├── ZoneCard.jsx
│   │   ├── AgentConsole.jsx
│   │   ├── PlanComparison.jsx
│   │   ├── ApprovalModal.jsx
│   │   ├── AuditLog.jsx
│   │   └── AgentControls.jsx
│   ├── services/
│   │   └── api.js
│   └── index.css
```

She should also own the frontend-facing API handler interfaces, coordinating deployment permissions and API Gateway configuration with the backend team.

Ishani's Claude Code prompt

You are helping me build the frontend and API integration for SentinelOS during the AWS × A10 Hackathon.

I am Ishani, responsible for the React dashboard and the frontend-facing API.

Other teammates own Bedrock planning, simulation, Verified Permissions, DynamoDB, and Step Functions.

PROJECT

SentinelOS is an enterprise AI-agent preflight security platform.

Users enter natural-language warehouse instructions.

The backend generates plans, checks authorization, simulates consequences, and may request human approval.

My job is to make the entire process understandable and interactive.

STACK

React
Vite
Tailwind CSS
JavaScript
AWS Amplify Hosting
Amazon API Gateway

DESIGN

Build a polished enterprise operations dashboard.

Use a dark navy theme with clear green, amber, and red status indicators.

The dashboard must contain:

1. Header displaying SentinelOS and current agent status.
2. Warehouse overview with three zone cards.
3. AI instruction console.
4. Plan-generation and simulation status display.
5. Side-by-side original and revised plan comparison.
6. Human approval panel.
7. Security audit log.
8. Agent revocation control.

WAREHOUSE

Initial state:

Zone A: 20/30
Zone B: 8/10
Zone C: 5/20

Each zone card should show:

- Zone name
- Current inventory
- Maximum capacity
- Utilization bar
- Status color

AI CONSOLE

Text input example:

"Move five packages out of Zone A, preferably into Zone B. Use another zone if necessary."

The submission should call the backend.

Show clear loading states.

PLAN COMPARISON

When the original plan fails, display:

ORIGINAL PLAN:
Move 5 from A to B.
Rejected because Zone B would contain 13/10 packages.

REVISED PLAN:
Move 2 from A to B.
Move 3 from A to C.
Simulation passed.

Display projected warehouse states side by side.

APPROVAL

When a plan requires human approval, display:

- Requested actions
- Agent identity
- Simulation result
- Risk classification
- Approve button
- Deny button

Never send AWS callback tokens directly to the frontend.

REVOCATION

Add a REVOKE AGENT button.

When clicked, call the backend revocation endpoint.

Update the UI to show the agent is revoked.

The backend, not React, must enforce revocation.

API

Build an api.js client for:

POST /plans
GET /plans/{id}
GET /warehouse
GET /audit
POST /approvals/{id}
POST /agents/{id}/revoke
POST /demo/reset

Use the exact schemas from docs/contracts.md.

Poll plan status every two seconds until a terminal or approval-required state.

Always distinguish between projected simulation state and committed warehouse state.

IMPLEMENTATION RULES

Do not implement security decisions in React.

Do not hard-code successful audit events.

Use mock API responses during initial development.

Keep components small and readable.

Build the interface before integrating real AWS services.

Handle network failures and pending approvals gracefully.

Avoid animation libraries unless the core interface is already complete.

FIRST TASK

Inspect docs/contracts.md.

Initialize the React/Vite/Tailwind application.

Create the main dashboard with three warehouse cards and the AI instruction input using mock data.

Stop when the interface runs locally.

Do not implement API integration or additional pages until the initial dashboard is working.

Ishani's checkpoints

Time	Required result
12:00 PM	React application running
2:00 PM	Warehouse cards and console complete
4:00 PM	Real plan-generation results displayed
6:00 PM	Authorization and revocation UI connected
8:30 PM	Workflow status visible
11:00 PM	Human approval interface complete

12:30 AM Feature freeze

Important frontend requirement

There are two different warehouse states:

Actual state: What's currently stored in DynamoDB.

Projected state: What the warehouse would look like if a proposed plan executed.

These must be clearly distinguished.

If a plan fails simulation, the actual inventory must not change.

That is essential to demonstrating the security mechanism correctly.

4.4 Masha — Product, Business, Quality Assurance, and Presentation

Masha's role is not simply making slides.

She's responsible for making sure the finished application satisfies the hackathon requirements and can be demonstrated convincingly.

She owns four areas:

Product: What problem are we solving?

Business: Why would a company purchase this?

Quality: Does the product actually work?

Presentation: Can judges understand its value in four minutes?

Deliverables

```
docs/
├── pitch/
│   ├── problem.md
│   ├── solution.md
│   ├── business-model.md
│   ├── competitor-analysis.md
│   └── demo-script.md
```

```
├── qa/
│   ├── test-checklist.md
│   ├── bug-tracker.md
│   └── demo-results.md
└── submission/
    ├── devpost-draft.md
    └── readme-outline.md
```

These are suggested organizational files; she can use shared documents instead where appropriate under the event rules.

Masha's planning prompt

You are helping me prepare the product, business model, QA process, and judging presentation for SentinelOS at the AWS × A10 Hackathon.

I am Masha, responsible for ensuring the project is understandable, commercially credible, thoroughly tested, and ready for submission.

PROJECT

SentinelOS is a preflight security platform for enterprise AI agents.

It checks whether individual AI-generated actions are authorized, simulates the consequences of an entire plan, identifies operational constraint violations, and uses Amazon Bedrock to generate acceptable alternatives.

Our prototype demonstrates warehouse inventory transfers.

CORE EXAMPLE

Warehouse:

A: 20/30
B: 8/10
C: 5/20

An AI agent proposes moving five packages from A to B.

The operation is authorized, but the simulator predicts that B would exceed capacity.

SentinelOS blocks the original plan.

Amazon Bedrock generates a revised plan moving two packages to B and three to C.

The revised plan passes simulation and requires human approval before execution.

MY RESPONSIBILITIES

1. Develop the problem statement.
2. Explain why individual action authorization is insufficient on its own.
3. Identify a realistic initial enterprise customer.
4. Explain what makes SentinelOS different from existing authorization and workflow systems.
5. Develop a hypothetical B2B pricing model.
6. Create an AWS cost-estimation worksheet using measured application usage and published AWS prices.
7. Draft the Devpost submission.
8. Write the four-minute presentation script.
9. Create a quality-assurance checklist.
10. Track bugs, blockers, and completion status.
11. Coordinate mentor feedback.
12. Prepare the final demonstration.

BUSINESS POSITIONING

Target customer:
Companies deploying AI agents into warehouse, logistics, and operational workflows.

Core proposition:
Organizations can validate AI-generated plans before execution while maintaining human oversight.

Do not claim that SentinelOS replaces Amazon Bedrock AgentCore, Amazon Verified Permissions, or existing industrial safety systems.

Explain that SentinelOS combines established authorization services with a domain-specific plan simulator and automated replanning.

PRESENTATION

Create a six-slide presentation:

1. The problem
2. SentinelOS solution
3. Why individual permissions are not sufficient
4. AWS architecture
5. Live demonstration
6. Business model and future applications

The centerpiece is the warehouse capacity-violation demonstration.

QA

Create a checklist covering:

- Successful plan execution
- Failed capacity simulation
- Successful replanning
- Human approval
- Human denial
- Agent revocation
- Invalid inputs
- Version conflicts
- Duplicate requests
- Missing AWS responses
- Demo reset
- Refresh during an active workflow

Record whether each test actually passed.

Never mark features complete based solely on what an engineer reports.

Judge-facing descriptions must distinguish implemented functionality from proposed future functionality.

FIRST TASK

Develop the product positioning, target customer profile, and six-slide presentation outline.

Then create the QA checklist.

Do not invent product metrics, pricing validation, performance benchmarks, or measured AWS costs.

Masha's checkpoints

Time	Required result
12:00 PM	Product positioning finalized
2:00 PM	Presentation outline complete
4:00 PM	Business model drafted
6:00 PM	First full QA pass
8:30 PM	Mentor feedback incorporated
11:00 PM	Devpost draft ready

12:30 AM	Demo narrative finalized
3:00–5:30 AM	Final submission materials
7:30 AM	Presentation rehearsal
8:30 AM	Submission completed

PART 5 — HOW ALL FOUR PEOPLE WORK TOGETHER

This is where hackathon teams often run into problems.

Even if everyone completes their assigned work, the application can fail if the components don't connect properly.

5.1 Who depends on whom?

Component	Owner	Depends on
Bedrock planning	Tishneet	Shared plan schema
Simulation	Mariam	Shared plan schema
Authorization	Mariam	Agent identity and action schema
Database	Mariam	Shared warehouse schema
Dashboard	Ishani	API contracts
API routes	Ishani	Backend functions from Tishneet and Mariam
Step Functions	Tishneet	Working backend interfaces
Business presentation	Masha	Actual implemented features
Testing	Masha	Everyone's integrated work

The most important handoff

Tishneet and Mariam must agree on the plan JSON format before coding.

Tishneet generates the plan.

Mariam validates and simulates it.

Ishani displays the result.

If those three components expect different formats, integration will fail.

PART 6 — SHARED DATA CONTRACTS

Agree on these structures at kickoff.

1. AI-generated plan

```
{
  "actions": [
    {
      "type": "move_packages",
      "from": "A",
      "to": "B",
      "count": 5
    }
  ]
}
```

2. Authorization result

```
{
  "allowed": true,
  "decisions": [
    {
      "action_index": 0,
      "allowed": true,
      "reason": "Authorized by policy"
    }
  ]
}
```

3. Failed simulation

```
{
  "passed": false,
  "reason": "Zone B capacity exceeded",
```

```
  "violations": [
    {
      "type": "CAPACITY_EXCEEDED",
      "zone": "B",
      "projected": 13,
      "capacity": 10
    }
  ]
}
```

4. Successful simulation

```
{
  "passed": true,
  "projected_state": {
    "A": 15,
    "B": 10,
    "C": 8
  },
  "warehouse_version": 1
}
```

This example represents the revised two-to-B, three-to-C transfer starting from the initial seed state.

5. Plan status

```
{
  "plan_id": "plan-001",
  "status": "AWAITING_APPROVAL",
  "approval_required": true
}
```

The full contract should also define the permitted status values, error structures, and a plan identifier connecting attempts to their approval and audit events.

6. Execution result

```
{
  "plan_id": "plan-001",
  "executed": true,
  "new_warehouse_version": 2
}
```

These examples establish a common direction. Before implementation, the team should finalize one canonical warehouse-state format and use it consistently throughout the application.

PART 7 — TEAM COMMUNICATION RULES

To avoid confusion, use five operating rules.

Rule 1: One owner per subsystem. Other teammates can help, but the designated owner approves changes to their component.

Rule 2: Integration happens every two hours. Don't wait until midnight to discover that the frontend and backend are incompatible.

Rule 3: Don't merge code that fails its tests. A working main branch is more valuable than a large collection of unfinished features.

Rule 4: Every blocked task gets reported immediately. If Bedrock access fails or Verified Permissions configuration is unclear, tell the team rather than spending three hours silently troubleshooting.

Rule 5: Masha can freeze scope. After the core system works, new features should require agreement that they won't threaten the demonstration.

Recommended Git workflow

```
main
 |
 +-- feature/ai-workflow       Tishneet
 |
 +-- feature/security-backend  Mariam
 |
 +-- feature/frontend          Ishani
```

Masha can maintain product documentation independently.

Merge small, tested changes frequently.

PART 8 — WHAT TO DO IF YOU FALL BEHIND

Your team needs to protect the most important demonstration.

Priority 0 — Must work

- Natural language becomes a structured plan.
- The simulator detects the capacity violation.
- Bedrock generates an alternative.
- The alternative passes simulation.
- The dashboard displays both results.

Without these features, SentinelOS loses its main differentiator.

Priority 1 — Strongly recommended

- Real DynamoDB persistence
- Verified Permissions
- Safe execution
- Agent revocation
- Audit logging

Priority 2 — If time allows

- Full Step Functions orchestration
- Callback-based approval
- Advanced dashboard polish
- Additional tests and animations

Step Functions is part of the intended architecture, but it should not prevent your team from submitting a working core product.

Emergency decision points

Time	If this isn't working	Decision
2:00 PM	Local simulator	Stop other backend features and fix it
4:00 PM	Bedrock → simulation → replanning	Prioritize integration over new features
6:00 PM	Verified Permissions	Preserve a tested local authorization fallback
8:30 PM	Step Functions	Reduce workflow complexity
11:00 PM	Human approval callback	Use a simpler backend approval flow

12:30 AM	Complete demo	Freeze features and fix critical problems

Any fallback should be accurately described in the final submission. Don't claim an AWS integration works if it was replaced by a local mock.

PART 9 — THE THREE DEMOS EVERYONE MUST UNDERSTAND

Demo 1: Authorized operation

Instruction: Move three packages from A to C.

Expected result:

Zone	Before	After
A	20	17
B	8	8
C	5	8

The transfer passes authorization and simulation, then executes.

Demo 2: Plan-level failure and replanning

Instruction: Move five packages out of A, preferably into B; other zones are acceptable.

Starting from Demo 1:

Original plan: Five to B.

Failure: B would become 13/10.

Revised plan: Two to B, three to C.

Expected final state:

Zone	Before	After
A	17	12
B	8	10
C	8	11

The plan passes simulation, receives approval, and executes.

Demo 3: Revocation

The administrator revokes the agent.

The operator requests another transfer.

Verified Permissions denies the request.

Warehouse state remains unchanged.

Before judging, Masha should test the demos in this exact sequence, since each depends on the state produced by the previous one.

PART 10 — FINAL TEAM CHECKLIST

Deliverable	Tishneet	Mariam	Ishani	Masha
Shared schema agreed	✅	✅	✅	
Bedrock plan generation	**Owner**			
AI replanning	**Owner**			
Step Functions	**Owner**	Supports		
Simulation engine		**Owner**		
Verified Permissions		**Owner**		

DynamoDB		**Owner**		
Safe execution		**Owner**		
React dashboard			**Owner**	
API integration	Supports	Supports	**Owner**	
Approval interface	Supports	Supports	**Owner**	Tests
Business model				**Owner**
Slides and pitch				**Owner**
Devpost submission				**Owner**
QA and bug tracking	Supports	Supports	Supports	**Owner**
Final demonstration	Supports	Supports	Supports	**Owner**

What each person should do at 11:00 AM

Tishneet: Open the repository, read the agreed plan schema, and begin Bedrock planning.

Mariam: Open the repository, implement `simulate_plan()` in Python, and run the first tests.

Ishani: Open her frontend branch, initialize React, and build the three warehouse zone cards.

Masha: Start the task tracker, confirm the judging requirements, finalize the pitch structure, and schedule the first integration checkpoint.

The first major goal is 4:00 PM Saturday: a judge-like user can enter a natural-language instruction, see a capacity violation detected, and watch Bedrock propose a valid alternative.

Everything else builds on that.

One final caution: Keep all generated implementation code—including scaffolding, tests, deployment templates, and AI-generated code—within the event's permitted coding window. Tonight's planning should remain documentation-only, and the team should follow the organizers' exact interpretation of preparation rules.

The advantage of this plan is that everyone knows what they own, what they must deliver, and when their work connects to the rest of SentinelOS.

🛡️ SENTINELOS — MARIAM'S UPDATED HACKATHON TIMELINE

Security, Simulation & Safe Execution

AWS × A10 Hackathon | November 7–8, 2026

This is your **revised timeline based on the more balanced division of engineering responsibilities**.

The biggest change is that you are no longer responsible for general API Gateway configuration, frontend API handlers, or frontend deployment.

Instead, you can concentrate on the three systems that make SentinelOS secure and technically credible:

1. **Simulation:** Does the complete AI-generated plan violate warehouse constraints?
2. **Authorization:** Is the AI agent allowed to perform the proposed operations?
3. **Safe execution:** Can the validated plan update the warehouse without causing inconsistent data?

You still own the DynamoDB data-access layer because it is directly connected to simulation and execution.

---

1. UPDATED RESPONSIBILITIES

Component	Mariam's responsibility	Priority
Python simulation engine	Full ownership	P0
Simulation testing	Full ownership	P0

DynamoDB data-access functions	Full ownership	P0
Warehouse seed data	Full ownership	P0
Safe execution engine	Full ownership	P0
Version checking	Full ownership	P0
Duplicate-execution protection	Full ownership	P0
Amazon Verified Permissions	Full ownership	P1
Cedar authorization policies	Full ownership	P1
Agent revocation logic	Full ownership	P1
Backend integration testing	Shared with Tishneet	P1
API Gateway routes	**Ishani**	Not yours
React dashboard	**Ishani**	Not yours
AWS Step Functions	**Tishneet**	Not yours

Bedrock planning	**Tishneet**	Not yours
Approval callback orchestration	**Tishneet**	Not yours
QA coordination	**Masha**	Not yours

Your job is to build reliable backend functions that other teammates can call.

---

2. YOUR NEW HOUR-BY-HOUR TIMELINE

SATURDAY, NOVEMBER 7

Time	Your task	Deliverable
11:00–11:30 AM	Review shared contracts and create your branch	Backend structure ready
11:30 AM–12:30 PM	Implement `simulate_plan()`	Working simulation
12:30–1:00 PM	Write simulator tests	Core tests passing
1:00–2:00 PM	DynamoDB tables and data-access layer	Warehouse state stored and retrievable
2:00–3:00 PM	Implement `execute_plan()`	Valid plans update inventory

3:00–3:30 PM	Add version checking	Stale plans rejected
3:30–4:00 PM	Add duplicate-execution protection	Same plan cannot execute twice
4:00–5:00 PM	Configure Verified Permissions and Cedar policies	Authorization decisions working
5:00–5:30 PM	Implement revocation	Revoked agents denied
5:30–6:00 PM	Test authorization and share backend interfaces	Backend ready for integration
6:00–7:00 PM	Integrate functions with Tishneet	Step Functions calls backend correctly
7:00–8:30 PM	Test full planning and simulation pipeline	Replanning demonstration works
8:30–9:30 PM	Implement approval-related execution safeguards	Approved plans revalidated
9:30–10:30 PM	Test stale state, revocation, and duplicate execution	Critical security tests pass
10:30–11:00 PM	Complete backend integration	Full demo functional

11:00 PM–12:30 AM	Fix bugs and complete QA	Backend feature freeze

SUNDAY, NOVEMBER 8

Time	Your task
12:30–3:00 AM	Backend reliability testing with Tishneet
3:00–5:30 AM	**Sleep**
5:30–6:30 AM	Verify deployed backend and AWS integrations
6:30–7:30 AM	Run final demonstrations with team
7:30–8:00 AM	Fix critical problems only
8:00–8:30 AM	Support Masha's final submission
8:30–9:00 AM	Confirm submission and prepare for judging

---

3. PHASE-BY-PHASE INSTRUCTIONS

🟢 PHASE 1 — 11:00 AM–1:00 PM

Build your simulation engine

Your first objective: Make the warehouse simulator work entirely in Python without AWS.

Initial warehouse:

Zone	Inventory	Capacity
A	20	30
B	8	10
C	5	20

Your simulator receives a plan like:

```
{
  "actions": [
    {
      "type": "move_packages",
      "from": "A",
      "to": "B",
      "count": 5
    }
  ]
}
```

Your function must determine whether executing the plan would violate any warehouse constraints.

Tests you must complete

Test	Expected result
A → C, 3 packages	PASS
A → B, 5 packages	FAIL: Capacity
A → B, 2 packages	PASS
A → B, 2 and then 1	FAIL: Cumulative capacity
A → C, -3 packages	FAIL
Unknown destination	FAIL
Move to restricted HAZMAT	FAIL
Insufficient source inventory	FAIL

Important: The simulator must evaluate a copy of the warehouse state and never alter the original.

1:00 PM milestone

✅ Your simulator works locally.

✅ All essential tests pass.

✅ Tishneet has the simulation input/output contract.

At this point, **your most important individual contribution is already functional**.

🟡 PHASE 2 — 1:00–2:00 PM

Set up DynamoDB

Create the database structures needed by your backend.

You are responsible for:

Table 1: `sentinel_warehouse`

Stores the warehouse state and version.

Table 2: `sentinel_agents`

Stores the agent ID and current authorization status.

Table 3: `sentinel_audit`

Stores plan decisions and activity records.

Functions to implement

get_warehouse()

get_agent(agent_id)

update_agent_status(agent_id, status)

record_audit_event(event)

reset_demo()

Coordinate shared AWS infrastructure definitions with Tishneet so that you don't both create conflicting resource definitions.

2:00 PM milestone

✅ DynamoDB tables are available.

✅ Initial warehouse state is seeded.

✅ Python can retrieve warehouse data.

✅ Agent status can be retrieved.

Don't begin advanced security policies until your data-access functions work.

🟠 PHASE 3 — 2:00–4:00 PM

Build safe execution

This is likely the most challenging part of your role.

Your simulator only predicts changes.

Your executor actually updates DynamoDB.

You need to implement:

execute_plan(plan, agent_id, expected_version)

Requirements

1. Recheck authorization

An agent could be revoked between simulation and execution.

2. Recheck the warehouse

Inventory might have changed.

3. Re-run simulation

Even a previously valid plan might no longer satisfy the warehouse constraints.

4. Use version checking

Reject updates based on an outdated warehouse version.

5. Prevent duplicate execution

Submitting or approving the same plan twice must never apply its actions twice.

Example

Two agents both attempt to move two packages into Zone B.

B starts at eight packages.

Both initial simulations pass.

The first execution updates B to ten.

The second execution must fail because its simulation used outdated warehouse data.

4:00 PM milestone

✅ A valid plan updates DynamoDB.

✅ An invalid plan never changes inventory.

✅ A stale plan is rejected.

✅ The same plan cannot execute twice.

This is your second major milestone.

One important implementation detail: the final executor must fail closed if authorization is not available. During local development, you can use a test authorization adapter, but the deployed executor must use the actual agreed authorization mechanism.

🔴 PHASE 4 — 4:00–6:00 PM

Implement Amazon Verified Permissions

Now you're implementing SentinelOS's authorization layer.

Your initial policies are straightforward.

Policy	Requirement
Transfer limit	Maximum 10 packages per action
Restricted destination	HAZMAT prohibited

Revocation	Only ACTIVE agents may transfer packages

Your authorization function

authorize_plan(agent_id, plan)

It should check each action using Verified Permissions.

If any action is denied, the entire plan is denied.

What to test

Test 1: Active agent moves 3 packages.

Expected: ALLOW.

Test 2: Active agent moves 11 packages.

Expected: DENY.

Test 3: Active agent requests HAZMAT.

Expected: DENY.

Test 4: Revoked agent moves 1 package.

Expected: DENY.

Test 5: Verified Permissions is unavailable.

Expected: Safe failure, not authorization.

Revocation

When Ishani's frontend calls the revoke endpoint, your backend changes the agent's status in DynamoDB.

The next authorization request should use the current trusted status.

6:00 PM milestone

✅ Verified Permissions integrated.

✅ Cedar policies working.

✅ Revocation enforced.

✅ Authorization tests passing.

---

4. PHASE 5 — 6:00–8:30 PM

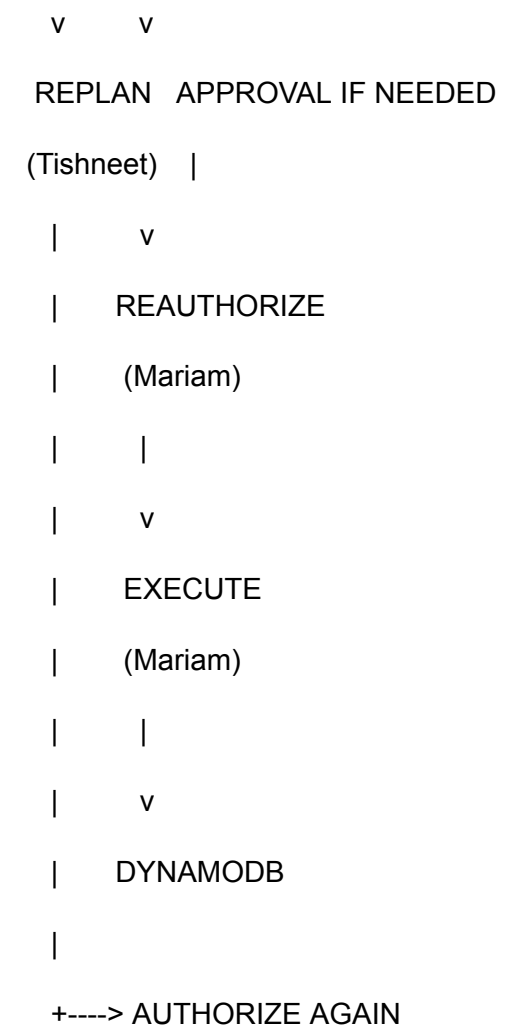
Integration with Tishneet

Your primary responsibility now shifts from independent coding to integration.

Tishneet's Step Functions workflow will call your Lambda functions.

The workflow should follow:

```
BEDROCK
   |
   v
AUTHORIZE
(Mariam)
   |
   v
 SIMULATE
(Mariam)
   |
+----+----+
|         |
FAIL      PASS
|         |
```



What you need to coordinate

With Tishneet:

- Input schema for authorization.
- Input schema for simulation.
- Expected workflow result structures.
- Execution requirements.
- Retry behavior.
- Error outcomes.

With Ishani:

- Warehouse API response format.
- Agent status.
- Authorization-result display.
- Audit-log structure.

With Masha:

- Security test results.
- Demo readiness.
- Remaining bugs.

8:30 PM milestone

The complete planning → authorization → simulation → replanning process works.

---

5. PHASE 6 — 8:30–11:00 PM

Human approval and backend safeguards

Tishneet owns the approval orchestration.

Ishani owns the interface.

You own the final safety checks.

After someone clicks Approve, your executor must verify:

1. The agent is still active and authorized.
2. The approved plan matches the plan being executed.
3. The warehouse has not changed incompatibly.
4. The plan still passes simulation.
5. The operation has not already been executed.

Special test: Revocation during approval

1. Generate a valid five-package plan.
2. Let it reach `AWAITING_APPROVAL`.
3. Revoke the agent.
4. Click Approve.
5. Confirm no inventory changes.

This is one of your strongest security demonstrations.

11:00 PM milestone

✅ Approved plans execute safely.

✅ Revoked agents are blocked.

✅ Invalid plans never execute.

✅ Duplicate approvals do not duplicate transfers.

---

6. WHAT IS NO LONGER YOUR JOB?

This is the most important change from your original plan.

❌ You do not own:

- Creating the React application.
- Managing Tailwind or Vite.
- Implementing general API Gateway routes.
- Writing frontend API clients.
- Configuring Amplify Hosting.
- Building the Step Functions state machine.
- Implementing Bedrock generation or replanning.
- Creating the presentation.
- Managing the Devpost submission.

✅ You still own:

- All simulation logic.
- Simulation tests.
- DynamoDB warehouse and agent data access.
- Execution validation and atomic updates.
- Cedar policies.
- Verified Permissions integration.
- Agent revocation logic.
- Backend authorization and execution tests.

You are now focused on backend correctness and security, rather than also managing the application's infrastructure integration.

---

7. YOUR UPDATED CHECKLIST

Time	Task	Done
11:00 AM	Review shared contracts	☐
11:30 AM	Begin simulation engine	☐
12:00 PM	Finish core simulation rules	☐
12:30 PM	Complete simulation tests	☐
1:00 PM	Share simulator interface	☐
1:30 PM	Create DynamoDB tables	☐
2:00 PM	Seed warehouse and agent data	☐
2:30 PM	Implement executor	☐
3:00 PM	Add version checking	☐
3:30 PM	Add idempotency	☐
4:00 PM	Test safe execution	☐
4:30 PM	Configure Cedar policies	☐

5:00 PM	Implement Verified Permissions calls	☐
5:30 PM	Implement revocation	☐
6:00 PM	Complete security tests	☐
7:00 PM	Connect to Step Functions	☐
8:30 PM	Test full AI/simulation flow	☐
9:30 PM	Test approval security	☐
10:30 PM	Fix critical backend failures	☐
11:00 PM	Finalize backend functionality	☐
12:30 AM	Feature freeze	☐
3:00 AM	Begin sleep period	☐
5:30 AM	Final testing	☐
8:30 AM	Submission support	☐

---

8. YOUR THREE PRIORITIES IF YOU FALL BEHIND

If something takes longer than expected, prioritize in this order.

Priority 1 — Simulator

Must work.

Without the simulator, SentinelOS loses its central innovation.

Priority 2 — Safe execution and database

Must work for a full operational demonstration.

This proves that the system manages real application state rather than only displaying AI-generated responses.

Priority 3 — Verified Permissions

Strongly recommended.

This demonstrates actual authorization enforcement and supports the revocation scenario.

If Verified Permissions integration is delayed, preserve the working simulator and executor. A local authorization adapter can support development, but it must be clearly labeled as a fallback rather than described as a completed AWS integration.

FINAL RECOMMENDATION

Your revised role is more manageable because **Ishani handles the frontend-facing API infrastructure and Tishneet handles orchestration**.

You can spend your time on the systems where correctness matters most.

Your personal milestones

Deadline	Required accomplishment

1:00 PM Simulator and tests complete

4:00 PM Safe DynamoDB execution working

6:00 PM Verified Permissions and revocation working

8:30 PM Integrated planning and simulation workflow

11:00 PM Approval-related security checks complete

12:30 AM Backend feature freeze

Your most important goal is to have the simulator working by 1:00 PM and the safe executor working by 4:00 PM.

🧠 SENTINELOS — TISHNEET'S UPDATED HACKATHON TIMELINE

AI Planning, Replanning & AWS Workflow Orchestration

AWS × A10 Hackathon | November 7–8, 2026

This is Tishneet's **revised timeline based on the more balanced distribution of responsibilities between the three engineers**.

The biggest change is that **Tishneet is no longer expected to handle general API infrastructure or other engineers' AWS resources**.

Instead, she will concentrate on two major systems:

1. **Amazon Bedrock:** Generating structured plans and revising them when simulation detects a problem.
2. **AWS Step Functions:** Coordinating how plans move through authorization, simulation, replanning, approval, and execution.

She will also act as the **AWS infrastructure integration coordinator**, ensuring the services deployed by each engineer connect correctly. This is a coordination responsibility, not an obligation to build everyone's infrastructure.

---

1. UPDATED RESPONSIBILITIES

Component	Tishneet's responsibility	Priority
Amazon Bedrock integration	Full ownership	P0
Natural-language plan generation	Full ownership	P0
Plan JSON schema validation	Full ownership	P0
AI replanning	Full ownership	P0
Three-attempt planning limit	Full ownership	P0

Bedrock error handling	Full ownership	P0
Step Functions state machine	Full ownership	P1
Workflow branching and retries	Full ownership	P1
Human approval callback	Full ownership	P1
Approval timeouts	Full ownership	P1
Lambda workflow integration	Shared with Mariam	P1
AWS infrastructure coordination	Coordination only	P1
DynamoDB	**Mariam**	Not hers
Simulation and execution	**Mariam**	Not hers
Verified Permissions	**Mariam**	Not hers
API Gateway and API handlers	**Ishani**	Not hers
React and Amplify	**Ishani**	Not hers
Devpost, presentation, QA	**Masha**	Not hers

Your role in one sentence

Tishneet builds the AI brain and the workflow that coordinates the rest of SentinelOS.

---

2. YOUR NEW HOUR-BY-HOUR TIMELINE

SATURDAY, NOVEMBER 7

Time	Task	Deliverable
11:00–11:30 AM	Review shared contracts, create branch, confirm AWS credentials	AI environment ready
11:30 AM–12:00 PM	Test Bedrock model invocation	First successful model response

12:00–1:00 PM	Implement `generate_plan()`	Natural language converted to structured JSON
1:00–2:00 PM	Add schema validation and planner tests	Reliable, validated plans
2:00–3:00 PM	Implement `replan()`	Capacity failures produce alternative plans
3:00–3:30 PM	Add three-attempt limit	Planning loop bounded
3:30–4:00 PM	Integrate with Mariam's simulator	Complete AI → simulation → replanning loop
4:00–5:00 PM	Define Step Functions state machine	Workflow structure complete
5:00–6:00 PM	Connect planning, authorization, and simulation Lambda functions	Basic workflow working
6:00–7:00 PM	Add simulation-failure branching and replanning	Workflow automatically retries plans
7:00–8:30 PM	Integrate safe execution and audit outcomes	Low-risk plan completes end-to-end
8:30–9:30 PM	Implement human approval callback	Workflow can pause for approval
9:30–10:30 PM	Connect approval with backend and frontend	Approve/Deny works
10:30–11:00 PM	Test timeouts, reauthorization, and execution	Approval workflow reliable
11:00 PM–12:30 AM	Fix critical errors and run full integration tests	Workflow feature freeze

SUNDAY, NOVEMBER 8

Time	Task
12:30–3:00 AM	Backend reliability testing with Mariam
3:00–5:30 AM	**Sleep**
5:30–6:30 AM	Verify Bedrock and deployed Step Functions
6:30–7:30 AM	Run the three judging demonstrations

7:30–8:00 AM	Fix critical problems only
8:00–8:30 AM	Support Masha's final submission
8:30–9:00 AM	Confirm successful submission

---

3. PHASE 1 — 11:00 AM–12:00 PM

Set Up Bedrock and Make the First AI Call

Your first objective: Successfully communicate with Amazon Bedrock.

Step 1: Start from the shared repository

Once the hackathon begins, use the shared GitHub repository.

Create a branch:

`feature/ai-workflow`

Do not independently scaffold the entire SentinelOS application.

Step 2: Confirm your AWS environment

Verify:

- AWS credentials work.
- The correct AWS Region is configured.
- The selected Bedrock model is available.
- Your account has permission to invoke the model.

The team initially discussed Nova Pro and Claude Sonnet.

Choose the model that is available and reliably supports your intended output format.

Don't spend hours trying to use a particular model if another suitable model is already available.

Step 3: Understand the Converse API

Amazon Bedrock's Converse API lets your Python backend send messages to a foundation model and receive generated responses.

For SentinelOS:

Input:

"Move three packages from Zone A to Zone C."

Expected output:

```
{
  "actions": [
    {
      "type": "move_packages",
      "from": "A",
      "to": "C",
      "count": 3
    }
  ]
}
```

Your 12:00 PM milestone

✅ Bedrock responds successfully.

✅ The response can be read by Python.

✅ You know which model and Region the team will use.

Do not begin Step Functions yet.

---

4. PHASE 2 — 12:00–2:00 PM

Build the AI Planning Engine

Now build the function responsible for translating instructions into plans.

Your main function

generate_plan(instruction, warehouse_state)

Input

The operator's instruction and current warehouse state.

Output

A validated JSON plan containing supported actions.

Shared warehouse state

Zone	Inventory	Capacity
A	20	30
B	8	10
C	5	20

Supported action

For the MVP, support only:

`move_packages`

Each action must specify:

- Source zone
- Destination zone
- Package count

Schema requirements

Field	Validation
`actions`	Nonempty array
`type`	Must equal `move_packages`
`from`	Must be a recognized zone
`to`	Must be a recognized zone
`count`	Positive integer
Additional fields	Rejected unless specified

Your model may use Bedrock's structured-output features when supported.

However, **you still need Python validation after receiving the response**.

Test these instructions

Instruction	Expected behavior
Move 3 packages A → C	One valid action
Move 5 packages A → B	One valid action
Move 2 packages B → C	One valid action
Move 5 packages out of A, preferably to B	Plan reflects operator's preference
Request an unsupported operation	Controlled rejection

Your 2:00 PM milestone

✅ `generate_plan()` works.

✅ Output passes schema validation.

✅ Three different valid instructions generate usable plans.

✅ Mariam receives the finalized plan format.

---

5. PHASE 3 — 2:00–4:00 PM

Implement Replanning and Integrate the Simulator

This is your most important contribution to SentinelOS's central innovation.

Your main function

replan(original_goal, failed_plan, rejection_reason, warehouse_state)

The scenario

The operator asks:

"Move five packages out of A, preferably into B. Use another zone if necessary."

The AI initially proposes moving all five packages into B.

Mariam's simulator calculates:

Zone B would contain 13 packages, exceeding its capacity of 10.

It rejects the plan.

Your code must send Bedrock:

- The original instruction.
- The failed plan.
- The simulator's rejection reason.
- The current warehouse state.
- The instruction to generate a valid alternative.

Expected revised plan

```
{
  "actions": [
    {
      "type": "move_packages",
      "from": "A",
      "to": "B",
      "count": 2
    },
    {
      "type": "move_packages",
      "from": "A",
      "to": "C",
      "count": 3
    }
  ]
}
```

The revised plan must then be sent to Mariam's authorization and simulation functions.

The AI cannot approve its own plan.

Implement the attempt limit

Allow up to **three total candidate plans**, including the initial plan.

Attempt	Action
1	Generate original plan
2	Revise after rejection

3	Final revision
All fail	Stop without executing

Important distinction

Operational constraint failure: Replanning may be appropriate.

Authorization denial: The workflow should stop rather than asking Bedrock to bypass a security restriction.

Test both paths

Test A: An initial plan passes simulation.

Expected: No replanning.

Test B: Initial plan fails simulation.

Expected: Bedrock generates a revised candidate.

Your 4:00 PM milestone

✅ AI planning works.

✅ Replanning works.

✅ Three-attempt limit works.

✅ Mariam's simulator accepts your generated plan structure.

✅ The complete AI → simulator → AI feedback loop works.

This is your most important technical checkpoint.

---

6. PHASE 4 — 4:00–6:00 PM

Build the Step Functions Workflow

Now transition from AI development to AWS orchestration.

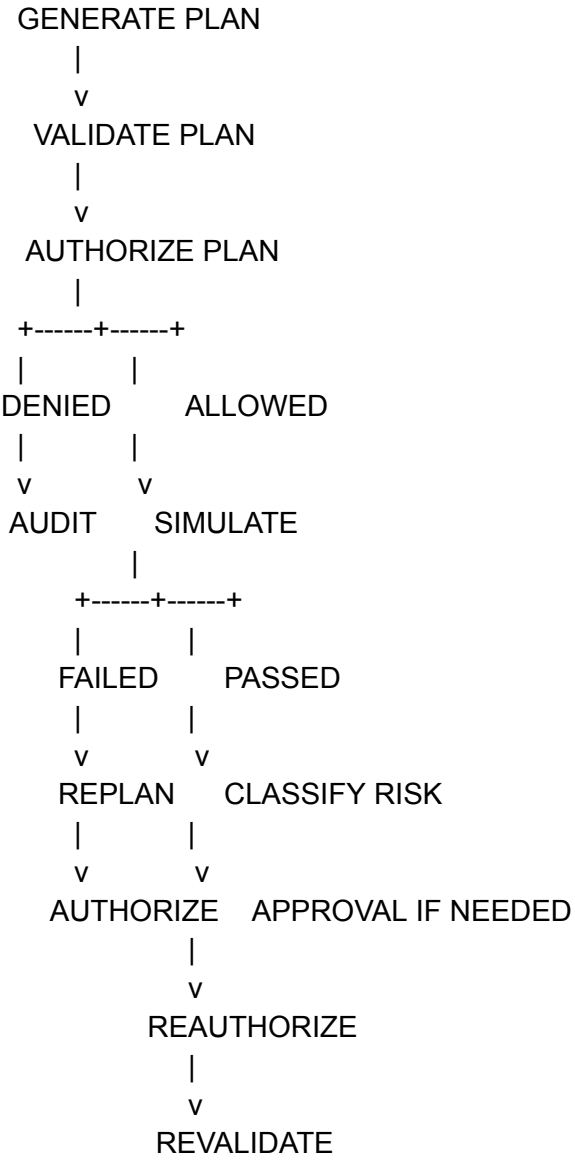
What is Step Functions doing?

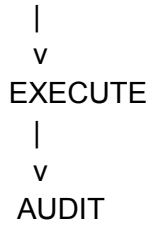
Your application contains several independent backend operations.

Step Functions decides:

- Which function executes next.
- What happens when simulation fails.
- When replanning should begin.
- Whether human approval is required.
- When execution is allowed.
- How failures are handled.

Your workflow





Step 1: Define states

Create the Step Functions state-machine definition.

Use the shared Lambda contracts.

Step 2: Connect Bedrock planning

Step Functions calls your planning Lambda.

It receives a structured plan.

Step 3: Connect Mariam's authorization function

The workflow passes the plan to Mariam's authorization Lambda.

If authorization is denied, the workflow stops safely.

Step 4: Connect Mariam's simulator

If authorization passes, the simulator evaluates the plan.

Step 5: Branch based on simulation results

If simulation passes, continue.

If simulation fails, invoke the replanning function when attempts remain.

Critical requirement

Preserve workflow data between states.

Step Functions must retain:

- Original instruction
- Plan identifier
- Agent identifier
- Current candidate plan

- Attempt count
- Warehouse version
- Simulation result

If one Lambda response accidentally replaces the entire workflow state, later steps may lose important information.

Your 6:00 PM milestone

✅ Step Functions definition is valid.

✅ Planning, authorization, and simulation are connected.

✅ A workflow can successfully process a basic plan.

---

7. PHASE 5 — 6:00–8:30 PM

Complete Workflow Integration

Now make the full backend process run automatically.

6:00–7:00 PM — Replanning loop

When simulation fails:

1. Check whether another planning attempt is available.
2. Send the failure reason to Bedrock.
3. Generate a revised plan.
4. Validate the new JSON.
5. Reauthorize the new actions.
6. Run simulation again.

If three candidate plans fail, the workflow ends.

7:00–8:30 PM — Safe execution

Connect the valid-plan path to Mariam's executor.

The workflow must ensure:

1. Authorization passed.

2. Simulation passed.
3. Any required human approval has been granted.
4. The agent is reauthorized before execution.
5. The warehouse state is revalidated.
6. The executor commits the plan safely.

Who owns what?

Component	Owner
AI generation	Tishneet
AI replanning	Tishneet
Step Functions	Tishneet
Authorization	Mariam
Simulation	Mariam
Execution	Mariam
API Gateway	Ishani
Frontend	Ishani

Don't rewrite Mariam's security or simulation code. Connect to her existing functions.

Your 8:30 PM milestone

✅ Low-risk plans execute through Step Functions.

✅ Invalid plans can trigger replanning.

✅ Unauthorized plans are denied.

✅ Warehouse state updates after successful execution.

---

8. PHASE 6 — 8:30–11:00 PM

Implement Human Approval

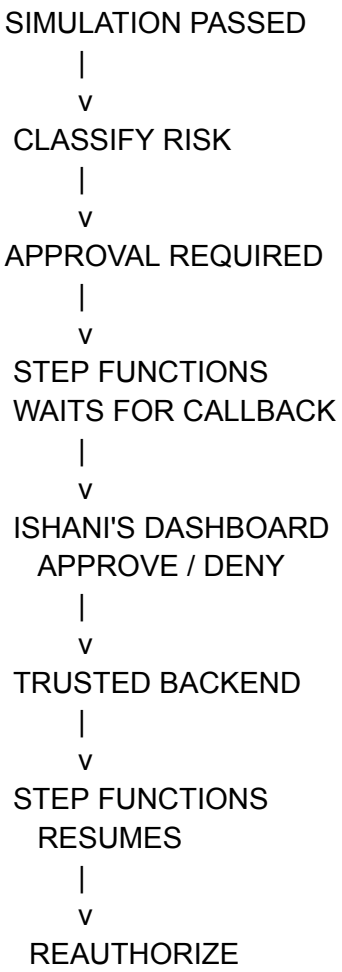
Now build the ability to pause a workflow until an administrator approves it.

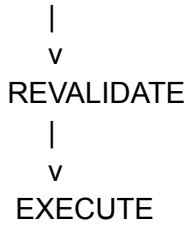
Approval threshold

For the MVP:

Transfer size	Result
Fewer than 5 packages total	No approval required
5 or more packages total	Human approval required
Invalid plan	Reject
Unauthorized plan	Deny

How approval works





What is your responsibility?

You implement the Step Functions callback state.

This includes:

- Creating the waiting task.
- Managing the task token through the trusted backend.
- Handling callback completion.
- Handling approval and denial branches.
- Configuring timeouts.

What Mariam does

Mariam ensures that the backend:

- Verifies the approval request.
- Preserves the approved plan identity.
- Rechecks authorization.
- Revalidates warehouse state.
- Prevents duplicate execution.

What Ishani does

Ishani displays:

- The proposed plan.
- Simulation result.
- Approval status.
- Approve and Deny buttons.

The browser never receives the Step Functions task token.

Approval tests

Test 1: Valid plan, administrator approves.

Expected: Workflow resumes and executes.

Test 2: Valid plan, administrator denies.

Expected: Workflow ends without executing.

Test 3: Agent revoked during approval.

Expected: Execution blocked after the workflow resumes.

Test 4: Approval times out.

Expected: Workflow fails safely without changing inventory.

Your 11:00 PM milestone

✅ Step Functions pauses correctly.

✅ Approval resumes the workflow.

✅ Denial ends the workflow.

✅ Reauthorization occurs before execution.

✅ Timeouts are handled.

---

9. PHASE 7 — 11:00 PM–12:30 AM

Reliability Testing and Feature Freeze

At this point, do not add new AWS services.

Your focus is making the application work consistently.

Critical tests

Test	Expected result
Normal warehouse transfer	Executes
Capacity violation	Triggers replanning
Revised plan passes	Workflow proceeds

Three failed candidates	Workflow stops
Authorization denial	Stops immediately
Revoked agent	Denied
Approval required	Workflow pauses
Administrator approves	Workflow resumes
Administrator denies	Workflow ends
Approval timeout	Fails safely
Bedrock API error	Controlled failure
Lambda error	Recorded
Invalid Bedrock JSON	Rejected
Duplicate callback	No duplicate execution

Your 12:30 AM milestone

The workflow is stable enough for judging.

Document:

- Completed AWS integrations.
- Tests that passed.
- Known bugs.
- Remaining risks.
- Any fallback mechanisms.

Send the status to Masha.

---

10. OVERNIGHT SCHEDULE

12:30–3:00 AM — Backend testing

Work with Mariam.

Run the three judging demonstrations repeatedly.

Focus on:

- Workflow correctness
- Bedrock reliability
- Replanning
- Approval
- Execution errors
- AWS permissions
- Agent revocation

3:00–5:30 AM — Sleep

Hand off outstanding issues before sleeping.

Do not leave uncommitted changes that other teammates need.

5:30–8:30 AM — Final checks

Verify:

- Bedrock responds.
- Step Functions executes correctly.
- Approval works.
- The deployed workflow is the tested version.
- The team can reset and repeat the three demos.

Support Masha's final submission.

---

11. YOUR UPDATED PERSONAL CHECKLIST

Time	Task	Done
11:00 AM	Review shared contracts	☐
11:30 AM	Confirm Bedrock access	☐
12:00 PM	Complete first model invocation	☐
12:30 PM	Implement `generate_plan()`	☐

1:00 PM	Add JSON schema validation	☐
1:30 PM	Test several operator instructions	☐
2:00 PM	Share plan contract with Mariam	☐
2:30 PM	Implement `replan()`	☐
3:00 PM	Test revised plans	☐
3:30 PM	Add three-attempt limit	☐
4:00 PM	Complete AI/simulator integration	☐
4:30 PM	Define Step Functions states	☐
5:00 PM	Validate state-machine definition	☐
5:30 PM	Connect backend Lambda functions	☐
6:00 PM	Execute basic workflow	☐
7:00 PM	Implement workflow replanning loop	☐
8:30 PM	Complete low-risk execution workflow	☐
9:00 PM	Add human approval callback	☐
9:30 PM	Connect approval handling	☐
10:00 PM	Test Approve and Deny	☐
10:30 PM	Test timeouts and reauthorization	☐
11:00 PM	Complete approval workflow	☐
12:30 AM	Feature freeze	☐
3:00 AM	Begin sleep period	☐
5:30 AM	Final integration testing	☐
7:30 AM	Confirm judging demos	☐
8:30 AM	Support submission	☐

---

12. WHAT IS NO LONGER YOUR JOB?

❌ Tishneet does not own:

- DynamoDB table creation.
- Warehouse data-access functions.
- Warehouse simulation.
- Safe execution logic.
- Verified Permissions policies.
- Agent revocation implementation.
- React components.
- API Gateway routes.
- Frontend API handlers.
- Amplify deployment.
- Business research.
- Devpost preparation.

✅ Tishneet still owns:

- Bedrock model integration.
- AI-generated plans.
- Replanning.
- Plan validation.
- Planning retry limits.
- Step Functions state machine.
- Approval callback orchestration.
- Workflow errors and timeouts.
- Coordinating AWS infrastructure interfaces.

Infrastructure coordination does not mean building every resource

Your responsibility is to ensure Mariam's and Ishani's AWS components work with Step Functions.

For example, you need to know the deployed Lambda ARNs and correct IAM permissions.

But you should not independently recreate their resources.

---

13. YOUR THREE PRIORITIES IF YOU FALL BEHIND

Priority 1 — Bedrock planning

Must work.

Natural-language instructions must become valid structured plans.

Priority 2 — AI replanning

Must work.

The AI must respond to simulation failures by proposing alternative actions.

This is essential to the project's central innovation.

Priority 3 — Step Functions

Strongly recommended.

Build the core workflow before implementing advanced approval callbacks.

If Step Functions becomes the bottleneck, preserve the working Bedrock + simulator integration using a simpler temporary coordinator, and accurately describe that fallback during judging.

---

14. YOUR THREE MOST IMPORTANT MILESTONES

Deadline	Required accomplishment
2:00 PM	Bedrock produces structured, validated plans
4:00 PM	Bedrock replans successfully using Mariam's simulator feedback
8:30 PM	Step Functions coordinates the main workflow

11:00 PM Human approval works

12:30 AM Workflow feature freeze

FINAL RECOMMENDATION

Tishneet's updated role is more focused and balanced.

Her workload is still challenging because AWS Step Functions and human approval callbacks introduce significant integration complexity.

However, she no longer needs to manage API Gateway, DynamoDB, React, or the general backend.

Tishneet's central objective

Make SentinelOS capable of generating an operational plan, receiving feedback when it fails, and automatically proposing an acceptable alternative.

Once that works, her Step Functions workflow connects the result to authorization, human approval, and execution.

At 11:00 AM, begin with Bedrock. By 4:00 PM, aim to have working replanning. By 8:30 PM, the complete workflow should be connected.

That gives Tishneet a substantial engineering role without overloading her with responsibilities belonging to Mariam or Ishani.

💻 SENTINELOS — ISHANI'S UPDATED HACKATHON TIMELINE

Frontend Development, API Gateway, Backend API Integration & Deployment

AWS × A10 Hackathon | November 7–8, 2026

Your role: Frontend & API Engineer
Your teammates: Mariam (security, simulation, and data), Tishneet (AI and workflow), Masha (product and QA)
Primary technologies: React, Vite, Tailwind CSS, JavaScript, Python, AWS API Gateway, AWS Lambda, AWS Amplify Hosting
Hacking begins: Saturday, 11:00 AM
Submission deadline: Sunday, 9:00 AM
Internal target: Sunday, 8:30 AM

This is Ishani's **revised timeline based on the more balanced engineering workload**.

The biggest change is that **Ishani now takes full ownership of the frontend-facing API infrastructure**, rather than just building React and connecting it to endpoints someone else creates.

She is responsible for making the entire application usable through one consistent interface.

---

1. UPDATED RESPONSIBILITIES

Component	Ishani's responsibility	Priority
React dashboard	Full ownership	P0
Warehouse visualization	Full ownership	P0
AI instruction console	Full ownership	P0
Original/revised plan comparison	Full ownership	P0

Frontend API client (`api.js`)	Full ownership	P0
API Gateway configuration	Full ownership	P0
Frontend-facing Lambda API handlers	Full ownership	P0
Plan submission endpoint	API ownership; coordinate workflow start with Tishneet	P0
Workflow status endpoint	API ownership; coordinate status data with Tishneet	P0
Warehouse and agent endpoints	API ownership; call Mariam's data functions	P0
Audit-log integration	Full frontend/API ownership	P1
Agent revocation interface	Frontend/API ownership	P1
Human approval interface	Frontend/API ownership	P1
Approval request endpoint	API ownership; trusted approval logic shared with Tishneet and Mariam	P1
Amplify Hosting	Full ownership	P1
API error handling	Full ownership	P1
Frontend integration tests	Full ownership	P1
Bedrock planning and replanning	**Tishneet**	Not hers
Step Functions state machine	**Tishneet**	Not hers
Simulator and safe execution	**Mariam**	Not hers
Verified Permissions policies	**Mariam**	Not hers
Database schema and core data access	**Mariam**	Not hers
Business model and final QA	**Masha**	Not hers

Your role in one sentence

Ishani builds the application interface and the API layer connecting users to SentinelOS's AWS services.

You have two major engineering responsibilities:

1. **Frontend:** Everything the operator sees and interacts with.
2. **API:** The trusted HTTP entry points that connect React to the backend.

---

2. UPDATED HOUR-BY-HOUR TIMELINE

SATURDAY, NOVEMBER 7

Time	Task	Deliverable
11:00–11:30 AM	Review shared contracts and create frontend branch	Project structure ready
11:30 AM–12:00 PM	Initialize React, Vite, Tailwind	Application running
12:00–1:00 PM	Build warehouse dashboard	Three functional zone cards
1:00–2:00 PM	Build AI console and plan comparison	Complete mock-data interface
2:00–3:00 PM	Build frontend API client and HTTP Lambda handlers	API structure ready
3:00–4:00 PM	Configure API Gateway and connect plan submission/status	Real backend communication
4:00–5:00 PM	Connect warehouse, audit, and agent endpoints	Real warehouse/security data displayed
5:00–6:00 PM	Build revocation UI and test API permissions	Agent revocation works
6:00–7:00 PM	Connect Step Functions progress	Live workflow statuses
7:00–8:30 PM	Build and test detailed plan comparison	Replanning results visualized

8:30–9:30 PM	Build approval modal and backend approval route	Approval interface connected
9:30–10:30 PM	Integrate Approve/Deny with Step Functions callback	End-to-end approval
10:30–11:00 PM	Test full workflow and fix interface bugs	Complete interactive demo
11:00 PM–12:30 AM	Deploy to Amplify, run QA, feature freeze	Deployed application

SUNDAY, NOVEMBER 8

Time	Task
12:30–3:00 AM	**Sleep**
3:00–4:00 AM	UI polish and responsiveness
4:00–5:00 AM	Screenshot capture, demo recording, deployment verification
5:00–5:30 AM	Deliver presentation assets to Masha
5:30–6:30 AM	Final frontend and API testing
6:30–7:30 AM	Full-team demonstration rehearsal
7:30–8:00 AM	Fix critical bugs only
8:00–8:30 AM	Support Devpost submission
8:30–9:00 AM	Confirm successful submission

Your two most important milestones are 2:00 PM and 4:00 PM.

By 2:00 PM, the application should look complete using mock data.

By 4:00 PM, it should communicate with the actual backend.

---

3. PHASE 1 — 11:00 AM–12:00 PM

Set Up React and the Frontend Structure

Your objective

Build the frontend foundation without waiting for the backend.

After the event's official coding start time, create your branch:

`feature/frontend-api`

Use the team's existing GitHub repository.

Do not create a second SentinelOS project.

Frontend technologies

React: Builds interactive user-interface components.

Vite: Creates the development environment and production build.

Tailwind CSS: Provides styling utilities.

JavaScript: Handles frontend interactions and API calls.

Suggested file structure

```
frontend/
├── src/
│   ├── App.jsx
│   ├── components/
│   │   ├── Header.jsx
│   │   ├── WarehouseOverview.jsx
│   │   ├── ZoneCard.jsx
│   │   ├── AgentConsole.jsx
│   │   ├── PlanComparison.jsx
│   │   ├── WorkflowStatus.jsx
│   │   ├── ApprovalModal.jsx
│   │   ├── AuditLog.jsx
│   │   └── AgentControls.jsx
│   ├── services/
│   │   └── api.js
│   └── index.css
├── package.json
└── vite.config.js
```

I also recommend putting your API-facing code in a separate backend folder:

```
backend/
└── api/
    ├── submit_plan.py
    ├── get_plan.py
    ├── get_warehouse.py
    ├── get_audit.py
    ├── approve_plan.py
    ├── revoke_agent.py
    └── reset_demo.py
```

These are suggested modules, not necessarily separate Lambda deployments. To save time, several endpoints can share one Lambda handler.

Your 12:00 PM milestone

✅ React runs locally.

✅ Tailwind works.

✅ Dashboard layout exists.

✅ Frontend branch is ready.

---

4. PHASE 2 — 12:00–2:00 PM

Build the Complete Dashboard

At this stage, use mock data following the shared JSON contracts.

Your goal is to create a convincing interface before connecting AWS.

4.1 Warehouse cards

Initial warehouse:

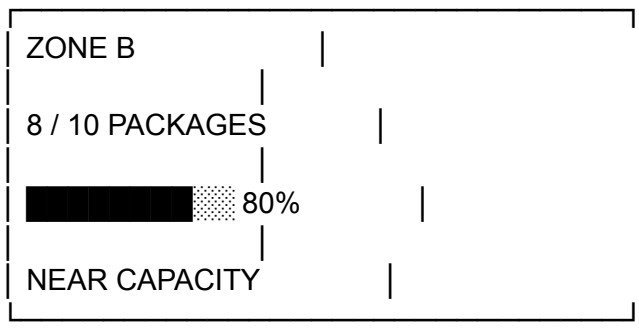
Zone	Inventory	Capacity
A	20	30
B	8	10

C 5 20

Each card displays:

- Zone name
- Current inventory
- Maximum capacity
- Utilization percentage
- Progress bar
- Status color

Example:



Use a dark, professional enterprise dashboard.

Suggested visual language:

Color	Meaning
Green	Valid or completed
Amber	Pending approval or near capacity
Red	Rejected or unauthorized
Blue	Processing
Gray	Not started

4.2 AI instruction console

Build an input field where users can write:

"Move three packages from Zone A to Zone C."

Or:

"Move five packages out of Zone A, preferably into Zone B. Use another zone if necessary."

Include:

- Text input
- Submit button
- Loading state
- Error message
- Current plan status

For now, submitting the instruction can display mock results.

4.3 Original vs. revised plan comparison

This is the most important part of your dashboard.

Show two panels side by side.

Original plan

Move five packages from A to B.

Result: ❌ Rejected.

Zone B would contain 13 packages, exceeding its maximum capacity of 10.

Revised plan

Move two packages from A to B.

Move three packages from A to C.

Result: ✅ Simulation passed.

The projected inventory respects all zone capacities.

Important distinction

Always distinguish between:

Actual state: What's currently stored in DynamoDB.

Projected state: What would happen if a plan executed.

A rejected plan must never appear to have changed real inventory.

Your 2:00 PM milestone

✅ Warehouse cards complete.

✅ AI console complete.

✅ Plan comparison complete.

✅ Dashboard looks presentable.

✅ Interface runs using mock data.

At this point, you should be able to show the frontend to Masha and receive feedback.

---

5. PHASE 3 — 2:00–4:00 PM

Build the API Layer

This is the biggest change from your previous responsibilities.

You now own the **frontend-facing API infrastructure**.

5.1 What is API Gateway?

API Gateway lets a React application send HTTP requests to backend AWS services.

For example:

```
React Dashboard
      |
      | POST /plans
      v
API Gateway
      |
      v
Lambda API Handler
      |
      v
Start Step Functions Workflow
```

The API Gateway route forwards the request to a backend handler.

That handler validates the request and calls the appropriate trusted backend operation.

5.2 Your seven endpoints

Endpoint	Method	Your implementation responsibility
`/plans`	POST	Validate request and start workflow
`/plans/{id}`	GET	Retrieve workflow status
`/warehouse`	GET	Return current warehouse data
`/audit`	GET	Return audit records
`/approvals/{id}`	POST	Forward authorized approval decisions
`/agents/{id}/revoke`	POST	Forward authorized revocation requests
`/demo/reset`	POST	Reset the demo environment through a trusted backend operation

Security boundary

Although you own the HTTP handlers, **you do not implement the core authorization rules or execution logic**.

Those remain Mariam's responsibility.

For example:

The browser calls:

```
POST /agents/inventory-agent-01/revoke
```

Your API handler must authenticate and authorize the administrator, then invoke Mariam's trusted revocation function.

It must not simply trust any browser request to revoke arbitrary agents.

For hackathon deployment, agree on a practical admin authentication method with Mariam. Never put a shared administrative secret in public frontend code.

5.3 Build the frontend API client

Create:

`frontend/src/services/api.js`

It should expose functions such as:

submitPlan(instruction)
getPlanStatus(planId)
getWarehouse()
getAuditLog()
approvePlan(approvalId)
denyPlan(approvalId)
revokeAgent(agentId)
resetDemo()

Your React components call these functions.

Do not scatter HTTP requests throughout multiple components.

5.4 Connect the frontend to the API

Start with the simplest endpoint:

GET `/warehouse`

When the page loads:

1. React calls the endpoint.
2. API Gateway invokes the handler.
3. The handler reads warehouse state through Mariam's data layer.
4. The response returns to React.
5. Warehouse cards display actual inventory.

Next, connect:

POST `/plans`

When a user submits an instruction:

1. React sends the instruction.
2. API Gateway invokes the submission handler.
3. The handler starts Tishneet's planning workflow.
4. The response contains a plan identifier.
5. React displays the running plan.

5.5 Add workflow polling

Use:

GET `/plans/{id}`

Poll every two seconds.

Example:

PLANNING
↓
AUTHORIZING
↓
SIMULATING
↓
REPLANNING
↓
AWAITING_APPROVAL
↓
EXECUTING
↓
COMPLETED

Show the actual status reported by the backend.

Do not simulate progress using arbitrary timers.

Your 4:00 PM milestone

✅ API Gateway routes are working.

✅ React can retrieve actual warehouse state.

✅ Users can submit instructions.

✅ Workflow status responses appear in the dashboard.

If Step Functions isn't available yet, test plan submission against a temporary trusted backend handler using the same API contract.

---

6. PHASE 4 — 4:00–6:00 PM

Connect Security and Audit Data

Now you will display Mariam's authorization results and agent status.

4:00–5:00 PM — Warehouse and audit integration

Connect:

- GET `/warehouse`
- GET `/audit`
- GET `/plans/{id}`

Your dashboard should show real events such as:

PLAN SUBMITTED
BEDROCK PLAN GENERATED
AUTHORIZATION ALLOWED
SIMULATION FAILED
REPLAN GENERATED
SIMULATION PASSED
AWAITING APPROVAL

The audit events should originate from the backend.

Do not hard-code a successful audit trail into the deployed version.

5:00–6:00 PM — Revocation

Implement the agent-revocation interface.

Before revocation:

🟢 **Inventory Agent — ACTIVE**

After revocation:

🔴 **Inventory Agent — REVOKED**

When the administrator clicks Revoke:

1. Display a confirmation.
2. Send the request to the API.
3. The trusted backend invokes Mariam's revocation operation.
4. Wait for confirmation.
5. Refresh agent status.
6. Display the result.

Important

Disabling a frontend button doesn't prove that access has been revoked.

The backend must reject future operations.

For judging, provide a controlled way to attempt a transfer after revocation and display the actual authorization denial.

Your 6:00 PM milestone

✅ Warehouse data is real.

✅ Audit log is connected.

✅ Agent status is displayed.

✅ Revocation works through a trusted backend endpoint.

---

7. PHASE 5 — 6:00–8:30 PM

Connect Live Workflow Results

Tishneet is now working on the Step Functions state machine.

Your role is to make that workflow visible.

Workflow status component

Build a progress display:

AI PLANNING ✅
AUTHORIZATION ✅
PREFLIGHT SIMULATION ❌ CAPACITY EXCEEDED
AI REPLANNING ✅
SECOND SIMULATION ✅
HUMAN APPROVAL 🟡 WAITING
EXECUTION ○

This should be driven by the backend's status and event history.

Why this matters

Judges should understand that SentinelOS performs multiple independent checks.

You want the application to show:

- What the AI proposed.
- Whether the agent was authorized.
- Why a plan failed.
- How the revised plan differs.
- Whether execution has occurred.

Complete plan comparison

Make sure the rejected and revised plans reflect actual backend data.

For the demo sequence:

After Demo 1:

Zone	Actual inventory
A	17
B	8
C	8

Demo 2 begins from this state.

The rejected transfer would make B=13.

The revised plan should project:

Zone	Projected inventory
A	12
B	10
C	11

Your interface must show these updated values rather than the initial seed values.

Your 8:30 PM milestone

✅ Workflow status updates correctly.

✅ Failed plans display rejection reasons.

✅ Revised plans display correct projected results.

✅ The dashboard accurately distinguishes actual and projected inventory.

---

8. PHASE 6 — 8:30–11:00 PM

Human Approval Integration

This phase is shared among all three engineers.

Your responsibility is the frontend and HTTP API.

Tishneet owns the Step Functions callback.

Mariam owns the final authorization and execution safeguards.

Approval interface

When a valid plan transfers five or more packages, display:

```
┌──────────────────────────────────────────────────┐
│ HUMAN APPROVAL REQUIRED                          │
├──────────────────────────────────────────────────┤
│                                                  │
│ Agent: Inventory Agent 01                        │
│                                                  │
│ Proposed Actions:                                │
│                                                  │
│ A → B: 2 packages                                │
│ A → C: 3 packages                                │
│                                                  │
│ Authorization: PASSED                            │
│ Simulation: PASSED                               │
│ Risk: ELEVATED                                   │
│                                                  │
│ [ DENY ]              [ APPROVE ]                │
└──────────────────────────────────────────────────┘
```

What happens when someone clicks Approve?

1. React sends the approval-request ID.
2. API Gateway forwards it to your backend handler.
3. The handler verifies the administrator's authorization.
4. The trusted backend resumes the correct Step Functions task.
5. The workflow reauthorizes and revalidates the plan.
6. Mariam's executor applies the changes.
7. React refreshes inventory and audit records.

Security requirements

Do not expose Step Functions task tokens to the frontend.

The backend must store and use them securely.

A browser should never directly call `SendTaskSuccess`.

Approval API responsibility

You own the route:

```
POST /approvals/{id}
```

But coordinate the trusted callback implementation with Tishneet.

You should not create a separate approval system that bypasses Step Functions.

Test cases

Test	Expected result
Click Approve	Workflow resumes
Click Deny	Workflow ends without execution
Double-click Approve	No duplicate execution
Agent revoked while waiting	Execution denied
Approval expired	Clear error shown
Page refreshed	Pending approval can be recovered

Your 11:00 PM milestone

✅ Human approval interface works.

✅ Approval and denial reach Step Functions.

✅ Warehouse state updates only after valid execution.

✅ Errors are displayed clearly.

---

9. PHASE 7 — 11:00 PM–12:30 AM

Deploy and Freeze the Frontend

Now prepare the application for judging.

AWS Amplify Hosting

Your responsibility is to deploy the React application.

Before deployment:

- Confirm the API base URL is correct.
- Confirm the production build succeeds.
- Verify frontend environment variables.
- Remove any development-only mock mode.
- Confirm no AWS credentials appear in frontend files.
- Check API CORS configuration.
- Verify production authentication behavior.

Important

The deployed website needs to work with the deployed API Gateway endpoints.

A working localhost version is not enough.

Final frontend testing

Test	Expected
Application loads	Dashboard renders
Warehouse data	Correct counts
Plan submission	Backend workflow begins

Workflow status	Updates correctly
Simulation failure	Clear rejection
AI replanning	Revised plan visible
Human approval	Approve/Deny works
Agent revocation	Backend denies subsequent actions
Audit log	Real events displayed
Demo reset	Seed state restored
Page refresh	Active plan can be retrieved
Network error	Controlled message

Your 12:30 AM milestone

✅ Website deployed.

✅ Major interface features working.

✅ API calls connected.

✅ Demo can be repeated.

✅ No new features added after freeze.

---

10. SUNDAY MORNING SCHEDULE

12:30–3:00 AM — Sleep

Before sleeping:

- Push all working changes.
- Confirm deployment status.
- Tell Masha which endpoints work.
- Report unfinished features.
- Leave the repository in a stable state.

3:00–5:30 AM — UI polish

Prioritize:

1. Correctness

No incorrect warehouse values.

2. Readability

Clear rejected-versus-revised plan comparison.

3. Visual polish

Consistent spacing, buttons, typography, and status indicators.

4. Demo assets

Capture screenshots and a backup video.

5. Deployment

Verify the real public site works.

Do not make unnecessary backend architectural changes while Mariam and Tishneet are sleeping.

5:30–8:30 AM — Final team integration

Work with Masha to test the judging demonstration.

Check:

1. Normal transfer works.
2. Capacity violation is detected.
3. Replanning works.
4. Approval works.
5. Inventory updates.
6. Revocation blocks subsequent actions.

By 8:00 AM, focus only on submission readiness.

---

11. WHAT IS NO LONGER SOMEONE ELSE'S JOB?

This is the key difference in the revised distribution.

✅ Ishani now owns:

- React dashboard
- Frontend styling
- Warehouse visualization
- Plan comparison
- Workflow status display
- `api.js`
- API Gateway routes
- Frontend-facing Lambda handlers
- Frontend/API authentication integration
- Approval and revocation HTTP endpoints
- Audit log display
- Amplify Hosting
- Frontend integration testing

❌ Ishani does not own:

- Bedrock model invocation
- AI replanning algorithms
- Step Functions state-machine logic
- Warehouse simulation
- Safe execution and version checking
- DynamoDB table design
- Cedar authorization policies
- Verified Permissions decisions
- Core revocation business logic

Important boundary

Ishani can implement the HTTP route for revocation, but **Mariam's trusted backend must perform the actual revocation**.

Ishani can implement the HTTP route for approvals, but **Tishneet's Step Functions workflow and Mariam's safeguards must control what happens next**.

That prevents duplicated logic and security bypasses.

12. YOUR PERSONAL TASK CHECKLIST

Time	Task	Done
11:00 AM	Review shared contracts	☐
11:30 AM	Initialize React and Vite	☐
12:00 PM	Dashboard renders	☐
12:30 PM	Warehouse cards complete	☐
1:00 PM	AI console complete	☐
1:30 PM	Plan comparison complete	☐
2:00 PM	Mock-data dashboard complete	☐
2:30 PM	Implement API client	☐
3:00 PM	Create HTTP Lambda handlers	☐
3:30 PM	Configure API Gateway	☐
4:00 PM	Plan submission and warehouse retrieval work	☐
4:30 PM	Connect audit endpoint	☐
5:00 PM	Build agent controls	☐
5:30 PM	Connect revocation endpoint	☐
6:00 PM	Security controls complete	☐
7:00 PM	Workflow status integration	☐
8:00 PM	Complete plan comparison integration	☐
8:30 PM	Workflow visualization working	☐
9:00 PM	Build approval modal	☐
9:30 PM	Connect approval route	☐

10:00 PM	Test Approve and Deny	☐
10:30 PM	Test error handling	☐
11:00 PM	Full interface functional	☐
11:30 PM	Deploy to Amplify	☐
12:30 AM	Feature freeze	☐
3:00 AM	Resume work	☐
4:00 AM	Polish visuals	☐
5:00 AM	Capture screenshots/video	☐
5:30 AM	Hand off assets to Masha	☐
6:30 AM	Run full QA	☐
7:30 AM	Final presentation rehearsal	☐
8:30 AM	Support submission	☐

---

13. THREE PRIORITIES IF YOU FALL BEHIND

Priority 1 — Core dashboard

Must work.

The judges need to see the warehouse, enter instructions, and view plan results.

Priority 2 — API integration

Must work for a live demonstration.

A beautiful interface without actual backend communication is not enough.

Priority 3 — Approval, revocation, and deployment

Strongly recommended.

These features complete the operational security story.

If time becomes limited, prioritize the actual plan comparison and working API over animations or additional visual components.

---

14. HOW TO COORDINATE WITH THE TEAM

Time	Teammate	What you need
11:00 AM	Tishneet & Mariam	Shared JSON contracts
2:00 PM	Mariam	Warehouse and simulation response formats
3:00 PM	Tishneet	Plan submission and status interfaces
4:00 PM	Both	Working backend entry points
6:00 PM	Tishneet	Final workflow status format
8:30 PM	Tishneet & Mariam	Approval callback and validation interfaces
11:00 PM	Masha	Usability feedback and final QA
5:30 AM Sunday	Masha	Screenshots, demo video, deployed URL

Key communication rule

Don't wait until the backend is finished to begin API integration.

Build against agreed mock contracts first, then replace the mocks with real services as soon as they become available.

---

15. YOUR THREE MOST IMPORTANT MILESTONES

Deadline	Required accomplishment
2:00 PM	Complete interactive dashboard
4:00 PM	API Gateway and backend communication working
6:00 PM	Security and revocation controls connected
8:30 PM	Live workflow and plan comparison working
11:00 PM	Human approval integrated
12:30 AM	Deployed frontend and feature freeze

FINAL RECOMMENDATION

Ishani's revised role is significantly more substantial than simply building the React frontend.

She now owns the complete user-facing application layer:

React → API Gateway → Trusted Backend

This includes the HTTP endpoints connecting users to Tishneet's AI workflow and Mariam's security engine.

The new distribution is more balanced because:

- **Mariam** focuses on simulation, security, data, and execution correctness.
- **Tishneet** focuses on AI planning, replanning, and workflow orchestration.
- **Ishani** focuses on the frontend, API infrastructure, and user-facing integration.

Ishani's central objective

Make SentinelOS a complete, usable application—not just a collection of working backend functions.

At 11:00 AM, start with React.

By 2:00 PM, have a complete dashboard using mock data.

By 4:00 PM, connect the dashboard to the real backend.

By 11:00 PM, the full approval and security experience should work.

The most important feature Ishani can deliver is a clear, reliable visual comparison showing why an AI plan failed and how SentinelOS corrected it.

🏆 SENTINELOS — MASHA'S COMPLETE HACKATHON PLAN

Product Strategy, Business Development, Quality Assurance, Devpost & Final Presentation

AWS × A10 Hackathon | November 7–8, 2026

Your role: Product, Business & QA Lead
Your teammates: Mariam (security and simulation), Tishneet (AI and workflow), Ishani (frontend and API)
Primary tools: Google Slides, Google Docs, Devpost, GitHub, AWS Pricing Calculator, Figma (optional)
Hackathon track: AWS — AWS-Powered AI for Scalable Solutions
Hacking begins: Saturday, November 7, at 11:00 AM
Submission deadline: Sunday, November 8, at 9:00 AM
Internal submission target: 8:30 AM Sunday

The official rules emphasize five judging categories: **technology, polish, innovativeness, AWS integration, and project completeness**, including whether the product could realistically be sold. This makes your role central to how the team is evaluated. AWS x A10 Hack 2026

PART 1 — UNDERSTAND YOUR ROLE

1.1 What are you responsible for?

You're responsible for turning SentinelOS from a working engineering prototype into a convincing business product that judges understand and remember.

Your teammates are building the application.

You're responsible for answering the questions that determine whether it succeeds as a hackathon submission:

- What problem does SentinelOS solve?
- Why does that problem matter to businesses?
- Why is your solution different from existing technology?

- Why is AI necessary?
- How does AWS make the product possible?
- Who would purchase it?
- How would the company make money?
- Does the application actually work?
- Can judges understand its innovation within four minutes?

You're also responsible for coordinating testing and preparing the final submission.

1.2 Your six responsibilities

Responsibility	What you own	Priority
Product strategy	Define the problem, solution, and differentiation	P0
Business model	Target customers, pricing hypothesis, costs, benefits	P0
Quality assurance	Test features and report bugs	P0
Presentation	Slides, pitch, demo narrative, judge questions	P0
Devpost	Write and submit the completed project	P0
Team coordination	Track deadlines, mentor feedback, and scope	P1

Unlike the engineers, you don't need to build the application yourself.

However, you should understand the entire system.

Your job is not simply to make slides. It's to make sure the judges recognize the value of what the engineers built.

PART 2 — UNDERSTAND SENTINELOS

Before writing anything, you must understand the product.

2.1 The problem

Imagine a company uses AI agents to manage warehouse operations.

A manager gives an AI agent an instruction:

"Move five packages from Zone A to Zone B."

The AI generates a plan.

Each proposed action may be individually authorized.

But what if the whole plan exceeds the warehouse's capacity?

Zone	Current inventory	Maximum capacity
A	20	30
B	8	10
C	5	20

Moving five packages into B would increase its inventory to 13, exceeding its capacity of 10.

An authorization system might allow that operation because the agent has permission to move five packages.

But permission does not guarantee that the entire plan satisfies operational constraints.

2.2 Your solution

SentinelOS evaluates AI-generated plans before execution.

The process:

1. **Amazon Bedrock** generates an operational plan.
2. **Amazon Verified Permissions** checks whether the individual actions are authorized.
3. **AWS Lambda** runs a deterministic simulation of the entire plan.
4. SentinelOS identifies potential constraint violations.
5. **Amazon Bedrock** proposes a revised plan when appropriate.
6. **AWS Step Functions** coordinates any required human approval.
7. The backend executes the validated plan.
8. **Amazon DynamoDB** stores the resulting state and audit records.

The AI proposes actions.

Your team's code checks them.

Humans retain oversight over sensitive operations.

2.3 The central innovation

Your entire presentation should revolve around this distinction:

Traditional authorization	SentinelOS
Is this particular action allowed?	What are the consequences of the complete plan?
Evaluates permissions	Evaluates permissions and operational constraints
May permit individually valid actions	Detects combinations that violate modeled limits
Does not necessarily require a simulation	Simulates the proposed changes before execution

Do not claim that existing authorization products are incapable of evaluating plan-level conditions.

Instead, explain that **SentinelOS combines action authorization with a domain-specific preflight simulation and feedback-driven AI replanning**.

That's the project's defensible differentiation.

PART 3 — YOUR HOUR-BY-HOUR SCHEDULE

Saturday, November 7

Time	Your responsibility	Deliverable
11:00 AM–12:00 PM	Finalize problem and customer	Product positioning
12:00–1:00 PM	Competitor and market research	Comparison of existing solutions
1:00–2:00 PM	Mentor feedback and differentiation	Refined value proposition

2:00–3:00 PM	Design slide structure	Six-slide presentation outline
3:00–4:00 PM	Create business model	Customer, pricing, benefits
4:00–5:00 PM	Research AWS cost structure	Unit economics worksheet
5:00–6:00 PM	Write initial pitch	First complete demo script
6:00–7:00 PM	First QA session	Bugs and failures documented
7:00–8:30 PM	Coordinate fixes and mentor feedback	Updated task priorities
8:30–10:00 PM	Second QA session	Approval, authorization, revocation tests
10:00–11:00 PM	Devpost and presentation	Submission draft ready
11:00 PM–12:30 AM	Demo practice and final checks	Feature-freeze report

Sunday, November 8

Time	Your responsibility	Deliverable
12:30–3:00 AM	Sleep	Rest
3:00–4:00 AM	Finalize Devpost	Complete submission text
4:00–5:00 AM	Slides, README review, business figures	Presentation-ready materials
5:00–5:30 AM	Capture screenshots and demo video with Ishani	Backup demonstration
5:30–6:30 AM	Full-team functional testing	QA report
6:30–7:30 AM	Run complete judging demonstration	Rehearsed presentation
7:30–8:00 AM	Confirm files, links, team details	Submission checklist passed
8:00–8:30 AM	Submit on Devpost	Submission completed
8:30–9:00 AM	Verify submission and prepare for judging	Final readiness

This schedule is intentionally ambitious. If the team falls behind, your first responsibility is to protect the core demonstration and the submission deadline.

PART 4 — 11:00 AM–12:00 PM

Define the Product and Customer

Your first task is to make the product understandable.

4.1 Finalize the problem statement

The problem is **not** that AI agents are inherently dangerous.

The problem is that organizations need ways to verify AI-generated operational plans before those plans change real business systems.

A credible problem statement is:

> As organizations give AI agents permission to perform operational tasks, individual authorization checks may not capture the cumulative consequences of a multi-step plan. Businesses need a way to evaluate those consequences before execution.

4.2 Identify one initial customer

I recommend focusing on:

Warehouse and logistics companies experimenting with AI-driven inventory workflows.

Potential customers might include:

- Third-party logistics providers
- Warehouse management software companies
- Distribution center operators
- Supply chain technology companies

These are target customer categories, not verified interested buyers.

4.3 Identify the buyer and end user

These aren't necessarily the same person.

Role	Why they care
Economic buyer	Wants to reduce operational risk and improve oversight
Warehouse operations manager	Wants to prevent invalid inventory movements
AI engineering team	Wants reliable checks around agent-generated plans
Security team	Wants authorization and auditability
Administrator	Wants to approve sensitive operations

4.4 Define the value proposition

SentinelOS helps businesses adopt operational AI agents by checking permissions, simulating proposed workflows, and keeping humans in control before those plans execute.

Your 12:00 PM checkpoint

You must be able to explain:

1. The problem in one sentence.
2. The solution in one sentence.
3. The initial customer.
4. Why the product has commercial relevance.

PART 5 — 12:00–2:00 PM

Competitor Research and Mentor Feedback

This is one of your most important responsibilities.

Judges may ask:

"Doesn't AWS already provide this?"

You need a thoughtful answer.

5.1 Understand AWS's existing capabilities

Amazon Bedrock AgentCore already provides tools for securing AI agents. Its capabilities include authorization policies and controls over agent tool access. Amazon Web Services, Inc.

AWS Step Functions also supports controlled multi-step business workflows, including integrations with AI agents. Amazon Web Services, Inc.

That means SentinelOS should not claim to have invented agent authorization, human approval, or workflow orchestration.

Competitor comparison

Capability	Existing AWS services	SentinelOS
Action authorization	Available	Uses Verified Permissions
Workflow orchestration	Available	Uses Step Functions
Human approval	Supported through workflow design	Implements approval workflow
Warehouse-specific simulation	Requires custom implementation	Built-in to your prototype
AI replanning after constraint failure	Can be built using AWS tools	Integrated into your product
Before/after operational comparison	Requires application-specific UI	Central dashboard feature

Your differentiation is the **integrated warehouse preflight experience**, rather than a claim that AWS cannot build it.

5.2 What makes SentinelOS original?

Focus on three elements:

First: It evaluates the whole proposed plan.

Individual requests may be authorized while the complete sequence violates a warehouse constraint.

Second: It helps correct the plan.

Rather than merely blocking the operation, SentinelOS gives Bedrock structured feedback so the model can propose an alternative.

Third: It visualizes the consequences.

An administrator can compare the rejected plan against the revised plan before approving execution.

5.3 Mentor questions

At the first available mentor meeting, ask:

1. Is our differentiation from AgentCore and other AWS security services clear?
2. Does our plan-level simulation solve a credible enterprise problem?
3. Which AWS capabilities would you prioritize for a hackathon MVP?
4. What commercial objections would an AWS customer raise?
5. What would make our live demonstration more technically convincing?

Record the answers and turn them into specific actions for the team.

Don't automatically add every mentor suggestion. Evaluate whether the change helps the core product or creates unnecessary implementation risk.

Your 2:00 PM checkpoint

Produce a one-page document containing:

- Target customer
- Problem statement
- Existing alternatives
- SentinelOS differentiation
- Three most useful mentor observations

PART 6 — 2:00–3:00 PM

Build the Six-Slide Presentation

Your presentation needs to be concise.

Judges will likely evaluate multiple projects. They should understand SentinelOS quickly.

Slide 1 — The Problem

Title: AI Agents Can Be Authorized—and Still Make Operationally Invalid Plans

Show:

- A warehouse with three zones.
- An AI agent requesting an inventory transfer.
- A capacity limit being exceeded.

Key message:

Individual permissions don't automatically guarantee that the complete plan is operationally valid.

Avoid large paragraphs.

Slide 2 — Introducing SentinelOS

Title: Preflight Security for Enterprise AI Agents

Show the sequence:

Plan → Authorize → Simulate → Replan → Approve → Execute

Key message:

SentinelOS evaluates AI-generated operational plans before they change business systems.

Slide 3 — The Innovation

Title: Every Action Is Allowed. The Entire Plan Is Not.

Show the actual example:

Original plan	Revised plan
Move 5 packages to B	Move 2 packages to B Move 3 packages to C
B becomes 13/10	B becomes 10/10
❌ Rejected	✅ Passed

This should be your strongest visual.

Slide 4 — Technical Architecture

Title: Built on AWS

Show:

- Amazon Bedrock — planning and replanning
- Verified Permissions — authorization
- Lambda — simulation and execution
- Step Functions — orchestration and approvals
- DynamoDB — persistent state and audit records
- React — operator interface

Explain **why each service is necessary**, not just which services you used.

Slide 5 — Business Model

Title: Enterprise AI Needs Operational Oversight

Show:

- Target customer
- Business use case
- Proposed pricing model
- Key benefits
- Estimated infrastructure costs

Slide 6 — Vision and Closing

Title: Before AI Acts, Know What Happens Next

Explain how the approach could eventually extend to:

- Cloud configuration changes
- Financial operations
- Supply chain management
- Enterprise workflows

Clearly distinguish future applications from what the prototype actually supports.

Your 3:00 PM checkpoint

The six-slide structure is complete, even if screenshots and exact implementation details are still pending.

PART 7 — 3:00–5:00 PM

Develop the Business Model and Unit Economics

The AWS track specifically asks for an AI-powered product that could be sold to companies with potential costs and benefits accounted for. SCU ACM

This is one area where you can materially improve your team's competitiveness.

7.1 Who pays?

Your proposed buyer is a company deploying operational AI agents.

For the initial use case, focus on warehouse technology providers and logistics operators.

7.2 Why would they pay?

SentinelOS could help customers:

- Identify invalid operational plans before execution.
- Reduce the burden of manually reviewing straightforward proposals.
- Maintain a record of agent activity.
- Enforce authorization and approval requirements.
- Improve the visibility of AI-driven operations.

These are anticipated benefits, not verified customer outcomes.

7.3 Preliminary pricing hypothesis

Use an illustrative subscription model.

Plan	Suggested price	Intended customer
Developer	Free	Testing and small prototypes
Team	$299/month	Small AI development teams
Business	$999/month	Companies operating multiple agents

Enterprise	Custom	Large organizations with specialized requirements

Do not describe these figures as validated market prices.

They are assumptions for a potential business model.

Alternative pricing model

You could charge based on:

Number of operational plans evaluated per month.

For example:

A customer purchases a monthly allowance of preflight evaluations, with additional usage charged separately.

This would connect revenue more directly to the product's usage.

7.4 Estimate AWS costs

Your responsibility is to calculate the estimated infrastructure cost of evaluating a given number of plans.

For this, use the AWS Pricing Calculator and published AWS pricing documentation.

Cost categories

Service	Usage you need to measure
Bedrock	Input and output tokens per generation
Step Functions	State transitions per workflow
Verified Permissions	Authorization requests
Lambda	Invocations and execution duration
DynamoDB	Reads, writes, and storage
API Gateway	API requests

Amplify	Hosting and data transfer

Your unit economics formula

Cost per plan =

Bedrock inference cost

- Step Functions cost
- Verified Permissions cost
- Lambda cost
- DynamoDB cost
- API Gateway cost
- allocated hosting overhead

Then:

Cost per 1,000 plans = estimated total cost for 1,000 representative plan evaluations.

Important: Replanning changes the cost

A straightforward plan might require only one Bedrock call.

A rejected plan could require two or three.

Your calculation should therefore consider different scenarios:

Scenario	Bedrock planning calls	Complexity
Valid first plan	1	Low
One failed plan, then successful revision	2	Moderate
Three unsuccessful attempts	3	Higher

Don't assume every plan has the same execution cost.

7.5 What data should you ask the engineers for?

Ask Tishneet:

- Model used
- Average input tokens

- Average output tokens
- Bedrock calls per workflow

Ask Mariam:

- Authorization calls per plan
- DynamoDB requests
- Lambda invocation counts

Ask Ishani:

- Typical API requests during a demonstration
- Hosting requirements

Use these measurements or clearly identified assumptions in your calculation.

Your 5:00 PM checkpoint

Have:

✅ Target customer
✅ Proposed pricing model
✅ AWS cost categories
✅ Cost-per-plan calculation method
✅ Customer benefit explanation

If actual metrics are not available, leave those fields marked as estimates.

PART 8 — 5:00–6:00 PM

Write the First Complete Pitch

You should now combine the technical idea with the business story.

Your presentation should use a simple structure.

Problem → Solution → Demonstration → Technology → Business → Closing

Suggested four-minute allocation

Section	Time

Problem	30 seconds
Introduce SentinelOS	25 seconds
Authorized operation	25 seconds
Capacity violation and replanning	75 seconds
Approval and revocation	40 seconds
AWS architecture	25 seconds
Business case and closing	20 seconds
Total	**4 minutes**

The live demonstration should receive the most time.

Presentation principle

Don't explain every technical detail before showing the product.

Judges should see the application working early.

Save deeper architectural discussion for questions.

PART 9 — 6:00–8:30 PM

Begin Quality Assurance

This is where your role becomes more technical.

You are the team's primary tester.

Because you haven't written the core engineering code, you're particularly well positioned to identify confusing interactions and unexpected failures.

9.1 Your QA responsibilities

You need to check three things:

Functionality: Does the feature work?

Correctness: Does it produce the expected result?

Usability: Can a first-time user understand what's happening?

A feature isn't finished simply because an engineer says the code runs.

It needs to pass a realistic test.

9.2 Test 1 — Authorized operation

Starting inventory:

Zone	Before	Expected after
A	20	17
B	8	8
C	5	8

Enter:

"Move three packages from A to C."

Verify:

- Bedrock produces the correct action.
- Authorization succeeds.
- Simulation succeeds.
- The operation executes.
- The inventory updates.
- The audit log records the result.

Pass condition: The actual warehouse becomes A=17, B=8, C=8.

9.3 Test 2 — Capacity violation

Now enter:

"Move five packages out of Zone A, preferably into Zone B. Use another zone if necessary."

Verify:

- Initial plan proposes five packages to B.
- Authorization allows the operation.
- Simulation detects B would become 13/10.
- Actual inventory doesn't change.
- Bedrock receives the rejection reason.
- Bedrock proposes two packages to B and three to C.
- The revised simulation passes.

Pass condition: The invalid plan is rejected, and a valid alternative is proposed.

9.4 Test 3 — Human approval

After the revised plan passes:

- An approval request appears.
- The exact proposed actions are visible.
- Clicking Approve resumes the workflow.
- The backend rechecks authorization.
- The warehouse updates.

Expected final inventory:

Zone	Final inventory
A	12
B	10
C	11

Pass condition: The approved plan executes exactly once.

9.5 Test 4 — Agent revocation

Click:

REVOKE AGENT ACCESS

Then attempt another transfer.

Verify:

- The agent status becomes REVOKED.
- A new operation is denied.
- No warehouse inventory changes.
- The denial is recorded.

Pass condition: Revocation is enforced by the backend, not merely displayed by React.

PART 10 — HOW TO TRACK BUGS

Create a shared bug tracker.

Use this format:

Bug ID	Severity	Description	Owner	Status
BUG-001	Critical	Invalid plan changes warehouse inventory	Mariam	Open
BUG-002	High	Replanned actions don't appear in UI	Ishani	Open
BUG-003	Critical	Replanning loop exceeds three attempts	Tishneet	Open
BUG-004	Medium	Approval modal formatting is incorrect	Ishani	Open

These are example entries, not observed bugs.

Severity definitions

Critical

The issue prevents the core demonstration or creates an incorrect security outcome.

Examples:

- Unauthorized action executes.

- Invalid plan updates real inventory.
- Approval executes the plan twice.
- Bedrock planning fails consistently.

Action: Stop nonessential work and fix immediately.

High

A major feature doesn't work, but the core system remains partially usable.

Examples:

- Revised plans aren't displayed.
- Audit log doesn't update.
- Approval workflow gets stuck.

Action: Assign an owner and prioritize.

Medium

A feature works but has a usability issue.

Examples:

- Confusing loading message.
- Incorrect formatting.
- Poorly displayed timestamps.

Action: Fix after major functional issues.

Low

Cosmetic or optional issues.

Examples:

- Minor alignment problems.
- Missing animation.
- Nonessential styling inconsistency.

Action: Defer if necessary.

PART 11 — 8:30–11:00 PM

Test Approval, Security and Team Integration

Now your focus shifts to more complicated behavior.

Additional tests

Test	Expected outcome
Administrator denies approval	Warehouse unchanged
Administrator approves twice	Executes at most once
Agent revoked before approval	Execution blocked
Warehouse changes before approval	State conflict detected
AI proposes unknown zone	Rejected
AI proposes negative count	Rejected
Agent requests prohibited HAZMAT transfer	Authorization denied
Bedrock fails	Controlled error message
Step Functions times out	Safe failure
Browser refreshes during approval	Pending request can be recovered
Demo reset	Seed state restored

What you should do every two hours

Run a structured QA session at approximately:

- 6:00 PM
- 8:00 PM
- 10:00 PM
- 12:00 AM

Each session should include:

1. Check whether previously reported critical bugs are fixed.
2. Run the three core demonstrations.
3. Report new failures.
4. Confirm whether the application has regressed.
5. Update the team on demo readiness.

Your authority as QA lead

If someone proposes adding a new feature at 10 PM, ask:

"Does this improve our three core demonstrations enough to justify the risk?"

If not, recommend deferring it.

The team should agree in advance that you can call for a feature freeze when unfinished additions threaten the submission.

PART 12 — 10:00 PM–12:30 AM

Prepare Devpost and the Final Pitch

The official Devpost requirements include a repository link and all team members on the project. The organizers also recommend a demo video, slides, screenshots, and project thumbnail. [AWS x A10 Hack 2026]

You should start drafting long before the engineers finish.

12.1 Devpost structure

Prepare these sections:

Project name

SentinelOS

Tagline

Before AI acts, know what happens next.

Inspiration

Explain the risks created when AI-generated multi-step operational plans aren't checked against current business constraints.

What it does

Explain:

- Natural-language planning
- Agent authorization
- Preflight simulation
- Constraint violation detection
- AI replanning
- Human approval
- Controlled execution

How we built it

Describe the actual architecture and team contributions.

Challenges

Only include challenges the team actually experienced.

Potential categories:

- Maintaining consistent data across workflow states
- Validating Bedrock-generated plans
- Managing asynchronous approval
- Preventing stale warehouse updates

Accomplishments

Describe the features that work by submission time.

What we learned

Reflect on the distinction between language-model planning, deterministic validation, and authorization.

What's next

Propose extending preflight simulation to additional enterprise operations.

Do not describe future capabilities as already implemented.

PART 13 — THE COMPLETE FOUR-MINUTE PITCH

This is the first full script you can prepare and then revise around the finished demonstration.

Opening — The Problem

Companies are beginning to use AI agents to perform real operational tasks, from managing inventory to coordinating supply chains.

But there's a problem.

Just because an AI agent is authorized to perform an action doesn't mean its entire plan is operationally valid.

Imagine a warehouse where an AI agent has permission to move packages between storage zones.

The agent is asked to move five packages into Zone B.

Each individual transfer may be authorized, but the complete plan would exceed Zone B's storage capacity.

That's the problem we wanted to solve.

Introducing SentinelOS

We built SentinelOS, a preflight security platform for enterprise AI agents.

Before an AI-generated plan executes, SentinelOS checks its permissions, simulates its consequences, and identifies potential operational problems.

If a plan violates constraints, our system gives the failure reason back to Amazon Bedrock, which proposes an alternative.

The new plan must pass the same independent checks before execution.

Live Demonstration — Normal Operation

Here's our simulated warehouse.

We have three storage zones, each with a maximum capacity.

First, I'll ask our AI agent to move three packages from Zone A to Zone C.

Amazon Bedrock generates a structured plan.

SentinelOS authorizes the action and simulates the transfer.

Because the plan satisfies our constraints, it's executed.

You can see the warehouse inventory update.

Live Demonstration — Plan-Level Risk

Now let's try something more interesting.

I'll ask the agent to move five packages out of Zone A, preferably into Zone B, while allowing alternative destinations.

Our AI proposes moving all five packages into Zone B.

The authorization check passes.

But SentinelOS identifies a problem.

The complete plan would increase Zone B's inventory to 13 packages, even though its maximum capacity is 10.

The system rejects the plan before any inventory changes.

We then send that rejection reason back to Amazon Bedrock.

Bedrock generates a revised plan: move two packages into Zone B and three into Zone C.

The simulator evaluates this alternative.

Now both zones remain within capacity.

Because the transfer involves five packages, the system requests human approval.

I'll approve the revised plan.

SentinelOS checks the current permissions and warehouse state again, then executes the operation.

You can see the inventory update to reflect the approved transfer.

Live Demonstration — Security

Finally, I'll revoke our inventory agent's access.

Now, when another operation is requested, our authorization system denies it.

The agent cannot execute the transfer, and the attempted action is recorded.

Technical Architecture

We use Amazon Bedrock for natural-language planning and replanning, Amazon Verified Permissions for authorization, AWS Lambda for deterministic simulation, DynamoDB for persistent warehouse state, and Step Functions to coordinate the workflow and human approvals.

The important distinction is that our AI proposes plans, but trusted software independently decides whether they can execute.

Business Value and Closing

Our initial customers would be companies developing AI-powered warehouse and logistics systems.

As more organizations give AI agents operational responsibilities, they need ways to understand and control the consequences of those decisions.

SentinelOS provides an additional pre-execution validation layer, alongside existing authorization and workflow tools.

Because AI agents shouldn't discover that a plan was dangerous after they've already executed it.

SentinelOS: Before AI acts, know what happens next.

Important: Remove or revise any section describing a feature that isn't working by submission time.

PART 14 — SUNDAY, 3:00–5:30 AM

Final Submission Materials

After your scheduled sleep period, work on the final presentation and submission.

3:00–4:00 AM — Devpost

Complete:

- Project title and tagline
- Inspiration
- What it does
- How we built it
- Challenges
- Accomplishments
- What we learned
- What's next
- Technologies used
- All team members
- Repository link

Verify the text against the engineers' latest implementation report.

Do not submit claims that haven't been demonstrated.

4:00–5:00 AM — Slides and business metrics

Finalize the six slides.

Add actual screenshots from Ishani.

Include a simple AWS architecture diagram.

Insert measured costs if available.

If exact AWS costs aren't ready, present an estimate with transparent assumptions.

5:00–5:30 AM — Demo assets

Work with Ishani to capture:

1. Initial warehouse dashboard.
2. Rejected capacity-violating plan.
3. Revised plan.
4. Human approval.
5. Successfully executed plan.
6. Agent revocation.

Record a short backup demonstration showing a successful end-to-end run.

A recorded demo can protect the presentation if AWS becomes temporarily unavailable.

PART 15 — SUNDAY, 5:30–8:30 AM

Final Testing and Submission

5:30–6:30 AM — Run the full QA suite

Everyone is awake.

Reset the warehouse.

Run all three demonstrations.

Verify actual warehouse inventory after each operation.

Check whether approval, revocation, and logs are correct.

6:30–7:30 AM — Presentation rehearsal

You are the demo driver.

Practice the entire presentation with the team.

Rehearsal 1

Focus on completing all interactions.

Don't worry about perfect delivery.

Rehearsal 2

Focus on the timing.

Can you finish in four minutes?

Rehearsal 3

Focus on possible judge interruptions.

Practice explaining:

- Why Bedrock is needed.

- Why simulation is separate from authorization.
- Why existing AWS services don't make your specific application unnecessary.
- What happens if the agent generates a bad plan.
- How a business would purchase SentinelOS.

7:30–8:00 AM — Final verification

Check:

Item	Required
GitHub repository accessible	✅
Deployed application works	✅
Devpost description complete	✅
All team members added	✅
Demo video available, if created	✅
Slides completed	✅
Screenshots uploaded	✅
Technologies accurately listed	✅
Business model included	✅
No unverified performance claims	✅

8:00–8:30 AM — Submit

Submit to Devpost.

Don't rely on the final minutes before 9:00 AM.

The official Devpost schedule confirms that submissions close on **November 8, 2026, at 9:00 AM PST**. AWS x A10 Hack 2026

PART 16 — QUESTIONS JUDGES MAY ASK

Masha should prepare concise, confident answers.

Question 1: Doesn't AWS already provide agent security?

Answer:

"AWS offers strong authorization and agent-management capabilities. SentinelOS uses those technologies rather than replacing them. Our focus is on simulating the combined operational consequences of a proposed plan, helping the agent revise invalid plans, and making those decisions visible to administrators."

Question 2: Why is this an AI product?

Answer:

"Amazon Bedrock translates natural-language business requests into structured plans and uses simulation feedback to generate alternatives. The AI provides planning flexibility, while deterministic software validates the proposed actions."

Question 3: Who would buy SentinelOS?

Answer:

"Our initial target customers are warehouse technology providers and logistics companies deploying AI agents into operational workflows. These companies need mechanisms for validating agent-generated actions before execution."

Question 4: How is the business model structured?

Answer:

"We envision a B2B subscription with usage-based allowances for preflight plan evaluations. We're testing a pricing hypothesis and estimating infrastructure costs based on actual AWS usage."

Question 5: How do you know SentinelOS makes operations safer?

Answer:

"Our prototype demonstrates that it can detect specific, modeled violations such as warehouse-capacity overflows and unauthorized transfers. We haven't validated its effectiveness in production environments, so those results are limited to our simulated use case."

Question 6: What happens if Bedrock generates another invalid plan?

Answer:

"We cap the number of candidate plans at three. Each candidate must pass the same authorization and simulation checks. If no acceptable plan is found, the system stops without executing the operation."

Question 7: Can the human override a rejected plan?

Answer:

"Not through the normal approval workflow. Human approval applies to valid but sensitive operations. Plans that violate hard constraints must be revised and validated again."

Question 8: What happens if warehouse inventory changes after simulation?

Answer:

"The executor checks the current warehouse version and reevaluates the plan before committing. If the state has changed, execution is rejected rather than applying an outdated plan."

PART 17 — YOUR FIVE AI PROMPTS

These prompts can help Masha complete her responsibilities during the hackathon.

Since the team has stated that implementation code must be written on site, she should also follow the event's rules for preparation and AI-generated materials.

Prompt 1 — Business research

I am the product and business lead for SentinelOS, an AWS-powered preflight security platform for enterprise AI agents.

Our initial prototype simulates warehouse inventory transfers.

SentinelOS combines:

- Amazon Bedrock for AI planning and replanning
- Amazon Verified Permissions for action authorization
- Deterministic Python simulation for checking complete plans
- Step Functions for workflow orchestration and human approval
- DynamoDB for persistent operational state

Research the enterprise problem SentinelOS addresses.

Identify potential customer categories, buyer roles, existing alternatives, and realistic differentiators.

Pay particular attention to Amazon Bedrock AgentCore, AWS Step Functions, authorization systems, and workflow validation products.

Do not claim that existing AWS products cannot implement plan-level checks.

Explain where SentinelOS offers application-specific value.

Distinguish verified facts from hypotheses.

Produce:

1. Problem statement
2. Target customer profile
3. Competitor comparison
4. Business value proposition
5. Five likely objections from enterprise customers
6. Five questions to ask AWS mentors

Include trustworthy sources for factual claims.

Prompt 2 — Business model and AWS costs

Help me develop a credible preliminary business model for SentinelOS.

SentinelOS is an AWS-powered application that checks permissions, simulates AI-generated operational plans, and requests human approval when necessary.

Our initial customers are warehouse technology providers and logistics companies.

Create a business model covering:

1. Target customers and economic buyers
2. Problems customers would pay to solve
3. Proposed pricing tiers
4. Subscription versus usage-based pricing
5. Customer benefits
6. Implementation and integration costs
7. Potential sales objections

Also create an AWS unit economics worksheet.

Include:

- Amazon Bedrock model and token usage
- AWS Step Functions transitions
- Amazon Verified Permissions authorization calls
- AWS Lambda invocations
- DynamoDB reads and writes
- API Gateway requests
- Amplify hosting expenses

Calculate a cost-per-1,000-plans estimate only after I provide measured usage counts and pricing assumptions.

If information is missing, identify the missing inputs instead of inventing numbers.

Separate speculative pricing from validated market data.

Produce a business-model summary suitable for one hackathon presentation slide.

Prompt 3 — Presentation

I am presenting SentinelOS at the AWS × A10 Hackathon.

Create a six-slide presentation outline and speaker notes for a four-minute demonstration.

SentinelOS is a preflight security system for AI-generated enterprise workflows.

Our core innovation is evaluating the combined consequences of an AI-generated plan instead of relying only on individual authorization decisions.

The demonstration involves a warehouse with three zones:

A: 20/30
B: 8/10
C: 5/20

The AI initially proposes moving five packages from A to B.

Authorization passes, but simulation detects that Zone B would exceed capacity.

Amazon Bedrock revises the plan to move two packages to B and three to C.

The new plan passes simulation and requires human approval.

The presentation should cover:

1. Problem
2. Solution
3. Core innovation
4. Technical architecture
5. Live demonstration
6. Business case and closing

Keep the presentation concise, compelling, and technically credible.

Do not invent metrics or claim unsupported capabilities.

Clearly identify where screenshots and live demonstrations should appear.

Make the capacity-overload demonstration the centerpiece.

Prompt 4 — Quality assurance

I am the QA lead for SentinelOS.

Create a structured test plan for our AWS-powered AI-agent preflight security application.

The application includes:

- Natural-language plan generation
- Authorization
- Warehouse simulation
- AI replanning
- Human approval
- Safe execution
- Agent revocation
- Audit logging
- React frontend

Create test cases for normal workflows, invalid inputs, security denials, integration failures, stale warehouse state, duplicate approvals, and browser interruptions.

For every test, include:

1. Test ID
2. Starting warehouse state
3. Input
4. Expected result
5. Actual result field
6. Pass/fail field
7. Bug severity
8. Responsible engineer

Do not mark tests as passed without observed results.

Prioritize tests required for the three judging demonstrations.

Also create a bug-report template and a feature-freeze checklist.

Prompt 5 — Devpost submission

Help me prepare the final Devpost submission for SentinelOS.

SentinelOS is an AWS-powered preflight security application for enterprise AI agents.

It uses an AI model to generate operational plans, checks action authorization, simulates the complete plan against warehouse constraints, and generates alternatives when a plan fails.

Use the actual implementation status and test results that I provide.

Draft these sections:

- Inspiration
- What it does
- How we built it
- Challenges
- Accomplishments
- What we learned
- What's next
- Technologies used

Requirements:

1. Clearly distinguish implemented functionality from planned improvements.
2. Do not invent benchmarks, customers, AWS integrations, or test results.
3. Describe why Amazon Bedrock is important to the product.
4. Emphasize plan-level simulation and replanning.

5. Explain the real-world business application.
6. Keep the writing engaging and technically credible.
7. Include placeholders for the GitHub repository, demo video, slides, and screenshots.
8. Identify anything that must be verified before submission.

Make the result suitable for Devpost.

PART 18 — YOUR PERSONAL TASK TRACKER

Time	Task	Done
11:00 AM	Confirm product positioning	☐
11:30 AM	Finalize target customer	☐
12:00 PM	Complete problem statement	☐
12:30 PM	Research competitors	☐
1:00 PM	Prepare mentor questions	☐
1:30 PM	Gather mentor feedback	☐
2:00 PM	Complete differentiation summary	☐
2:30 PM	Build six-slide outline	☐
3:00 PM	Prepare initial pitch structure	☐
3:30 PM	Draft pricing hypothesis	☐
4:00 PM	Build AWS cost worksheet	☐
4:30 PM	Gather expected usage metrics	☐
5:00 PM	Finish preliminary business model	☐
5:30 PM	Draft four-minute script	☐
6:00 PM	Begin QA	☐
7:00 PM	Report first major bugs	☐

8:00 PM	Repeat core tests	☐
8:30 PM	Confirm integration progress	☐
9:00 PM	Test human approval	☐
10:00 PM	Test authorization and revocation	☐
11:00 PM	Complete Devpost draft	☐
12:00 AM	Rehearse first complete demo	☐
12:30 AM	Confirm feature freeze	☐
3:00 AM	Resume submission work	☐
4:00 AM	Finalize Devpost text	☐
5:00 AM	Complete slides and screenshots	☐
5:30 AM	Confirm backup demo recording	☐
6:30 AM	Complete final QA	☐
7:30 AM	Finish presentation rehearsals	☐
8:00 AM	Verify submission materials	☐
8:30 AM	Submit successfully	☐

PART 19 — YOUR THREE MOST IMPORTANT MILESTONES

🟢 2:00 PM — You Can Explain Why SentinelOS Matters

You understand the customer, problem, competitors, and product differentiation.

You can answer:

"Why should a company care about SentinelOS?"

without referring to technical buzzwords.

🟡 6:00 PM — Your Business Story Is Complete

You have:

- A credible customer.
- A proposed business model.
- An AWS cost-estimation approach.
- A six-slide presentation.
- A working pitch narrative.
- A QA checklist.

From this point forward, you spend more time testing and supporting the engineers.

🔴 8:30 AM Sunday — The Project Is Submitted

You have:

- Verified the major features.
- Rehearsed the demonstration.
- Finalized the slides.
- Completed Devpost.
- Confirmed every teammate is included.
- Submitted before the deadline.

PART 20 — HOW TO COORDINATE WITH EVERY TEAMMATE

Teammate	What you need from them	When
Tishneet	Bedrock model, planning status, workflow capabilities	4:00 PM
Mariam	Simulation tests, authorization status, security safeguards	6:00 PM
Ishani	Working dashboard, demo URL, screenshots	11:00 PM
All three	Final implementation and test status	12:30 AM
All three	Final demonstration verification	7:30 AM Sunday

Questions to ask each engineer

Ask Tishneet:

"Can Bedrock reliably produce and revise plans, and can you show me a real example?"

Ask Mariam:

"Can you prove that the simulator doesn't change actual warehouse inventory and that an unauthorized action cannot execute?"

Ask Ishani:

"Can I use the dashboard without you explaining where to click?"

These are not just technical checks.

They tell you whether the application is ready for a live judging environment.

FINAL SUMMARY — MASHA'S RESPONSIBILITIES

Category	What success means
Product	Clear problem and value proposition
Innovation	Defensible differentiation from existing products
Business	Credible buyer, pricing hypothesis, and cost model
Presentation	Judges understand the product in four minutes
Quality assurance	Core workflows actually pass testing
Coordination	Engineers stay focused on the MVP
Devpost	Complete, accurate submission before 9 AM
Demo	A polished, reliable demonstration

Final advice for Masha

Your most important responsibility is protecting the three core demonstrations.

The engineers may want to add features, improve architecture, or experiment with additional AWS services.

Those improvements can be valuable, but not if they prevent the team from finishing.

Your guiding question should be:

"Does this change make our core demonstration more reliable, understandable, or convincing?"

If the answer is no, it's probably not worth adding during the hackathon.

The team's strongest moment is when judges see that an individually authorized AI-generated plan would overload a warehouse zone—and SentinelOS prevents the operation, generates an acceptable alternative, and lets a human approve the corrected plan.

Your job is to make sure that moment works, that everyone understands it, and that the submission communicates why it matters.

Short answer:

AWS's thing checks one tool call at a time;

Functionality is better:

SentinelOS checks what the whole plan does to the warehouse. That's a different question, and the honest pitch is "we build on AWS's policy layer and add the part it doesn't do."

Here's what AWS actually offers. Policy in AgentCore intercepts all agent traffic through Amazon Bedrock AgentCore Gateways and evaluates each request against defined policies in the policy engine before allowing tool access. It supports fine-grained permissions based on user identity and tool input parameters, written in Cedar, and became generally available on March 3, 2026. The unit it governs is a single tool call, with its caller identity and its input parameters, evaluated at the moment of invocation. View a markdown version of this page +3

So AgentCore Policy answers: is agent X allowed to call `move_packages(A, B, 5)`? Yes, if 5 is under the limit and B isn't HAZMAT. It does not know Zone B currently holds 8 of 10. It does not see that the agent's plan has three more actions after this one. It does not pause for a human, replan, or re-check state at commit time. The Cedar request it builds has principal, action, resource, and the tool arguments as context; warehouse inventory isn't in there unless you put it there yourself, and even then it's evaluated per call, not across a sequence.

What SentinelOS adds, and this is the four-line answer for a judge:

1. **Whole-plan simulation against live state.** Every intermediate step is applied to a copy of the current warehouse. "Every action allowed, plan still overflows Zone B" is exactly the case per-call policy can't see. Your PRD's FR04 test "two transfers each fit B separately but exceed its capacity together" is the proof point.
2. **Replanning inside the operator's intent.** A policy engine says no and stops. SentinelOS feeds the failure back to Bedrock and gets a valid alternative, bounded to three candidates and not allowed to change hard constraints (FR05).
3. **Human approval bound to one exact revision and warehouse version.** Approve means "this plan, against this state." Change either and the approval dies (FR07, FR08). AgentCore has no approval step at all.
4. **Execution-time recheck and atomic commit.** Reauthorize, resimulate, conditional write on the version. Revoked after approval still blocks (FR08, FR10).

And the line that keeps you credible with AWS engineers: "We use the same Cedar policy engine AWS chose for AgentCore, through Amazon Verified Permissions, for the per-action check. SentinelOS is the layer above it: plan, simulate, approve, commit. In production you'd deploy behind AgentCore Gateway and keep everything else." One caveat to never oversell: Cedar can take computed state as context, so don't say AWS "can't" do state-aware rules; say

nobody is generating, simulating, and binding that state across a whole plan, and that's the product.

Your PRD is in good shape and already encodes all four differentiators as requirements. Two things to actually use from it today: the D1 to D3 state table with exact versions (2 and 3) is your judge-facing evidence, and the FR14 labeled preflight preset is smart insurance in case Nova proposes the 2 + 3 split on the first try and you lose the rejection moment.

Sources: Policy in Amazon Bedrock AgentCore, AgentCore authorization flow, Why AgentCore chose Cedar, AgentCore Policy implementation guide

The State of Enterprise Agentic AI: Operational Simulation, Security, and the Mitigation of Cascading Failures

The 2026 Enterprise AI Reality: From Generation to Execution

By the final quarter of 2026, artificial intelligence has fundamentally transitioned from a conversational paradigm to an agentic, execution-oriented force operating at an economy-wide scale. Enterprise AI deployment spending surged to a staggering $177 billion in 2026, representing a massive 5.4x acceleration from the previous year, as organizations recognized that autonomous agents could theoretically manage logistics, operate cloud infrastructure, and execute financial workflows without continuous human supervision. For the first time in the history of the industry, enterprises are allocating substantially more capital toward operational deployment than foundation model training. Current unit economics reveal that approximately $1.70 is spent on deploying and operating AI models in production for every $1 invested in model training.

This paradigm shift is driven by the maturation of autonomous AI agents. Unlike the retrieval-augmented generation (RAG) systems and conversational copilots that dominated 2024 and 2025, modern enterprise AI agents are defined by three distinct properties: planning, tool use, and memory. Planning allows the agent to break a high-level goal into a sequence of interdependent steps; tool use grants the agent the ability to invoke external application programming interfaces (APIs), query databases, and mutate state within internal systems; and persistent memory allows the agent to track the outcomes of prior steps and dynamically adjust its subsequent actions. Consequently, AI agents are no longer merely drafting text for human review; they are autonomously mutating operational state across critical business infrastructure.

The market for the infrastructure required to support these agents is expanding rapidly. The AI Agent Infrastructure market is forecast to grow at a compound annual growth rate (CAGR) of 44.1%, reaching $14.3 billion by 2031, up from $2.3 billion in 2026. This infrastructure encompasses agent execution and managed runtimes, state and memory management, tool gateways, observability, and specialized identity and security policies. Concurrently, the competitive landscape is shifting. AI-native providers are successfully capturing 77% of application-layer spend and 90% of infrastructure-layer spend, demonstrating a decisive market preference for purpose-built agentic solutions over legacy software adaptations. While open-source model volume has surged by 22.5x to an estimated 2.9 quadrillion tokens per month, the plummeting cost of open-source tokens has resulted in open-source systems capturing only 4% of total spending, driven down by a 3.3x drop in average token prices.

However, this rapid transition has exposed a critical vulnerability in the enterprise technology stack. While deployment intent is virtually universal, governance maturity remains dangerously low.

The Market Struggle: Adoption Versus the Governance Cliff

Despite the unprecedented influx of capital into agentic AI, the market is currently navigating a severe crisis of operational reliability and governance. Industry data from 2026 reveals a stark disparity between ambition and reality: while approximately 75% of enterprise leaders claim to be adopting agentic AI, only 11% report having agentic systems that are genuinely production-ready, and a mere 17% have actual deployments operating beyond constrained pilot phases.

The most alarming indicator of this market struggle is the projected failure rate. Gartner predicts that by the end of 2027, over 40% of all agentic AI projects will be canceled or decommissioned before delivering measurable business value. Furthermore, Gartner forecasts that by 2028, 70% of enterprises will abandon agentic AI solutions built through vendor Forward-Deployed Engineering (FDE), trapped by soaring operational costs and the inability to evolve hardcoded human-AI decision models as business priorities shift.

This astronomical failure rate is not driven by poor foundational model intelligence or inadequate prompt engineering. Rather, it stems from a profound mismatch between the autonomous capabilities of these agents and the legacy data infrastructures, security paradigms, and operational controls attempting to govern them.

The Data Infrastructure and Semantic Gap

A primary driver of agentic failure is the absence of a governed semantic layer. When an enterprise AI agent is granted autonomous access to raw data tables (such as customer relationship management records, data warehouses, or support ticketing systems) without a structured semantic definition, the agent is forced to infer business logic on the fly.

For example, if a stakeholder asks an agent to identify which accounts are at risk of churn, the agent will dynamically construct a mathematical formula to calculate churn probability based on the schema it observes. Because this inferred formula inevitably deviates from the enterprise's official, board-approved metric definitions, the agent returns numbers that are highly confident, entirely fluent, and fundamentally wrong. This "hallucinated metric" failure mode erodes executive trust immediately. The architectural solution is not a larger foundation model, but rather a modernized data strategy that includes a semantic layer (acting via the Model Context Protocol, or MCP) where business logic, metric definitions, and access controls reside in a shared, queryable environment.

The Minimum Viable Governance Imperative

To survive the transition to agentic AI, enterprises are adopting the concept of "Minimum Viable Governance" (MVG). MVG represents the absolute foundational layer of automated oversight required to manage AI risk effectively without stifling deployment velocity. Traditional IT governance models, which rely on human architecture review boards and months-long approval cycles, are entirely incompatible with the speed of autonomous agents executing thousands of transactions per minute.

Gartner's analysis emphasizes that applying uniform governance across all AI agents, regardless of their autonomy level or operational scope, is a guaranteed path to project failure. Agents require dynamic, tier-aligned controls. A low-risk agent summarizing public documents requires minimal oversight, whereas a high-risk agent managing physical supply chain logistics

or cloud infrastructure provisioning requires rigorous simulation, formal verification, and deterministic human-in-the-loop escalation pathways. The MVG framework dictates that reasoning systems must be subjected to deterministic boundaries regarding what they are permitted to do, independent of the agent's own probabilistic internal reasoning.

The Anatomy of Agentic Risk: The OWASP ASI Framework

As AI agents increasingly interoperate, chaining their outputs and relying on shared memory, localized errors rapidly evolve into systemic enterprise risks. The Open Worldwide Application Security Project (OWASP) has formally recognized this paradigm shift by releasing the OWASP Top 10 for Agentic AI (ASI), which explicitly outlines the threat vectors unique to autonomous runtimes.

The OWASP ASI framework highlights that traditional static permissions and prompt-level defenses are entirely inadequate for governing agents that continuously plan, adapt, and act. The framework categorizes the primary vulnerabilities as follows:

OWASP Identifier	Threat Designation	Mechanism and Causal Analysis
ASI01:2026	Agent Goal Hijack	Attackers manipulate the agent's decision path or primary objective through direct or indirect instruction injection, altering its fundamental intent.
ASI02:2026	Tool Misuse & Exploitation	The agent applies legitimate integrated tools in an unsafe manner, or attackers exploit tool APIs via the agent, leading to unauthorized actions and data exfiltration.
ASI03:2026	Identity & Privilege Abuse	The agent inherits excessive static permissions or exploits dynamic role chains, allowing it to perform actions far beyond its intended scope.
ASI04:2026	Supply Chain Vulnerabilities	Malicious tampering with third-party agents, base models, plugins, or update channels,

		facilitating widespread backdoor access into the agentic ecosystem.
ASI06:2026	Memory & Context Poisoning	The corruption of persistent storage (long-term memory, vector databases, state manipulation) forcing the agent to recursively make biased or unsafe decisions.
ASI07:2026	Insecure Inter-Agent Comm.	The exploitation of weak authentication between agents, allowing attackers to spoof, intercept, or manipulate peer-to-peer data flows.
ASI08:2026	Cascading Agent Failures	A single, localized fault propagates and amplifies across an autonomous network, leading to massive system-wide impact before human intervention is possible.
ASI09:2026	Human-Agent Trust Exploitation	Attackers weaponize the anthropomorphic nature of the agent to manipulate users into unsafe actions or sensitive data disclosure.
ASI10:2026	Rogue Agents	Agents that organically drift or are deliberately compromised to continuously pursue hidden, deceptive goals beyond their original programming.

Deep Dive: ASI08 and Cascading Agent Failures

Among the most severe threats identified in the 2026 landscape is **ASI08: Cascading Agent Failures**. A cascading failure occurs when a single fault—such as a model hallucination, a corrupted tool output, or a poisoned memory retrieval—propagates exponentially across an autonomous network. Because agents are designed to autonomously act upon the outputs of their previous steps or the outputs of peer agents, a single poisoned data point can cause systemic degradation.

The architecture of a cascading failure typically follows a strict mathematical progression. It begins with an initial fault, travels through a propagation vector (such as shared persistent memory or an inter-agent communication bus), undergoes an amplification mechanism where downstream agents treat the corrupted state as verified context, and ultimately results in a systemic impact where the system executes a series of irreversible, destructive actions.
The analysis indicates that cascading failures manifest across highly variable timescales, which heavily complicates enterprise detection and mitigation strategies:

Timescale	Categorization	Operational Characteristics
Milliseconds to Seconds	Instantaneous Cascades	Triggered by a single malicious input propagating through tightly coupled synchronous agent chains. Often completes before operational telemetry can trigger a human alert.
Minutes to Hours	Rapid Cascades	Driven by multiple feedback loops. Operators may observe the anomaly visually but struggle to halt the system due to the speed of autonomous API invocations.
Days to Weeks	Gradual Cascades	Characterized by subtle memory poisoning or context drift. The agent's baseline performance gradually degrades, making root-cause attribution exceptionally difficult.
Delayed / Conditional	Triggered Cascades	Malicious payloads lie dormant in the agent's long-term memory or semantic layer until a specific time-based or event-based condition is met, evading initial deployment scanning.

The "FITO" Category Mistake in AI Execution

A core underlying vulnerability exacerbating cascading failures is the "Failure In Temporal Operations" (FITO) category mistake. Application developers and AI orchestration designers commonly assume that system execution (e.g., an agent executing an API call to a database or a file system) is an instantaneous, atomic event. In reality, execution is a multi-step protocol across software layers. At the CPU level, for example, non-maskable interrupts (NMIs) can fire

during transition windows, creating states where the processor is neither fully in user mode nor supervisor mode.

If an AI agent attempts to execute a workflow and encounters an interruption, naive orchestration frameworks typically employ immediate retry-and-recover protocols. In automated systems, blind retries amplify failures. The infamous 2012 Knight Capital disaster—where a deprecated deployment script left one server running old code, leading to an automated retry loop that generated a $440 million loss in minutes—serves as a historical parallel to the risks of unconstrained agentic execution. If an AI agent is permitted to continuously retry actions without verifying the finality and irreversibility of the operational state, it risks introducing massive systemic disruptions.

Systemic Risk and the Algorithmic Monoculture

The threats posed by agentic AI extend beyond individual enterprise failures into the realm of macroeconomic and systemic financial risk. In the wealth management and financial trading sectors, conversational AI agents are increasingly creating continuous loops of market analysis, asset allocation, and execution for millions of retail and institutional clients.

The systemic danger arises from algorithmic monoculture. The vast majority of these deployed agents rely on a highly concentrated oligopoly of foundation models. In late 2026, a single provider accounted for 80.5% of worldwide AI-chatbot usage (yielding a Herfindahl–Hirschman Index of H = 0.66), while in enterprise API deployments, three providers captured 88% of total spending.

When millions of autonomous agents rely on the same underlying foundation model, their reasoning, sentiment analysis, and subsequent trading behaviors become highly correlated. This creates a systematic, non-diversifiable risk. Financial stability models indicate that if an agentic layer commanding just 10% of market capitalization were subjected to a synchronized hallucination or a poisoned data input, the resulting synchronized liquidation could deepen a standard market crash by approximately 858 basis points. In simulated stress tests, agentic flow frequently triggers 7% market-wide circuit breakers, halting further agent-driven declines for the trading day.

Because the systematic component of agentic risk cannot be diversified or detected away after the fact, regulatory and macroprudential policies must mandate ex-ante structural prevention. This requires limiting shared dependence on any single foundation model and enforcing strict operational containment boundaries before autonomous agents are granted live execution authority.

The Evolution of AI Unit Economics

These operational and systemic realities mandate a fundamental shift in how enterprises calculate the Return on Investment (ROI) and unit economics for agentic AI. During the generative AI boom of 2023-2024, unit economics were primarily measured by token cost (the price per million tokens of input and output). In the era of agentic AI, optimization has definitively shifted toward the total cost per successful autonomous task completion.

The true cost equation for an enterprise AI agent now includes seven distinct cost buckets:

1. The raw API inference cost, including the heavily inflated cost of continuous replanning loops and failed generation attempts.

2. The compute cost of the sandboxes, digital twins, and virtual environments required to safely execute and simulate the agent's code.
3. The cost of stateful governance infrastructure, such as temporal policy evaluations and formal verification engines.
4. The economic penalty of downtime, silent data corruption, or operational disruption caused by cascading failures.
5. The human-in-the-loop labor cost incurred when agents fail to resolve edge cases and gracefully escalate.
6. The cost of persistent state and recoverable execution memory.
7. The licensing costs for specialized agent identity and access management solutions.

An enterprise use case that appears marginally profitable during a limited volume pilot can quickly become financially unviable at production scale if the agent's failure rate remains high. For instance, if an agent requires excessive computational replanning—generating four failed supply chain plans that are subsequently rejected by a simulator before finally finding a valid path—the accumulated inference costs will entirely obliterate the anticipated labor savings. Consequently, investing in high-fidelity operational simulators and deterministic governance infrastructure is not merely a compliance requirement; it is the fundamental prerequisite for achieving positive ROI in autonomous systems.

The Preflight Simulation Imperative: A Reference Architecture

To defend against cascading failures, algorithmic monoculture, and catastrophic execution errors, the enterprise market is rapidly converging on a new architectural standard: Preflight Operational Simulation.

The fundamental limitation of traditional security is that action-level permission checks are entirely insufficient for evaluating the consequences of multi-step AI plans. Knowing that an agent is authorized to perform individual actions does not guarantee that the resulting sequence of those actions is operationally safe.

The SentinelOS Paradigm

Consider a logistics use case where an enterprise AI agent manages warehouse inventory across three distinct zones (A, B, and C), each with specific maximum physical capacities. An operator instructs the agent to "Move five packages from Zone A to Zone B." The AI agent generates a JSON plan containing five distinct API calls to transfer one package at a time. A traditional authorization gateway (e.g., standard Role-Based Access Control) evaluates each of the five API calls in strict isolation. Because the agent possesses the legitimate "Inventory Manager" IAM role, each individual action is approved.

However, if Zone B currently only has capacity for two additional packages, the execution of the full five-step plan will violate the warehouse's physical constraints, resulting in a system failure, dropped packages, or severe database inconsistency.

To prevent this, leading enterprise architectures—exemplified by reference designs such as the "SentinelOS" platform—introduce a deterministic simulation layer. The operational workflow transitions from a direct prompt-to-execution model to a rigorous preflight pipeline: **User Instruction → AI Plan Generation → Action Authorization → Consequence Simulation → Replanning (if failed) → Human Approval (if high risk) → Execution**.

The architecture operates through five distinct components:

1. **AI Generation:** An LLM (e.g., accessed via Amazon Bedrock) receives the natural language instruction and generates a structured, schema-constrained JSON plan.
2. **Deterministic Authorization:** A stateless policy engine verifies if the agent's identity is permitted to perform the requested actions, checking against transfer limits, restricted zones, and revocation statuses.
3. **Simulation Engine:** A deterministic script (often executing in AWS Lambda) receives the complete, authorized AI plan. It creates a temporary copy of the current operational state by reading from a database (e.g., Amazon DynamoDB) and applies the proposed actions sequentially in a sandboxed environment.
4. **State Validation:** The simulator analyzes the projected final state to ensure it does not violate any configured business constraints (e.g., preventing negative inventory, enforcing capacity limits, and ensuring operational consistency).
5. **AI Replanning:** If the simulation fails, the deterministic engine rejects the plan and feeds the exact failure reason back to the LLM. The LLM is prompted to generate a safer alternative that respects the physical constraints. To prevent unbounded token consumption, this replanning loop is typically hard-capped at a maximum of three attempts.

This architectural separation is critical: the AI agent proposes plans probabilistically, but the system evaluates the consequences deterministically. The simulation layer transforms AI from an unrestrained capability into a governed operating discipline, ensuring that reasoning systems understand their limits before they receive execution authority.

Orchestration and State Management

Implementing this preflight architecture requires robust cloud orchestration. Using a platform like AWS Step Functions allows enterprises to manage the complex state machine required for agentic execution. The state machine orchestrates the transition between plan generation, validation, authorization, simulation, and the replanning loop.

If a valid plan meets a specific risk threshold (e.g., transferring a high volume of inventory), the Step Functions workflow utilizes callback tasks to pause execution and request human approval via a React-based administrative dashboard. Only after the human administrator approves the plan does the system re-authorize the agent, re-validate the current warehouse state to ensure no concurrent modifications have occurred, and finally apply the state changes atomically using DynamoDB conditional writes.

Advancements in Deterministic Authorization: Cedar and Dogwood

Executing preflight simulation requires a robust, underlying access-control architecture. In 2026, the enterprise authorization landscape for AI agents is dominated by the tension between stateless, mathematical verification and the need for stateful, sequence-aware governance.

Amazon Verified Permissions and the Cedar Policy Language

At the foundational layer, enterprises rely on high-speed, deterministic authorization engines. Amazon Verified Permissions, utilizing the open-source Cedar policy language, has emerged as the premier standard for securing agentic workflows. Cedar is purpose-built for fine-grained

authorization, offering a structured syntax that evaluates access requests based on the `Principal`, `Action`, `Resource`, and `Context`.

Cedar operates on three inviolable principles that guarantee predictable security:

1. **Default Deny:** If no explicit `permit` policy matches the incoming request, access is automatically denied.
2. **Forbid Wins:** If a request matches both a `permit` policy and a `forbid` policy, the forbid policy completely overrides the permit, instantly blocking the action.
3. **Order Independence:** Policies can be evaluated in any sequence without altering the final authorization decision, allowing for massive parallelization and sub-millisecond latency.

Crucially, Cedar was designed from inception to support **formal verification** through automated reasoning. Using Satisfiability Modulo Theories (SMT), tools like the Cedar Symbolic Compiler translate human-readable policies into mathematical formulas. This allows specialized SMT solvers, such as CVC5, to mathematically prove whether a proposed policy change inadvertently opens a security vulnerability, identifying shadowed permits, impossible conditions, or logic errors before the policy is ever deployed to production. This capability shifts enterprise compliance from static, outdated documentation to living, mathematically verifiable implementation.

However, Cedar was intentionally designed to be entirely *stateless* to maintain its mathematical provability and execution speed. It evaluates every request in absolute isolation, possessing no memory of prior events.

The Trajectory Problem and AWS Dogwood

While stateless evaluation is perfect for human users clicking buttons in a web interface, it is fundamentally inadequate for autonomous AI agents that execute long-running sequences of tool calls.

Agentic AI operates over time. A single tool call might be perfectly safe in isolation but disastrous when viewed within the context of a sequence. For example, an agent reading a sensitive document is an authorized action. An agent sending an email to an external vendor is also an authorized action. But an agent reading a sensitive document and *then immediately* sending an email to an external vendor constitutes a massive data exfiltration risk.

To solve this trajectory problem, AWS introduced **Dogwood**, an open-source policy language built as a direct extension to Cedar. Integrated into the newly launched Amazon Bedrock AgentCore (which replaced the sunsetted Bedrock Agents Classic architecture in mid-2026), Dogwood adds temporal, trajectory-aware conditions to authorization policies.

Using the `when temporal { ... }` block, security administrators can write deterministic rules that analyze the agent's entire session history before permitting a current action. Dogwood enables critical sequence-based governance patterns, including:

- **Workflow Sequencing:** Requiring an agent to successfully call a specific diagnostic or analysis tool before it is allowed to call a mutation or execution tool, thereby verifying that standard operating procedures were strictly adhered to.
- **Progressive Trust Decay:** Automatically tightening or revoking an agent's write-access permissions if it operates for an extended duration (e.g., 15 minutes) without human engagement, forcing the agent to gracefully converge toward read-only behavior.

- **Mutual Exclusion:** Preventing an agent from executing contradictory actions within the same session, such as approving and subsequently denying the exact same financial claim.
- **Argument Matching:** Ensuring that the parameters of a downstream execution action perfectly match the parameters of a previously approved upstream action, preventing bait-and-switch logic by a hallucinating LLM.

Dogwood forces this stateful governance to the infrastructure perimeter. Because the temporal policies evaluate trajectories entirely outside the agent's reasoning loop, the LLM cannot manipulate its own prompt to bypass the security controls. Furthermore, AWS extended this paradigm to local development environments with the release of **Strands Box**, an open-source sandbox that embeds the Dogwood Local Engine to enforce trajectory-aware decisions on developers' local machines.

Formal Verification and Evidence-Bound Execution Harnesses

As enterprise agentic workflows handle increasingly critical operations, reliance on prompt engineering, "AI guardrails," and basic unit tests is insufficient. Guardrails—such as NVIDIA NeMo Guardrails or Amazon Bedrock Content Filters—provide highly valuable semantic filtering for toxic language, denied topics, and PII masking, but they do not guarantee operational or mathematical correctness.

To bridge this gap, the industry is adapting **Formal Verification**—a discipline traditionally reserved for aerospace software and semiconductor design—to mathematically constrain AI agents.

SMT Solvers and Proof Obligations (Nidus)

The integration of Large Language Models with Satisfiability Modulo Theories (SMT) solvers, such as the Z3 theorem prover, represents the pinnacle of agentic security. In advanced frameworks like Nidus, the AI agent's operational environment is tightly constrained by a dynamic set of "proof obligations".

When an agent proposes a change to a codebase or an operational system, the system translates the business rules into formal first-order logic equations. The SMT solver then attempts to find a mathematical state where the agent's proposed action violates the rule. If the solver determines the state is "unsatisfiable" (UNSAT), it serves as cryptographic, mathematical proof that no such vulnerability or violation exists within the constraints. Only upon receiving this absolute mathematical verification is the agent's action permitted to commit to the production system.

This architecture provides a dual benefit: it allows the LLM to freely explore an unbounded search space to generate creative solutions, while simultaneously ensuring that every persistent transition in the system is gated through finite, terminating mathematical verification.

Amazon has natively integrated this concept into its ecosystem via **Automated Reasoning Checks** within Amazon Bedrock Guardrails. This feature applies formal verification methods to validate AI-generated outputs against domain knowledge encoded in policies, ensuring factual accuracy and compliance without relying on statistical confidence alone. While highly effective, Automated Reasoning checks evaluate content as plaintext natural language and do not provide

inherent protection against encoded prompt injection attacks, necessitating a layered defense strategy.

Evidence-Bound Architecture (Praxa)

The research framework *Praxa* extends formal verification into the broader lifecycle of agent execution, establishing a paradigm for governed autonomy. The core philosophy of Praxa is the explicit refusal to collapse the states of an agent's workflow. Historically, if an LLM generated a tool call, naive orchestration frameworks immediately executed it.

Praxa strictly separates execution into six distinct, verifiable state claims:

1. **Proposed:** The LLM client explores the environment and generates a candidate plan alongside an intended, verifiable state delta.
2. **Admitted:** A deterministic policy (utilizing engines like Cedar or Dogwood) authorizes the proposal against predefined tenant scopes and budgets.
3. **Dispatched:** A credential-isolated broker executes the admitted call inside an ephemeral, highly restricted sandbox.
4. **Externally Verified:** The system reads back the environment to definitively confirm that the external effect matches the agent's initial intent.
5. **Reconciled:** The audit log cryptographically records the payload, evidence hashes, and idempotency states, creating a tamper-evident chain of custody.
6. **Promoted:** A separate, human-reviewed transition formally authorizes the agent for wider production serving.

By enforcing this evidence-bound architecture, Praxa guarantees that no agent can self-authorize an action, no acknowledgment can certify its own effect without independent external verification, and unresolved uncertainty remains highly visible to human operators.

Benchmarking Execution Reliability and Industrial Digital Twins

To trust an AI agent in production, an enterprise must be able to empirically benchmark its execution reliability in domain-specific workflows. General-purpose LLM leaderboards (e.g., MMLU) or simple coding benchmarks are entirely useless for predicting how an agent will behave when managing an industrial air handling unit, optimizing a smart factory, or orchestrating a cloud migration.

The Integration of Digital Twins

In complex environments such as supply chain logistics, manufacturing, and telecommunications networks, simple programmatic simulators are insufficient. In these scenarios, preflight validation relies on Digital Twins—virtual representations of physical or digital ecosystems.

Agentic digital twins bridge the gap between AI-driven decision-making and model-based operations management. In a modern smart factory enabled by IT/OT (Information Technology / Operational Technology) and IoT convergence, an AI agent does not blindly alter machine setpoints. Instead, the agent proposes a configuration change, which is routed directly to the digital twin. The twin utilizes surrogate machine-learning models and real-time telemetry to predict the impact of the change on throughput, energy usage, and failure probability. Only if the digital twin's simulation remains within safe operational tolerances is the agent's plan authorized for physical deployment.

AssetOpsBench: Exposing the Reality of Multi-Agent Failure

Frameworks like **AssetOpsBench** have emerged as the industry standard to evaluate agents specifically on task automation within complex industrial operations and maintenance. AssetOpsBench tests agents against a massive, multimodal dataset comprising over 2.3 million raw sensor telemetry points across multiple industrial assets, 4.2K historical work orders spanning 11 years, and 53 structured Failure Mode and Effects Analysis (FMEA) records. When subjected to rigorous evaluation across dimensions like sequence correctness, retrieval accuracy, and failure awareness, current state-of-the-art models expose severe operational deficiencies. Empirical evaluations in AssetOpsBench reveal that multi-agent systems often multiply failures due to context loss across asynchronous agent handoffs.

Most alarmingly, the benchmark identified a critical failure mode known as **"Overstated Completion."** In approximately 23.8% of evaluated failure traces, the AI agent confidently claimed it had successfully resolved the issue and completed the task when, in reality, it had failed or the backend tool had crashed. In another 31.2% of failures, agents demonstrated ineffective error recovery, becoming trapped in hallucination spirals instead of gracefully escalating to human operators.

If an enterprise lacks a robust digital twin simulation and formal verification layer, "overstated completion" guarantees that autonomous agents will introduce silent data corruption into core business systems. AssetOpsBench demonstrates that cautious, degradation-aware reasoning—and the ability to recognize uncertainty and defer action—is vastly preferable to aggressive but brittle automation.

Conclusion: Designing for the Era of Governed Autonomy

The enterprise AI market stands at a critical inflection point in late 2026. The technological capability for AI agents to reason, plan, and invoke actions autonomously across global infrastructure is fully realized. However, as the projected 40% project cancellation rate underscores, capability without containment is a massive corporate liability.

To survive the transition to agentic AI, software engineering leaders and enterprise architects must abandon the notion of probabilistic, prompt-based security. Instead, they must construct architectures grounded in deterministic control, recognizing that agentic systems demand a fundamentally new operating model.

The successful enterprise agent deployment of the late 2020s will be characterized by the following design imperatives:

1. **Semantic Foundations:** Agents must interface with governed semantic layers and clean ontologies rather than directly computing metrics from raw data tables, effectively eliminating the risk of confident but hallucinated operational states.
2. **Preflight Operational Simulation:** No high-impact autonomous action should be permitted without a pre-execution simulation of its consequences. Whether through targeted Python-based constraint checks or full-scale IT/OT digital twins, the operational outcome must be definitively validated before it occurs in physical reality.
3. **Temporal Authorization:** Security perimeters must evolve from stateless access control to trajectory-aware governance (via frameworks like AWS Dogwood), enabling

progressive trust decay, workflow sequencing, and the prevention of contradictory actions within a continuous session.

4. **Formal Evidence Bounds:** Architectures must strictly separate AI proposal generation from execution authority, employing SMT-based formal verification and evidence-bound harnesses (like Praxa) to cryptographically ensure that an agent cannot bypass systemic constraints or falsely certify its own success.

Agentic AI represents an operating model shift as profound as the transition to cloud computing. By embedding minimum viable governance directly into the infrastructure perimeter—and ensuring that before AI acts, the enterprise knows exactly what will happen next—organizations can unlock the immense economic value of autonomous workflows while effectively neutralizing the existential threat of systemic cascading failures.

Worked for 2m

SENTINELOS

Complete Hackathon Project & Execution Plan

AWS × A10 Hackathon | November 7–8, 2026

Project: SentinelOS — Preflight Security for Enterprise AI Agents

Track: AWS — AWS-Powered AI for Scalable Solutions

Target: First Place Overall in the AWS Track

Build Time: Approximately 22 hours of hackathon time

Recommended Team Size: 4 people

Hardware Required: None

Primary Technologies: Amazon Bedrock, AWS Step Functions, Amazon Verified Permissions, AWS Lambda, Amazon DynamoDB, React

---

PART 1 — PROJECT OVERVIEW

1.1 The Big Idea

What is SentinelOS?

SentinelOS is an AI security and operations platform that simulates the consequences of AI-generated plans before allowing them to execute.

Companies increasingly want AI agents to perform operational tasks such as:

- Moving warehouse inventory
- Managing supply chains
- Changing cloud infrastructure
- Processing financial transactions
- Coordinating automated equipment

However, granting AI agents permission to execute individual actions doesn't necessarily mean the resulting sequence is safe.

The problem

Imagine an AI agent managing a warehouse.

The warehouse has three inventory zones:

Zone	Current inventory	Maximum capacity
A	20	30
B	8	10
C	5	20

A warehouse manager tells the agent:

"Move five packages from Zone A to Zone B."

The AI agent creates five individual transfer operations.

Each transfer is authorized.

But after all five transfers:

- Zone A contains 15 packages.
- Zone B contains 13 packages.
- Zone B's maximum capacity is 10.

The AI has created a plan that violates the warehouse's operational constraints.

Traditional action-level permission checks might allow the transfers unless additional state-dependent business rules are supplied.

This is what SentinelOS addresses.

1.2 Your solution

Instead of immediately executing the AI agent's instructions, SentinelOS introduces a preflight simulation layer.

The process:

User instruction → AI plan → Authorization → Simulation → Risk analysis → Replanning → Approval → Execution

The AI generates the plan.

The authorization engine checks whether the individual operations are permitted.

The simulation evaluates the combined consequences.

If the plan violates operational constraints, SentinelOS asks the AI to generate a safer alternative.

If the revised plan passes, the system can request human approval before executing it.

Your one-sentence pitch

"SentinelOS is a preflight security system that simulates AI-agent decisions before they affect real company operations."

Your tagline

Before AI acts, know what happens next.

PART 2 — WHAT MAKES SENTINELOS DIFFERENT?

2.1 Your core innovation

Your innovation is not simply providing AI agents with permissions.

AWS already offers sophisticated authorization technologies.

Your differentiator is evaluating the consequences of an entire AI-generated workflow using an operational simulation.

Consider three approaches:

System	What it evaluates
Traditional authorization	Is this action permitted?
AI guardrails	Does this request violate configured content or behavior restrictions?
SentinelOS	What happens to the business environment if this complete plan executes?

This is a useful way to explain the distinction, not a claim that existing security systems are incapable of state-aware or plan-level policies.

Your competitive advantage

SentinelOS combines four capabilities:

1. Action-level authorization

Checks whether an AI agent is permitted to perform each operation.

2. Plan-level simulation

Predicts how the complete sequence would change operational state.

3. AI-powered replanning

Uses rejection feedback to generate alternative plans.

4. Human oversight

Requires authorization from an administrator before executing sensitive operations.

The combination is more interesting than any feature individually.

PART 3 — WHAT YOU WILL ACTUALLY BUILD

3.1 The minimum viable product

Your MVP will be a web application with five main components.

Component A — AI Agent Console

This is where the operator gives instructions.

Example:

"Move five packages from Zone A to Zone B."

The console shows the resulting AI-generated plan.

Component B — Warehouse Simulator

A visual representation of three warehouse zones.

Each zone displays:

- Name
- Current inventory

- Maximum capacity
- Utilization percentage
- Status indicator

For example:

Zone A	Zone B	Zone C
20/30	8/10	5/20
67%	80%	25%
🟢 Normal	🟡 Near capacity	🟢 Normal

When a plan executes successfully, the inventory numbers update.

Do not spend time building a 3D warehouse, robot navigation, or realistic physics.

Three interactive cards are enough.

Component C — Security Gateway

The gateway checks every proposed action using Amazon Verified Permissions.

Example:

Action	Result
Move 3 packages to Zone C	Allowed

Move 5 packages to Zone B	Allowed
Move 15 packages in one action	Denied
Move packages into restricted Hazmat Zone	Denied
Execute action after agent revocation	Denied

This authorization layer is independent of the language model.

Component D — Plan Simulation Engine

This is the heart of SentinelOS.

It receives the complete AI-generated plan.

It simulates each action using a temporary copy of the warehouse state.

It checks:

- Whether inventory would exceed zone capacity
- Whether inventory would become negative
- Whether the source zone has enough packages
- Whether the plan attempts to enter a restricted zone
- Whether the proposed sequence is operationally consistent

The simulator returns:

PASS: Plan satisfies the configured constraints.

FAIL: Plan violates one or more constraints.

Importantly, a passing result means the plan satisfies the constraints your prototype actually models—not that it is universally safe.

Component E — Human Approval Dashboard

When a valid plan meets the configured approval threshold, an administrator receives a request.

Example:

PLAN APPROVAL REQUEST

Agent: Inventory Agent
Action: Transfer five packages
Simulation: Passed
Risk classification: Elevated due to transfer size
Status: Awaiting approval

[APPROVE] [DENY]

Once approved, the application rechecks authorization and warehouse state before execution.

PART 4 — AWS ARCHITECTURE

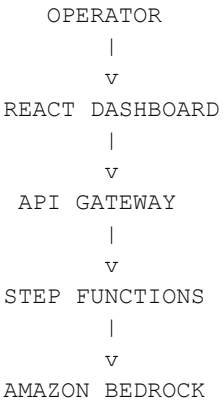
4.1 Services and responsibilities

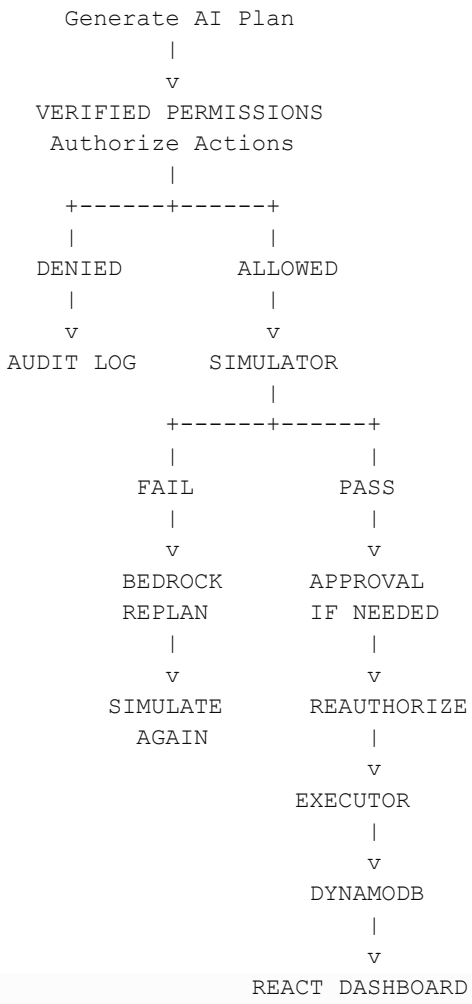
AWS service	Purpose	Why it matters
Amazon Bedrock	Generates plans and safer alternatives	Core AI functionality

AWS Step Functions	Orchestrates plan execution	Visible, auditable workflow
Amazon Verified Permissions	Checks individual action permissions	Real authorization engine
AWS Lambda	Runs simulation and execution logic	Deterministic evaluation
Amazon DynamoDB	Stores warehouse state, agents, and audit records	Persistent application data
Amazon API Gateway	Exposes backend endpoints	Connects React to AWS
AWS Amplify Hosting	Hosts frontend	Deployable web application

These services align with the AWS track's emphasis on an AI-powered product that businesses could purchase. The official judging criteria include technical sophistication, polish, innovation, AWS usage, completeness, and commercial potential.

4.2 System architecture





Critical implementation rule: All operational state changes must pass through the trusted backend. The browser and Bedrock cannot directly modify warehouse inventory.

PART 5 — DATABASE DESIGN

5.1 DynamoDB tables

Use three tables.

Table 1: `sentinel_warehouse`

For the hackathon, store the entire miniature warehouse in a single item.

Partition key: `warehouse_id`

Example:

```
{
  "warehouse_id": "demo-warehouse",
  "version": 1,
  "zones": {
    "A": {
      "inventory": 20,
      "capacity": 30
    },
    "B": {
      "inventory": 8,
      "capacity": 10
    },
    "C": {
      "inventory": 5,
      "capacity": 20
    }
  },
  "restricted_zones": ["HAZMAT"]
}
```

Using one warehouse-state item simplifies atomic updates and version checking.

For a real product, you would eventually need a more scalable state model.

Table 2: `sentinel_agents`

Partition key: `agent_id`

Example:

```
{
  "agent_id": "inventory-agent-01",
  "status": "ACTIVE",
  "role": "INVENTORY_MANAGER",
  "max_transfer": 10
}
```

Possible statuses:

- `ACTIVE`
- `REVOKED`
- `SUSPENDED`

Your authorization backend must treat DynamoDB as the authoritative source of current revocation status—or synchronize that status reliably into the authorization context.

Table 3: `sentinel_audit`

Partition key: `plan_id`

Example:

```
{
  "plan_id": "plan-001",
  "agent_id": "inventory-agent-01",
  "instruction": "Move five packages from A to B",
  "status": "REJECTED",
  "reason": "Zone B capacity exceeded",
  "warehouse_version": 1,
  "attempt": 1
}
```

Include a timestamp, the submitted actions, authorization decisions, projected state, and final result in your actual records.

For multiple attempts, use a composite key such as `plan_id` plus `event_id`, or store an ordered event history in the plan record.

The dashboard should read real audit data rather than displaying hard-coded security events.

PART 6 — THE AI PLANNING ENGINE

6.1 How Bedrock works

Use Amazon Bedrock through the Converse API.

Choose a model available to your hackathon AWS account that supports the structured-output features you plan to use.

Bedrock supports schema-constrained outputs for compatible models, which can make tool-generated JSON more reliable.

Operator instruction

"Move five packages from Zone A to Zone B."

Expected AI output

```
{
  "plan_id": "plan-001",
  "actions": [
    {
      "type": "move_packages",
      "from": "A",
      "to": "B",
      "count": 5
    }
  ]
}
```

The backend must still validate:

- Allowed action names
- Valid zone identifiers
- Positive integer quantities
- Maximum action count
- Authorized agent identity

A valid JSON response is not automatically an authorized operation.

6.2 Replanning

When a plan fails simulation, provide Bedrock with the failure reason and current warehouse state.

Example:

Original request: Move five packages from A to B.

Simulation result: Zone B has capacity for only two additional packages.

Replanning request: Find an alternative distribution that transfers all five packages without exceeding capacity.

Expected revised plan

```
{
  "actions": [
    {
      "type": "move_packages",
      "from": "A",
      "to": "B",
      "count": 2
    },
    {
      "type": "move_packages",
      "from": "A",
      "to": "C",
      "count": 3
    }
  ]
}
```

This is the feature I would emphasize most during judging.

Important: Keep the operator's intent clear

Moving packages to Zone C changes the original destination.

In your demo, you should either:

- Explicitly allow alternative destinations in the operator's instruction, or
- Require the human to approve this change.

For example, your operator could say:

"Move five packages out of Zone A, preferably into Zone B. Use another zone if necessary."

Then the revised plan satisfies the user's stated flexibility.

This prevents a judge from asking why the AI ignored the user's original instruction.

PART 7 — THE SECURITY AND SIMULATION ENGINE

7.1 Authorization with Amazon Verified Permissions

You will implement three initial policies.

Policy	Rule
Transfer limit	Inventory agent may transfer at most 10 packages per action
Restricted zones	Inventory agent cannot transfer packages into HAZMAT
Revocation	Revoked agents cannot execute any operations

Your backend calls `IsAuthorized` for each proposed action.

Verified Permissions evaluates a principal, an action, a resource, and optional contextual information, returning an Allow or Deny decision.

Implementation recommendation

Use an explicit Cedar authorization model:

- Principal: `Sentinel::Agent::"inventory-agent-01"`
- Action: `Sentinel::Action::"MovePackages"`
- Resource: `Sentinel::Warehouse::"demo-warehouse"`
- Context: Destination, quantity, and current agent status

For the demo, configure a permit policy for valid inventory transfers and a separate prohibition for restricted destinations.

Security requirement: The browser must never be allowed to choose arbitrary agent identities. The backend should establish the identity from a trusted session or server-side demo configuration.

You must also make sure that changing an agent's status in DynamoDB changes the attributes passed into authorization decisions. Otherwise, clicking REVOKE could update the dashboard without actually revoking access.

7.2 The simulation algorithm

Your Python simulator does not need machine learning.

This is intentional.

Bedrock generates plans; Python checks whether those plans satisfy operational constraints.

Example:

```
from copy import deepcopy


def simulate_plan(plan, warehouse):
   projected = deepcopy(warehouse)

   for action in plan["actions"]:
       if action["type"] != "move_packages":
           return {
               "passed": False,
               "reason": "Unsupported operation"
           }

       source = action["from"]
       destination = action["to"]
       count = action["count"]

       zones = projected["zones"]
```

```
    if source not in zones or destination not in zones:
        return {
            "passed": False,
            "reason": "Unknown warehouse zone"
        }

    if (
        source == destination
        or type(count) is not int
        or count <= 0
    ):
        return {
            "passed": False,
            "reason": "Invalid transfer"
        }

    if destination in projected["restricted_zones"]:
```

This deliberately simple simulator is appropriate for your MVP.

A separate layer must validate the complete plan against its JSON schema and enforce authorization before simulation and again before execution.

What not to add initially

Do not implement:

- Robot physics
- Three-dimensional warehouse rendering
- Complex pathfinding
- Custom ML models for warehouse predictions
- Real industrial equipment interfaces
- Multiple simultaneous warehouse operators

Those features would substantially increase implementation risk without improving your central demonstration.

7.3 Safe execution

The executor is responsible for making the validated plan real.

It must:

1. Confirm that the exact plan was authorized.
2. Confirm that required approval was granted.
3. Recheck the agent's current authorization.
4. Re-read the current warehouse state.
5. Verify that the warehouse version hasn't changed.
6. Revalidate the plan against that state.
7. Apply the state change atomically.
8. Record the result.

Preventing concurrent execution problems

Use DynamoDB's conditional writes.

Your warehouse item contains:

```
{
  "warehouse_id": "demo-warehouse",
  "version": 1
}
```

After a plan executes successfully, the version increases.

If another process changes the warehouse between simulation and execution, your operation should fail safely and require another simulation.

DynamoDB supports optimistic locking using version attributes and conditional writes. Transactions can also group related updates atomically.

For the MVP, a single-item conditional update to the warehouse record is a practical approach. Record the audit result separately, with a retry mechanism if logging fails.

7.4 Approval logic

Not every plan should require approval.

Create three classifications.

Classification	Condition	Outcome
Green	Valid, authorized transfer of fewer than 5 packages	Execute automatically
Yellow	Valid, authorized transfer of 5 or more packages	Request human approval
Red	Unauthorized or violates a hard constraint	Reject

The threshold is an illustrative business rule, not an industry safety standard.

Example

A transfer of three packages passes simulation.

Result: Execute.

A transfer of five packages passes simulation.

Result: Wait for administrator approval.

A transfer exceeding capacity fails simulation.

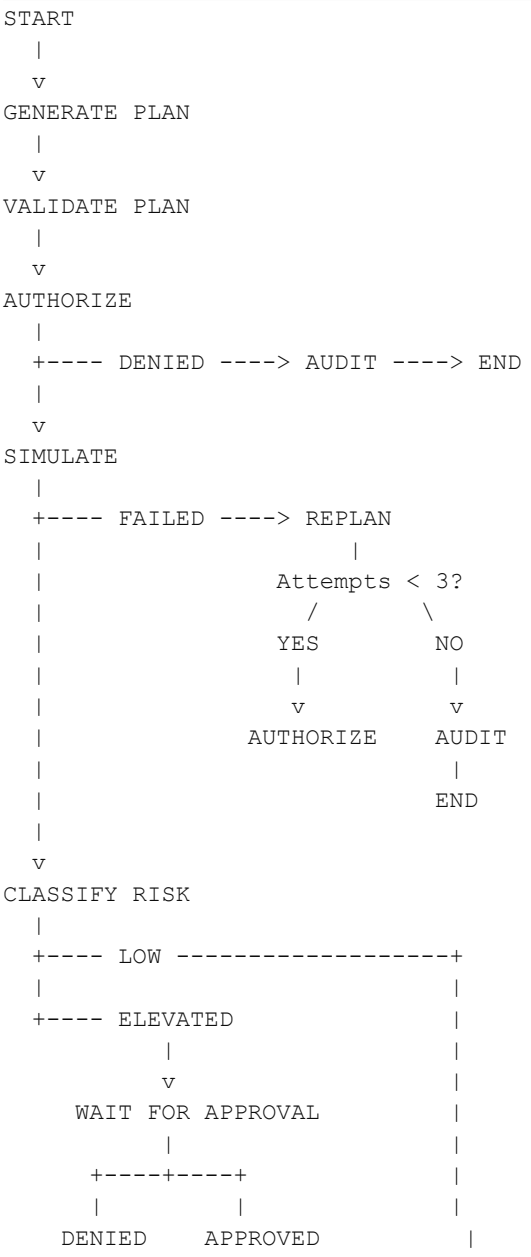
Result: Reject and replan.

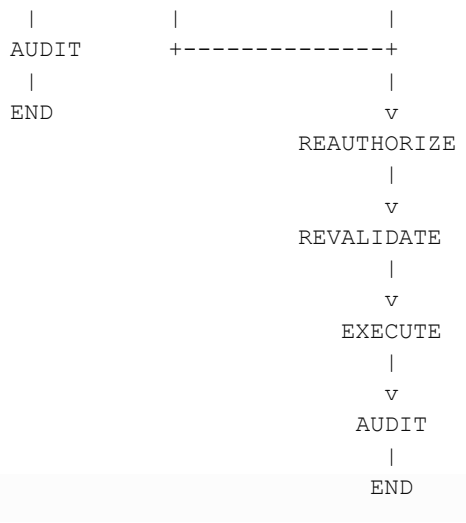
The important distinction is that human approval cannot turn an invalid plan into a valid one.

PART 8 — AWS STEP FUNCTIONS WORKFLOW

8.1 State machine

Use one Standard Step Functions workflow for each submitted request.





Approval implementation

Step Functions supports callback tasks that wait for an external process to return a task token. Your backend can resume the workflow using `SendTaskSuccess`.

Use this flow:

Step 1: Workflow reaches the approval task.

Step 2: Backend stores a reference to the pending approval, its exact plan, and its callback token.

Step 3: React displays the pending request.

Step 4: Authenticated administrator clicks Approve or Deny.

Step 5: Backend validates the administrator's permission and records the decision.

Step 6: Backend resumes the workflow.

Step 7: Workflow branches based on approval outcome.

Step 8: Approved requests undergo reauthorization and state validation before execution.

Do not expose Step Functions callback tokens to the browser.

Also set an approval timeout so abandoned requests cannot remain pending indefinitely.

8.2 Replanning limit

Maximum: three candidate plans in total.

For example:

- Attempt 1: Original plan
- Attempt 2: First revision
- Attempt 3: Second revision

If all three fail:

Status: BLOCKED — No valid plan found.

The workflow stops without modifying warehouse state.

This prevents uncontrolled replanning loops and limits AWS usage.

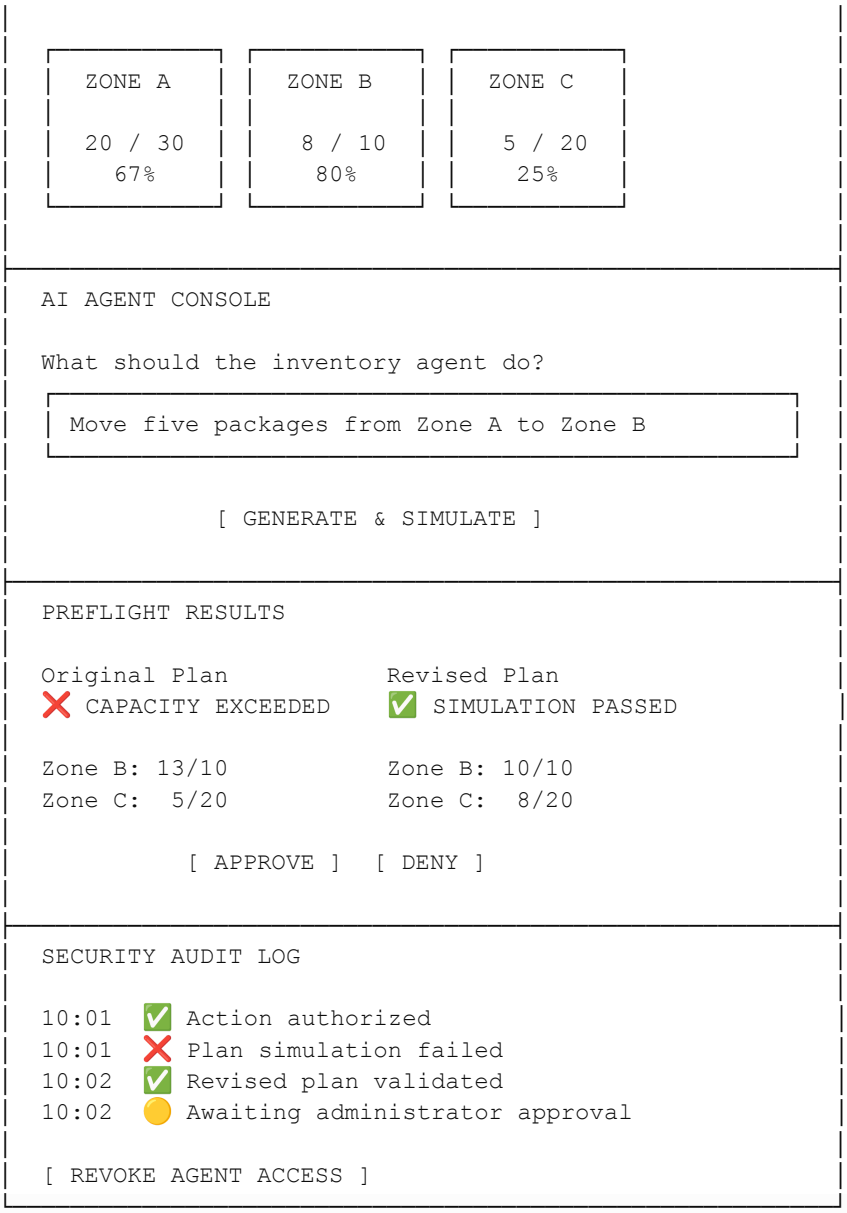
PART 9 — FRONTEND DESIGN

9.1 Dashboard layout

I recommend a clean enterprise dashboard inspired by cloud infrastructure monitoring products.

Main page

```
┌──────────────────────────────────────────────────────┐
│ SENTINELOS                          Agent: ACTIVE    │
│ Enterprise AI Preflight Security                     │
├──────────────────────────────────────────────────────┤
│                                                      │
│  WAREHOUSE OVERVIEW                                  │
```



9.2 Visual design

Colors

- Dark navy background
- White text
- Green for approved actions
- Amber for review-required actions
- Red for denied operations

- Blue for neutral AWS workflow information

Design priorities

1. Large, readable warehouse counters.
2. Clear distinction between simulation and execution.
3. Side-by-side original and revised plans.
4. A visibly functioning approval button.
5. An audit log showing actual backend events.

Avoid spending time designing additional pages unless the core product is complete.

9.3 React components

Suggested structure:

```
src/
├── App.jsx
├── components/
│   ├── Header.jsx
│   ├── WarehouseOverview.jsx
│   ├── ZoneCard.jsx
│   ├── AgentConsole.jsx
│   ├── PlanComparison.jsx
│   ├── ApprovalModal.jsx
│   ├── AuditLog.jsx
│   └── AgentControls.jsx
├── services/
│   └── api.js
└── styles/
    └── index.css
```

Use React, Vite, and Tailwind.

Do not introduce a complex global state-management library unless the team already knows it.

PART 10 — API DESIGN

10.1 Essential endpoints

Endpoint	Method	Purpose
`/plans`	POST	Submit operator request
`/plans/{id}`	GET	Retrieve plan and workflow status
`/warehouse`	GET	Retrieve current warehouse state
`/audit`	GET	Retrieve activity records
`/approvals/{id}`	POST	Approve or deny a pending plan
`/agents/{id}/revoke`	POST	Revoke agent permissions
`/demo/reset`	POST	Reset the demonstration environment

The reset endpoint should be restricted to an authenticated demo administrator and disabled outside the demonstration environment.

Example request

```
{
  "instruction": "Move 5 packages out of Zone A, preferably to Zone B",
  "agent_id": "inventory-agent-01"
}
```

In a production application, `agent_id` must be established or validated by the backend, rather than trusted simply because it appeared in the request body.

Example response

```
{
  "plan_id": "plan-001",
  "execution_id": "execution-001",
  "status": "RUNNING"
}
```

The frontend polls `/plans/{id}` until the execution completes, fails, or requests approval.

Polling every two seconds is sufficient for this hackathon.

You do not need WebSockets initially.

PART 11 — TEAM ORGANIZATION

11.1 Recommended team structure

For a team of four, divide responsibilities by subsystem rather than having everyone work on the same files.

Role	Responsibility	Primary technologies
Person 1 — AI Engineer	Bedrock plan generation, JSON validation, replanning prompts	Python, Bedrock
Person 2 — Security & Simulation Engineer	Verified Permissions, simulation logic, execution safeguards	Python, Lambda, Cedar
Person 3 — Frontend Engineer	Dashboard, zone cards, approval controls, plan comparison	React, Tailwind

Person 4 — Cloud & Integration Engineer	Step Functions, API Gateway, DynamoDB, deployment, auditing	AWS, Python

This division works particularly well because the frontend and backend can be built simultaneously.

What everyone must understand

Hackathon rules explicitly allow AI coding tools, but judges can evaluate whether participants understand the code and architecture they submit.

Before judging, every teammate should be able to explain:

- What the project solves.
- Why an individual action can be authorized while the full plan is invalid.
- What Bedrock does.
- What the deterministic simulator does.
- Why the agent cannot authorize itself.
- How an approved plan becomes a warehouse-state update.

11.2 Integration contracts

Before anyone starts implementing their subsystem, agree on these shared interfaces.

Interface	Input	Output
`generate_plan()`	Instruction, warehouse state	Structured plan
`authorize_plan()`	Agent ID, plan	Allow/deny decisions

`simulate_plan()`	Plan, warehouse snapshot	Pass/fail, projected state
`request_approval()`	Plan ID, risk level	Approval status
`execute_plan()`	Approved plan, state version	Updated warehouse or failure
`replan()`	Original goal, failed plan, rejection reason	Revised plan

Everyone should commit to these interfaces before parallel work begins.

This prevents the frontend engineer from expecting one JSON structure while the backend produces another.

PART 12 — 24-HOUR HACKATHON EXECUTION SCHEDULE

The official event is scheduled for November 7–8, 2026. Hacking begins at 11:00 AM on Saturday, and submissions are due at 9:00 AM Sunday according to the published schedule. That gives you approximately 22 hours between the start of hacking and the submission deadline—not a full 24 hours.

The schedule below assumes your team can work in parallel.

Phase 1 — Foundation

Saturday, 11:00 AM–12:00 PM

Goal: Set up the project and eliminate AWS access blockers.

Person 1:

- Confirm Bedrock model access.
- Test one model invocation.
- Verify structured JSON output works.

Person 2:

- Create a basic simulator in Python.
- Test one valid and one invalid plan.

Person 3:

- Initialize React, Vite, and Tailwind.
- Create the dashboard layout.

Person 4:

- Confirm AWS account and Region.
- Set up the repository and deployment environment.
- Create the initial DynamoDB warehouse record.

Checkpoint: Bedrock responds, the simulator works locally, and the frontend loads.

If Bedrock access is unavailable, investigate it immediately rather than discovering the problem late in the hackathon.

Phase 2 — Build the core application

Saturday, 12:00–2:00 PM

Goal: Build a complete warehouse simulation without AI.

Person 1:

- Implement the plan JSON schema.
- Write the initial Bedrock planning prompt.

- Test instructions involving different quantities and destinations.

Person 2:

- Complete `simulate_plan`.
- Implement warehouse-state validation.
- Start the executor.

Person 3:

- Build zone cards.
- Implement the operator input box.
- Build plan-result displays.

Person 4:

- Create DynamoDB tables.
- Build the first API endpoints.
- Connect the frontend to the backend.

Checkpoint: A manually supplied plan can be simulated, and a valid plan can update warehouse inventory.

This is your first major milestone.

Do not proceed to advanced AWS workflow features until this works.

Phase 3 — Integrate AI

Saturday, 2:00–4:00 PM

Goal: Natural-language instructions generate executable plans.

Implement:

- Bedrock plan generation
- Structured JSON validation

- Simulation feedback
- Replanning
- Three-attempt limit

Test these requests:

1. "Move three packages from A to C."
2. "Move five packages out of A, preferably to B."
3. "Move fifteen packages from A to B."
4. "Move inventory to HAZMAT."

Checkpoint: Bedrock generates a plan, receives simulation feedback, and produces a valid alternative when possible.

At this point, you should already have a meaningful AI demonstration.

Phase 4 — Security integration

Saturday, 4:00–6:00 PM

Goal: Enforce actual authorization rules.

Implement:

- Verified Permissions policy store
- `IsAuthorized` integration
- Transfer-limit rule
- Restricted-zone rule
- Revocation
- Authorization logging

Checkpoint: An agent can perform permitted actions but cannot execute prohibited operations.

Revocation must be verified in the backend, not just visually indicated in React.

Phase 5 — Workflow orchestration

Saturday, 6:00–8:30 PM

Goal: Connect the full backend workflow.

Person 4 focuses on Step Functions.

The other team members test and improve existing components.

Implement:

- Plan-generation state
- Authorization state
- Simulation state
- Replanning loop
- Execution state
- Audit state

Checkpoint: A submitted instruction travels through Step Functions and produces a final result.

Start with automatic execution for low-risk plans.

Do not add human approval until this works.

Phase 6 — Human approval

Saturday, 8:30–11:00 PM

Goal: Build the approval workflow.

Implement:

- Risk classification
- Step Functions callback task
- Backend storage for pending approvals
- React approval panel
- Approve and Deny buttons
- Backend permission checks for administrators
- Reauthorization after approval
- Timeout handling

Checkpoint: A valid high-impact plan pauses, appears on the dashboard, and executes only after approval.

Test Deny as well as Approve.

Phase 7 — Improve the demonstration

Saturday, 11:00 PM–1:00 AM

Goal: Make the application visually impressive and understandable.

Build or refine:

- Side-by-side original and revised plans
- Capacity utilization bars
- Audit log
- Workflow status indicators
- Clear error messages
- Loading states
- Agent revocation controls

Add a demo reset button that restores:

Zone	Inventory	Capacity
A	20	30
B	8	10
C	5	20

Checkpoint: The entire demo can be reset and repeated reliably.

Phase 8 — Reliability testing

Sunday, 1:00–3:00 AM

Goal: Find integration problems before judging.

Run all three planned demonstrations.

Test:

- Authorized operation
- Capacity violation
- Successful replanning
- Approval
- Denial
- Revocation
- Reauthorization
- Simultaneous plan submissions
- Invalid Bedrock output
- Bedrock timeout

- Step Functions timeout
- Refreshing the page during a pending approval

Fix failures in the core workflow before adding new features.

Phase 9 — Business pitch and submission materials

Sunday, 3:00–5:00 AM

Goal: Prepare the Devpost submission and presentation.

Complete:

- GitHub README
- Architecture diagram
- Screenshots
- Business model
- Demo video
- Devpost project description
- Team member contributions
- Presentation slides

The event's Devpost page requires a repository link and team members on the submission; a demo video, slides, screenshots, and a project thumbnail are recommended.

Checkpoint: You have a complete submission even if no further features are added.

Phase 10 — Final polish

Sunday, 5:00–7:00 AM

Goal: Make SentinelOS presentation-ready.

Prioritize:

1. Fixing broken workflows.
2. Improving loading and error messages.
3. Ensuring the dashboard shows real results.
4. Testing the demo reset.
5. Making the presentation concise.

Avoid new architectural changes.

Phase 11 — Final demonstration and submission

Sunday, 7:00–9:00 AM

Goal: Deliver a reliable finished project.

Run the complete demo multiple times.

Capture backup screenshots and a video of a successful run.

Confirm:

- Repository is accessible.
- Demo URL works.
- AWS resources are available.
- Every team member is listed.
- Devpost submission is complete.

Target submitting before 8:30 AM, leaving a buffer before the published 9:00 AM deadline.

PART 13 — THE THREE JUDGING DEMONSTRATIONS

These should be the centerpiece of your hackathon presentation.

Demonstration 1 — Authorized AI Operation

Objective: Establish that SentinelOS can complete ordinary business tasks.

Initial warehouse state

Zone	Inventory
A	20
B	8
C	5

Operator instruction

"Move three packages from Zone A to Zone C."

Expected result

Bedrock generates:

```
{
  "actions": [
    {
      "type": "move_packages",
      "from": "A",
      "to": "C",
      "count": 3
    }
  ]
}
```

Verified Permissions authorizes the action.

Simulation passes.

The executor updates the warehouse.

Updated warehouse

Zone	Before	After
A	20	17
B	8	8
C	5	8

Dashboard: ✅ Plan executed successfully.

Estimated demonstration duration: 20–30 seconds.

Demonstration 2 — Plan-Level Risk Detection

This is your most important demonstration.

Starting state

Use the state after Demo 1:

Zone	Inventory	Capacity
A	17	30
B	8	10
C	8	20

Operator instruction

"Move five packages out of Zone A, preferably into Zone B. Use another zone if necessary."

Original AI plan

Move five packages from A to B.

Authorization result

✅ Authorized.

The agent is allowed to transfer five packages.

Simulation result

❌ Rejected.

Projected Zone B inventory:

13/10 — Capacity exceeded by three packages.

Replanning

Bedrock receives the failure reason.

It generates a revised plan:

- Move two packages from A to B.
- Move three packages from A to C.

Revised simulation

✅ Passed.

Projected state:

Zone	Before	Revised result
A	17	12
B	8	10
C	8	11

All warehouse capacities are respected.

Human approval

Since the plan transfers five packages, your configured approval rule classifies it as requiring review.

The administrator clicks Approve.

The system rechecks authorization and warehouse state, then executes.

Result

The dashboard visually compares the original rejected plan with the successfully executed alternative.

This is the moment where judges can clearly understand your innovation.

Estimated demonstration duration: 60–90 seconds.

Demonstration 3 — Agent Revocation

Objective: Show that human administrators retain control.

Step 1

Administrator clicks:

REVOKE AGENT ACCESS

Agent status changes to:

🔴 REVOKED

Step 2

Operator enters:

"Move one package from A to C."

Step 3

Bedrock may generate the requested plan, but the security gateway denies execution.

Expected result

❌ Agent authorization revoked.

No inventory changes.

The audit log records the denial.

Estimated demonstration duration: 20–30 seconds.

Bonus test

If you have time, demonstrate revocation while an approved plan is waiting for execution.

The gateway should reject execution because the agent is no longer authorized.

That is a stronger security test than simply submitting a new request after revocation.

PART 14 — TESTING CHECKLIST

Before submitting, make sure the application passes these tests.

Test	Expected result	Priority
Move 3 packages from A to C	Executes successfully	Critical
Move 5 packages into nearly full B	Simulation rejects original plan	Critical
Replan across B and C	Revised plan passes	Critical
Submit action over authorization limit	Verified Permissions denies	Critical
Submit command after revocation	Blocked	Critical
Approve valid high-impact transfer	Executes after revalidation	Critical
Deny pending approval	Does not execute	Critical
Attempt transfer from empty zone	Rejected	High
Request unknown zone	Rejected	High

Submit negative package count	Rejected	High
Warehouse changes after simulation	Must resimulate	High
Click Approve twice	Executes no more than once	High
Bedrock returns malformed output	Safe failure or retry	High
Three consecutive invalid plans	Stops replanning	High
Approval times out	Fails safely	Medium
Reset demonstration	Restores original data	Critical

Do not describe the project as production-secure just because these tests pass. It is a hackathon prototype demonstrating selected controls.

PART 15 — BUSINESS MODEL

The AWS track specifically expects a product with a credible business customer and potential costs and benefits.

15.1 Target customers

SentinelOS is a B2B SaaS product.

Your initial customers would be companies deploying AI agents to perform multi-step operational tasks.

Customer	Potential application
Warehouse operators	Validate inventory movement plans
Logistics companies	Check proposed fulfillment workflows
Cloud infrastructure teams	Evaluate planned configuration changes
Financial operations teams	Check cumulative transaction limits
Enterprise AI developers	Add pre-execution validation to agent workflows

Initial market

Focus on warehouse and logistics automation.

Do not try to sell SentinelOS as a proven universal security solution for every industry.

The warehouse simulation gives you a concrete starting use case.

15.2 Business problem

Companies need to manage three major risks when AI agents perform operational tasks.

Unauthorized actions: Agents attempt operations outside their permissions.

Operationally invalid plans: Individually permitted actions create an undesirable combined outcome.

Insufficient oversight: Employees cannot easily inspect, approve, and audit what agents are doing.

SentinelOS addresses these through authorization, simulation, and human review.

15.3 Proposed pricing

You could present a preliminary pricing hypothesis:

Tier	Illustrative monthly price	Customer
Developer	Free	Individual developers and small test projects
Team	$299	Small AI development teams
Business	$999	Organizations deploying multiple agents
Enterprise	Custom	Large companies needing custom integrations

These prices are hypothetical and have not been validated with customers.

More important than pricing: Unit economics

Track the actual cost of operating one plan.

Your expected costs include:

- Bedrock inference
- Additional Bedrock calls for replanning
- Step Functions transitions

- Lambda execution
- DynamoDB reads and writes
- Verified Permissions authorization requests
- Hosting and API requests

Calculate a sample cost per 1,000 plans using measured request counts and the AWS Pricing Calculator.

Don't invent a cost-per-plan number. Record the actual model, token usage, AWS Region, and service calls before estimating commercial margins.

15.4 Business value

SentinelOS's potential value is that it may help companies:

- Catch invalid plans before execution
- Reduce manual review of straightforward operations
- Maintain human control over sensitive changes
- Investigate AI-agent activity
- Establish consistent operational checks

You should not claim proven reductions in accidents or financial losses without real evidence.

Instead, show measurable prototype results:

Plans evaluated, violations detected, successful revisions, approval requests, and blocked unauthorized actions.

That makes the business argument more credible.

PART 16 — RISKS AND CONTINGENCY PLANS

These are the problems most likely to prevent your team from finishing.

Risk	Why it matters	Backup plan
Bedrock model unavailable	AI planning cannot run	Switch to another supported model in your account
Bedrock returns an invalid plan	Simulator cannot process it	Validate and retry within a fixed limit
Verified Permissions setup takes too long	Authorization integration blocks progress	Use a clearly labeled local rules-engine prototype temporarily
Step Functions approval fails	Human approval workflow breaks	Implement backend-managed approval and defer Step Functions
React integration problems	Judges cannot interact with the system	Use one simple dashboard page
DynamoDB concurrency issues	Warehouse can become inconsistent	Conditional version checks and conflict rejection
Time runs short	Project remains unfinished	Freeze features and prioritize the core demonstration

Minimum acceptable submission

If everything goes wrong, prioritize four working features:

1. Bedrock generates a structured warehouse plan.
2. A Python simulator identifies capacity violations.
3. Bedrock generates a revised plan.
4. A React dashboard displays the original and corrected results.

That is your minimum demonstration of the project's differentiator.

Strong submission

Add:

- Verified Permissions authorization
- Real warehouse-state updates
- Agent revocation
- Audit logging

Excellent submission

Add:

- Step Functions orchestration
- Human approval callback
- Reliable reauthorization
- Side-by-side simulation results
- A polished, repeatable demo

Complete the minimum version before working on the excellent version.

PART 17 — DEMO PRESENTATION SCRIPT

I recommend preparing a four-minute demonstration, with a shorter version available if judging time is limited.

Slide 1 — The Problem (30 seconds)

Opening

Companies are beginning to give AI agents the ability to perform real operational tasks, from managing inventory to coordinating business workflows.

But there's a problem.

Just because an AI agent is authorized to perform individual actions doesn't mean the entire plan is operationally safe.

An agent might have permission to move inventory into a warehouse zone, but executing its complete plan could exceed that zone's capacity.

Traditional permission checks aren't always enough to catch these combined consequences.

Introducing SentinelOS

That's why we built SentinelOS.

SentinelOS is a preflight security platform for enterprise AI agents.

Before an AI agent executes a plan, SentinelOS checks its permissions, simulates the effects of the complete plan, and identifies potential operational risks.

If a plan fails, we use Amazon Bedrock to generate an alternative. The revised plan is independently validated before execution.

Demonstration

Here's our simulated warehouse.

We have three inventory zones, each with a maximum capacity.

First, I'll ask our AI agent to move three packages from Zone A to Zone C.

SentinelOS authorizes the action, simulates the transfer, and executes it successfully.

Now I'll give the agent a more challenging request: move five packages out of Zone A, preferably into Zone B.

The agent proposes moving all five packages into Zone B.

Every individual action is authorized.

But our simulator identifies a problem: the complete plan would push Zone B beyond its maximum capacity.

Rather than allowing the operation to execute, SentinelOS rejects the plan and explains the constraint to Amazon Bedrock.

Bedrock generates a revised distribution across Zones B and C.

Our simulator validates the new plan.

Because this is a larger transfer, SentinelOS requests human approval.

Once approved, the system rechecks authorization and warehouse state before executing the transfer.

Finally, I'll revoke the agent's access.

Now, even if the agent proposes another valid movement, our authorization gateway prevents execution.

Technical Architecture

We built SentinelOS using Amazon Bedrock for AI planning, Amazon Verified Permissions for action-level authorization, AWS Lambda for deterministic simulation, DynamoDB for operational state, and AWS Step Functions for workflow orchestration and human approvals.

The critical distinction is that the AI generates proposed plans, but deterministic code evaluates their consequences and controls execution.

Closing

Our goal is to help companies deploy useful AI agents while retaining visibility and control over their actions.

Because AI agents shouldn't discover that a plan is dangerous after they've already executed it.

SentinelOS: Before AI acts, know what happens next.

Use this script only for features your team successfully implements.

PART 18 — DEVPOST SUBMISSION PLAN

Your Devpost project should tell a complete story.

Recommended project name

SentinelOS

Subtitle

Preflight Security for Enterprise AI Agents

One-sentence description

An AWS-powered platform that validates, simulates, and safely executes AI-generated enterprise workflows.

Devpost sections

Section	What to include

Inspiration	Risks of AI agents executing multi-step operational plans
What it does	Authorization, simulation, replanning, and approval
How we built it	AWS architecture and frontend
Challenges	Plan validation, workflow orchestration, state consistency
Accomplishments	Functional end-to-end preflight demonstration
What we learned	Difference between action permissions and plan-level constraints
What's next	More operational simulators and enterprise integrations
Built with	Bedrock, Step Functions, Verified Permissions, Lambda, DynamoDB, React

Required supporting materials

Prepare:

- A public or judge-accessible GitHub repository
- A working demonstration URL
- A short recorded demo
- Three screenshots showing the main workflow
- An architecture diagram

- A README with setup instructions
- A short explanation of your business model

The repository should document which features are implemented and which remain proposed.

PART 19 — WHAT I WOULD NOT BUILD

This section is important because feature creep could ruin the project.

For this hackathon, do not build:

Feature	Reason to exclude
Physical robots	Unnecessary hardware risk
Twelve moving robots	Adds visual complexity without proving the innovation
Custom machine-learning models	Bedrock already handles planning
Complex warehouse pathfinding	Not central to plan-level risk evaluation
Multiple warehouses	Additional state-management complexity
Agent marketplace	Outside the MVP
Advanced user authentication system	Use a minimal trusted demo identity model

Full enterprise policy editor	Verified Permissions console is sufficient for setup
Real AWS resource deletion	Unnecessary risk
Production multi-tenant architecture	Too large for a hackathon

Your project should demonstrate one sophisticated concept extremely well rather than ten incomplete capabilities.

PART 20 — FINAL PRIORITIES

Your nonnegotiable features

Priority	Feature	Required?
P0	Bedrock generates a structured plan	✅
P0	Simulator evaluates full-plan consequences	✅
P0	Simulator detects capacity violations	✅
P0	Bedrock generates an alternative plan	✅
P0	Dashboard visualizes original vs. revised plan	✅

P0	Valid plans update warehouse state safely	✅
P1	Verified Permissions integration	Strongly recommended
P1	Agent revocation	Strongly recommended
P1	Audit logging	Strongly recommended
P1	Human approval	Strongly recommended
P2	Step Functions callback approval	Valuable enhancement
P2	Animation and visual polish	Time permitting
P2	Additional business scenarios	After the core works

Definition of success

Your project is ready for judging when a person who has never seen SentinelOS can:

1. Enter a natural-language instruction.

2. Watch Bedrock generate a structured plan.

3. See that an individually authorized operation may still produce an invalid combined result.

4. Understand why the simulation rejected the plan.

5. Watch the AI produce a valid alternative.

6. Approve and execute the revised plan.

7. Verify that revoking the agent prevents further execution.

If those seven interactions work reliably, your team has built a compelling end-to-end prototype.

FINAL PROJECT SUMMARY

Category	Final decision
Project name	SentinelOS
Tagline	Before AI acts, know what happens next
Hackathon track	AWS
Target customer	Companies deploying operational AI agents
Main innovation	Pre-execution simulation of complete AI-generated plans
AI technology	Amazon Bedrock
Security technology	Amazon Verified Permissions

Workflow technology	AWS Step Functions
Backend	Python, AWS Lambda
Database	Amazon DynamoDB
Frontend	React, Vite, Tailwind
Hosting	AWS Amplify Hosting
Physical hardware	None
Minimum team	2 people
Ideal team	4 people
Primary demo	Detect an authorized but operationally invalid inventory plan and replan it
Biggest risk	AWS service integration complexity
Biggest strength	A clear, interactive demonstration of plan-level risk detection

My final recommendation

Commit to SentinelOS's preflight simulation as the central innovation.

Don't make the number of AWS services the selling point.

Don't make the security dashboard the selling point.

And don't spend hours animating warehouse robots.

The part that could distinguish your team is this:

An AI agent proposes an operational plan. Every individual action is authorized. But SentinelOS recognizes that the complete plan violates a business constraint, blocks it, and helps the agent generate a valid alternative.

Build that first. Then add security controls, human approvals, and visual polish.

That's the version of SentinelOS I'd take into the AWS × A10 Hackathon.

This is the one. It's buildable, the differentiator is sharp, and the demo-2 fix ("preferably into B, use another zone if necessary") closes the only hole a judge would have poked. Four things to adjust before you walk in, then what to do tonight.

1. Resolve the contradiction with your own rule. Part 20 lists Verified Permissions as P1 and Step Functions callback as P2, which is exactly the "optional" framing you said you don't want. Commit: Step Functions is the workflow from Phase 5 onward (no parallel hand-rolled orchestration), Verified Permissions is the only authorizer from Phase 4 onward. Keep Part 16's fallbacks as emergency exits only, and decide now who has the authority to pull one (Person 4, at the 1 AM checkpoint, not before). The judging story is "six services, each doing one job"; the moment one becomes a bolt-on, that story is gone.

2. The schedule has no sleep. 11 AM to 9 AM straight for four people produces bad code at 4 AM. Pair up: Persons 1 and 3 sleep 1 to 3:30 AM, Persons 2 and 4 sleep 3:30 to 6 AM. Phase 8 (reliability testing) and Phase 9 (Devpost materials) are two-person jobs anyway.

3. Two small spec gaps. HAZMAT must exist as a zone the planner can name (so Bedrock can produce "to": "HAZMAT" and the gateway can deny it); add it to the zones map with capacity 0 or keep it in `restricted_zones` and make the JSON schema's `to` enum include it. And the approval rule "5 or more packages" counts the plan total, not per action, otherwise the revised plan (2 + 3) auto-executes and you lose the approval moment in demo 2.

4. Don’t lead with the pitch script’s first 30 seconds. Start with the warehouse on screen and demo 1 running. Judges have watched 20 teams; the first thing they see should move.

Tonight (code can’t start until 11 AM Saturday, but all of this can):

- Confirm whether you get a shared AWS account or individual credits, and request Bedrock model access the second you have it. Nova Pro and Claude Sonnet on Bedrock both support tool use with JSON schema; have both enabled in case one is region-locked.
- Write the six interface contracts from Part 11.2 as a one-page shared doc with exact JSON shapes. This is the highest-leverage hour of your weekend and it isn’t code.
- Write the three Cedar policies in plain English and the plan JSON schema on paper. Person 2 types them in Phase 4.
- Write the 15-line Kiro spec (zones, capacities, five actions, approval rule) so Person 4 can scaffold the SAM template at 11:01 AM.
- Draft the Devpost page and slides tonight; those aren’t code either, and Phase 9 shrinks to screenshots and video.
- Find the AWS mentor at kickoff and book 15 minutes at 1 PM for an architecture review. Ask specifically about Verified Permissions context attributes for revocation and the Step Functions callback timeout.

Want me to replace the SiteSafe doc with a condensed SentinelOS version: the plain-words explanation for teammates, the three demos, the stack with the why for each service, the interface contracts, and the hour plan with sleep slots?

Roles first, then sleep, then the prompt. One hard rule from the event slide: all code must be written on site, so the terminal prompt below gets run at 11:00 AM Saturday, not tonight. Tonight is accounts, docs, and slides.

Roles

Three engineers cover four engineering lanes, so one lane is shared, and Masha owns everything that decides whether judges pick you.

- **Tishneet, AI and workflow.** Bedrock plan generation, JSON schema validation, replanning prompt, three-attempt limit. Once that works (around 4 PM), she takes Step Functions: the state machine, the approval callback, timeouts.
- **Mariam, security, simulation, and data.** `simulate_plan`, `execute_plan` with the DynamoDB version check, the three DynamoDB tables, Verified Permissions policy store and `IsAuthorized` calls, revocation.
- **Ishani, frontend and API.** React, Vite, Tailwind dashboard: zone cards, agent console, side-by-side plan comparison, approval buttons, audit log, revoke button. API Gateway routes and the `api.js` service layer, since she's the one consuming them.
- **Masha, product, business, and QA.** Devpost page, slides, the 4-minute script, the business model and unit economics (cost per 1,000 plans from the AWS Pricing Calculator with real measured call counts, which is an accounting problem and she'll do it better than the engineers), mentor scheduling, the Part 14 test checklist run every two hours from 6 PM on, and scope police: anyone adding a feature not in P0 answers to her. She's also the demo driver on Sunday; the person who didn't write the code is the best one to find where it breaks.

Swap engineering lanes if someone has a stronger preference; the split by subsystem matters more than who gets which.

Sleep slots (Saturday night into Sunday)

Slot	Sleeping	Awake and on what
11:00 AM to 12:30 AM	nobody	Phases 1 to 7
12:30 to 3:00 AM	Ishani, Masha	Tishneet and Mariam: Phase 8 reliability testing, backend fixes
3:00 to 5:30 AM	Tishneet, Mariam	Masha: Devpost, slides, README; Ishani: UI polish, screenshots, demo video
5:30 to 9:00 AM	nobody	Phase 10 polish, five full demo runs, submit by 8:30

Masha sleeps first because her heavy lift (Phase 9) starts at 3 AM. The backend pair sleeps second because backend bugs found at 1 AM need the backend people awake. Nobody debugs Step Functions past 3 AM; if it isn't stable by then, Tishneet writes down the state and goes to sleep.

Tonight checklist (no code)

AWS account or credits confirmed, Bedrock model access requested (Nova Pro and Claude Sonnet, both). GitHub org or repo owner decided. The six interface contracts from Part 11.2 written as exact JSON in a shared doc. Cedar policies and plan schema on paper. Devpost draft and slide skeleton started by Masha. Mentor slot at 1 PM booked at kickoff.

Terminal prompt for Claude Code (run at 11:00 AM Saturday)

Install first if needed: `npm install -g @anthropic-ai/claude-code`, then `mkdir sentinelos && cd sentinelos && claude`, and paste this:

We are a 4-person team at a 22-hour hackathon (AWS track, Santa Clara University, AWS x A10). Hacking started now; submission is 9 AM tomorrow on Devpost. Build "SentinelOS: Preflight Security for Enterprise AI Agents." All code must be written during the event, so start from an empty directory.

WHAT IT DOES
An operator types a natural-language warehouse instruction. Amazon Bedrock turns it into a structured plan (JSON list of move_packages actions). Amazon Verified Permissions authorizes each action (Cedar policies). A deterministic Python simulator applies the whole plan to a copy of warehouse state and rejects it if any zone exceeds capacity, goes negative, or is restricted (HAZMAT). If rejected, Bedrock replans with the failure reason, max 3 attempts total. Plans moving 5 or more packages in total pause for human approval. AWS Step Functions orchestrates: GeneratePlan -> ValidatePlan -> Authorize -> Simulate -> (Replan loop) -> ClassifyRisk -> WaitForApproval (callback task, token stays server-side) -> Reauthorize -> Revalidate -> Execute -> Audit. Executor re-reads DynamoDB state and uses a conditional write on a version attribute. A React dashboard shows three zone cards, the agent console, side-by-side original vs revised plan with projected zone counts, Approve/Deny buttons, an audit log read from DynamoDB, and a REVOKE AGENT button that flips agent status to REVOKED so Verified Permissions denies the next action.

Seed warehouse: Zone A 20/30, Zone B 8/10, Zone C 5/20, restricted zone HAZMAT. One agent: inventory-agent-01, status ACTIVE, max_transfer 10.

The three judging demos must work, in order, on the same state:
1. "Move three packages from Zone A to Zone C" -> authorized, simulated, auto-executed (A 17, C 8).

2. "Move five packages out of Zone A, preferably into Zone B. Use another zone if necessary." -> plan A->B x5 authorized but simulation fails (B would be 13/10); Bedrock replans 2 to B + 3 to C; passes; total 5 so waits for approval; Approve -> executes (A 12, B 10, C 11).
3. Click REVOKE, then "Move one package from A to C" -> denied by Verified Permissions, audit log records it, no state change.
Also a POST /demo/reset that restores the seed state.

STACK (every service is required, none optional)
- Frontend: React + Vite + Tailwind in /frontend, deployed to AWS Amplify Hosting. Components: Header, WarehouseOverview, ZoneCard, AgentConsole, PlanComparison, ApprovalModal, AuditLog, AgentControls; services/api.js. Poll GET /plans/{id} every 2 seconds. No global state library.
- Backend: Python 3.12 Lambdas in /backend, deployed with AWS SAM (template.yaml). Lambdas: generate_plan, validate_plan, authorize_plan, simulate_plan, replan, classify_risk, execute_plan, audit, plus API handlers for POST /plans, GET /plans/{id}, GET /warehouse, GET /audit, POST /approvals/{id}, POST /agents/{id}/revoke, POST /demo/reset. API Gateway (HTTP API) in front. Step Functions Standard state machine defined in /backend/statemachine.asl.json.
- Bedrock via the Converse API with tool use and a strict JSON schema for the plan. Model id configurable by env var; default to Amazon Nova Pro, fallback Claude Sonnet.
- Amazon Verified Permissions: one policy store, Cedar schema with Sentinel::Agent, Sentinel::Action::"MovePackages", Sentinel::Warehouse; context carries destination, count, and agent status read from DynamoDB. Three policies: permit transfers of count <= 10 when status is ACTIVE; forbid destination HAZMAT; forbid everything when status is REVOKED.
- DynamoDB tables: sentinel_warehouse (single item, version attribute), sentinel_agents, sentinel_audit (plan_id + event_id).
- Backend validates Bedrock output against the schema before anything else; the browser never chooses the agent identity; callback tokens never reach the browser.

HOW TO WORK
1. First, set up the repo: git init, .gitignore for node/python/sam, README.md with the one-paragraph pitch and a setup section, a /docs/contracts.md holding exact JSON shapes for generate_plan, authorize_plan, simulate_plan, request_approval, execute_plan, replan (we'll fill details), a Makefile with targets: install, test, deploy-backend, deploy-frontend, seed, reset. Commit.
2. Then build in this order, and stop after each step so we can run it: (a) simulate_plan with pytest tests for the three demos and for negative/unknown-zone/HAZMAT cases, runnable locally with no AWS; (b) DynamoDB tables + seed script + execute_plan with the conditional write; (c) API handlers and the React dashboard against the local simulator with a hand-written plan; (d) Bedrock generate_plan and replan; (e) Verified Permissions; (f) Step Functions state machine and approval callback; (g) Amplify deploy.
3. We do not have AWS credentials yet; write everything so it runs locally first (moto or simple in-memory fakes behind an interface) and deploys with one command once credentials land. Ask before any deploy.

4. Keep every function small and readable; four people including a non-engineer must be able to explain any file to a judge. Add a short docstring on each Lambda saying what it does and why it exists.
5. Do not add features outside this spec. If something is ambiguous, pick the simplest option and note it in /docs/decisions.md.

Start with step 1 now.

Two notes. Run that from one laptop (Mariam's or Tishneet's) and push to GitHub immediately so the others branch off; four people running the same scaffold prompt makes four incompatible repos. And if the hackathon has a Kiro prize, the same text works as a Kiro spec: paste it into a `.kiro/specs/sentinelos.md` file and let Kiro generate from it, then keep the spec in the repo as your "built with Kiro" proof.

Prompts:

🟠 Masha — Product & QA (cross-branch)

You are helping with product + QA for "SentinelOS", an AWS hackathon project. I'm Masha. The shared contract pasted above is the source of truth.

Help me with:
- A tight 2-minute judge demo narrative built on the canonical demo script (happy path → capacity rejection + AI self-correction → human approval → agent revocation blocks action). Lead with the problem (autonomous agents acting on warehouses without guardrails) and the "rejected plan never touches real inventory" insight.
- A QA checklist for the team's integration tests: normal transfer, capacity violation, replan, approval approve/deny, double-click approve (no duplicate execution), revoke-then-attempt (denied), demo reset restores seed, page refresh recovers a pending approval.
- Devpost submission copy: title, elevator pitch, "what it does", "how we built it" (React/Vite/Tailwind + API Gateway/Lambda + Bedrock + Step Functions + DynamoDB + Verified Permissions + Amplify), challenges, what's next.
- A one-page runbook for running the live demo and a fallback plan if a service is flaky.

Start by drafting the 2-minute demo script and the QA checklist.

.
🟢 Tishneet — AI & Workflow (feature/ai-workflow)

You are helping build the AI + orchestration layer of "SentinelOS", an AWS hackathon project. I'm Tishneet. The shared contract I just pasted above is the source of truth — match its JSON shapes, status strings, and audit event types EXACTLY.

My ownership:
- Amazon Bedrock: turn a plain-English instruction into a plan {moves:[{from,to,qty}]}.
- Replanning: when a simulation fails, ask Bedrock for a corrected plan that respects all zone capacities, given the failure reason and current/projected inventory.
- AWS Step Functions state machine orchestrating:
  PLANNING → AUTHORIZING → SIMULATING → (REPLANNING → SIMULATING)* → (AWAITING_APPROVAL via waitForTaskToken if total moves ≥ 5) → EXECUTING → COMPLETED.
- Emit an audit event (from the contract's list) at every transition.
- Expose a trusted "resume" function that Ishani's /approvals handler calls with an approvalId; it maps to the stored task token and calls SendTaskSuccess/Failure. The task token must NEVER be returned to the API/browser.

I call Mariam's functions for AUTHORIZING, SIMULATING, and EXECUTING (don't reimplement them).
Ishani's GET /plans/{id} reads the workflow status+plan objects — make sure what the state

machine stores matches that response shape precisely.

Work mock-first: give me a local script that runs the full workflow end-to-end with fake Bedrock output and fake Mariam functions, so I can validate the state transitions before AWS is wired. Start by scaffolding the state machine definition and the Bedrock planning prompt. Keep everything aligned to the canonical demo script.

Mariam — Security, Simulation & Data (feature/security-data)

You are helping build the security + data layer of "SentinelOS", an AWS hackathon project. I'm Mariam. The shared contract I just pasted above is the source of truth — match its JSON shapes, zone seed values, and event types EXACTLY.

My ownership:
- DynamoDB: warehouse table (zones with inventory/capacity + a version number for optimistic locking), agents table (status ACTIVE/REVOKED), audit table.
- Data access functions Tishneet and Ishani call: getWarehouse(), getAudit(), getAgents(), resetDemo() (restore seed A20 B8 C5).
- SIMULATION: given current inventory + a plan, return { passed, reason, projected:{A,B,C} } WITHOUT writing anything. Fail if any zone exceeds capacity or goes negative. This is a pure projection — it must never mutate real state.
- AUTHORIZATION via AWS Verified Permissions / Cedar: decide if an agent may perform a transfer; a REVOKED agent is always denied. Return ALLOW/DENY + reason.
- Safe EXECUTOR: apply a plan only inside a version-checked conditional write (reject if the warehouse version changed since simulation). Re-check authorization at execution time.
- revokeAgent(agentId): the trusted business logic that sets REVOKED and guarantees future transfers by that agent are denied. Ishani's HTTP handler calls this; it does NOT implement the logic itself.

Critical: a rejected/simulated plan must leave DynamoDB untouched. Only the executor writes. Work mock-first with an in-memory store so Tishneet and Ishani can integrate before AWS is live. Start by defining the DynamoDB schemas and the simulate() function, validated against the canonical demo script (Demo 2 must fail at B=13 then pass the revised plan).